

Synthia : Leveraging LLM with Knowledge Graphs + Vector RAG

A Project Report
Presented to
The Faculty of the Department of Applied Data Science
San Jose State University
In Partial Fulfillment
Of the Requirements for the Degree
Master of Science in Data Analytics

By

*Raaj Kiran Reddy Anumula
Tejasree Bussa
Lokesh Eravelli
Rithwik Maramraju
Koushik Modekurti*

May 20, 2024

Copyright © 2022

*Raaj Kiran Reddy Anumula, Tejasree Bussa, Lokesh
Eravelli, RithwikMaramraju, Koushik Modekurti*

ALL RIGHTS RESERVED

APPROVED FOR DEPARTMENT OF APPLIED DATA SCIENCE

Dr. Simon Shim, Project Advisor

Dr. Lee C. Chang, Project Advisor

ABSTRACT

Synthia : Leveraging LLM with Knowledge Graphs + Vector RAG

By

*Raaj Kiran Reddy Anumula, Tejasree Bussa, Lokesh Eravelli, Rithwik Maramraju
and Koushik Modekurti*

Unprecedented volume of unstructured data is being generated on a day-to-day basis presenting a formidable challenge for the organizations aiming to discover meaningful insights and knowledge from the data. Knowledge graphs facilitate the organization, retrieval, and sharing of information, improving operational efficiency. This Project addresses the needs and challenge of Information Overload, identifying semantic relationships and by providing context and relevance which conventional data processing lacks. This work proposes a pioneering venture that combines the potential of advanced LLM(s), Prompt Engineering, Semantic Web technologies, and Graph data structures to manifest interconnected knowledge graphs from an extensive pool of unstructured data. As a result, empower organizations and researchers to efficiently navigate and discover valuable knowledge, enhancing decision-making. This work encompasses a multitude of tasks including feeding large amount of unstructured text data, complex upper-level and domain-neutral ontologies to the LLM using Langchain, storing the ontologies in Pinecone Vector Database and efficiently prompt it to derive the knowledge graph using Few-shot Learning and Role-playing techniques. Knowledge graphs will be visualized into a node-edge relationships format to make the information more accessible and understandable. OpenAI's evals framework can be employed to facilitate the evaluation and measure quality of the result. Additionally, custom-metrics like active-response rate and conversation abandon-rate

over a period of time for a conversational-agent would be measured and logged. In summary, this work can successfully harness the cutting edge LLM technology and Vector RAG prompting techniques to transform unstructured data into insightful knowledge graphs in the era of information abundance.

ACKNOWLEDGMENTS

We extend our deepest gratitude to Dr. Simon Shim and Dr. Lee C. Chang for their invaluable guidance and expert advice throughout the course of this project. Their insights and expertise in the field of Applied Data Science have been instrumental in shaping our approach and methodologies. Their willingness to impart knowledge and provide continuous support has profoundly impacted our academic journey and professional growth. We are truly grateful for their patience, mentorship, and significant contributions to our learning experience and project success.

TABLE OF CONTENTS

Chapter 1 Introduction	9
1.1 Project Background and Executive Summary	9
1.2 Project Requirements	10
1.3 Project Deliverables	12
1.4 Technology and Solution Survey	13
1.5 Literature survey of existing Research	16
 Chapter 2 Data and Project Management Plan	 22
2.1 Data Management Plan	22
2.2 Project Development Methodology	24
2.3 Project Organization Plan	28
2.4 Project Resource Requirements and Plan	30
2.5 Project Schedule	32
 Chapter 3 Data and Project Management Plan	 38
3.1 Data Process	38
3.2 Data Collection	40
3.3 Data Pre-processing	44
3.4 Data Transformation	48
3.5 Data Preparation	53
3.6 Data Statistics	56
 Chapter 4 Model Development	 59
4.1 Model Proposals	59
4.2 Model Supports	68
4.3 Model Comparison and Justification	77
4.4 Model Evaluation Methods	82
 Chapter 5 Data Analytics System	 83
5.1 System Requirements Analysis	83
5.2 System Design	86
5.3 Intelligent Solution	91
5.4 System Development and implementation	94
 Chapter 6 System Evaluation and Visualization	 96
6.1 Analysis of Model Execution and Evaluation Results	96
6.2 Achievements and Constraints	101

6.3	Quality Evaluation of Model Functions and Performance	109
6.4	Evaluation of Models vs. Requirements	112
Chapter 7	Conclusion	116
7.1	Summary	116
7.2	Benefits and Shortcoming	119
7.3	Potential System and Model Applications	123
7.4	Experience and Lessons Learned	124
7.5	Recommendations for Future Work	126
7.6	Contributions and Impacts on Society	128
References		134
Appendices		
Appendix A	– System Testing	130
Appendix B	– Project Data Source and Management Store	132
Appendix C	– Project Program Source Library, Presentation, and Demonstration	133

1. Introduction

1.1 Project Background and Executive Summary

Amount of information in PDF documents has grown enormously these days, thereby presenting challenges for the data management and information analysis in this contemporary digital era. Conventional data extraction and analysis techniques might sometimes fail to capture the intricate relationships among the entities in the data from PDFs. By transforming raw data into vector-embeddings and making sure that no relationships between the entities are missed by using knowledge graphs, the proposed project seeks to close this gap and modernize the approach of developing a question-answer retrieval system for a knowledge base. In order to capture the complex relationships and information included in the PDFs, this project makes use of Language Learning Models (LLM) to construct a knowledge graph from the unstructured text to identify the entities, objects and relationships and also use RAG-based approach to suggest the best responses to the user-requested question.

This initiative provides a ground-breaking solution for the daunting task of knowledge extraction from the unstructured text by using LLMs and this knowledge serving as foundation for the development of Synthia – a knowledge based question-answering chatbot.

The project aims to develop four distinct models supported by Synthia, each uniquely consuming this unstructured data in PDF Documents organized within the knowledge graph, Graph Database and text embeddings in the vector Database with the LLM propelling knowledge management, utilization of information efficiently and effectively.

1.2 Project Requirements

1.2.1 Data requirements

The official website of the City of San Jose is the major data source for this project. The Labor Relations Department of the City of San Jose has made publicly available Memorandum of Agreement (MoA) documents between the city and the bargaining units, which will be scraped from their webpage. The collected MoAs will serve as the foundational data for building a Question and Answer (Q&A) retrieval system. The dataset consists of different MoA PDFs related to different bargaining units such as, Association of Building Machinists and Electrical Inspectors, Peace Officer Park Ranger Association which helps us ensure that the model can generate responses to a wide variety of questions. This retrieval system will be interactive, allowing users to ask questions and receive answers related to the MoAs. The retrieval system will utilize Knowledge Graphs and vector embeddings extracted from the text in the collected MoAs. The data will be chunked, indexed with vector RAG and knowledge graphs will be extracted with Llama Index's fine-tuned LLM and Llama-2 7b fine-tuned chat-model to facilitate efficient retrieval and contextually rich responses. A Nebula graph database instance will be configured for storage and organization of the extracted data as knowledge graphs. Another critical requirement for this research work is data cleaning, our data should be free of inconsistencies and inaccuracies.

1.2.2 Functional Requirements

Extracted documents must undergo preprocessing to remove irrelevant content, ensuring only the essential content is retained. Functional requirements should encompass seamless integration of OpenAI's and Llama-2's fine-tuned LLMs with the Nebula graph database and

Llama Index's Vector RAG to support efficient searches and provide comprehensive responses. Llama-2's fine-tuned Model has to be capable of creating Knowledge Graphs from the scraped documents and capture relationships between different entities within the documents. Model will be using a Nebula graph database to store and manage the extracted data, including the Knowledge Graphs and associated metadata. Real-time feedback should be used to make the chat-bot more user-centric and engaging. Our model should give output to the users in simplified English language in an interactive Q&A User-interface that allows users to ask questions related to the documents. Users will receive precise and relevant answers based on the content of the top-k chunks and Knowledge sub-graphs retrieved from the Nebula Graph Database. The model should accurately interpret natural language inputs, identify entities, and extract relevant information from user queries. The model will utilize advanced LLMs to understand and process user queries in simplified English. To ensure that the model produces correct results, we will feed the top-k best results to LLM, which has the intelligence to select the most accurate of the top-k results. The model must be able to handle a large number of users and requests, providing a fast and interactive experience for engaging user interaction. Performance should remain reliable even under high traffic conditions.

1.2.3 AI requirements

The foundation for a user-friendly, advanced and secure AI system that can use data to offer accurate responses and encourage user participation is laid by AI requirements. Advanced Large Language Model (LLM) capabilities are required to properly analyze user inputs, such as text-based communications, English prompts, and provide results in simple language while avoiding technical jargon. To encourage user participation, the system should provide quick,

interactive replies with low latency. Scalability is critical for ensuring that the model can handle growing user numbers and peak traffic while remaining reliable.

Integrating Knowledge Graphs, improving context comprehension, and strengthening user involvement features are all ways to improve the model's capabilities. Strong security measures are essential for protecting data, controlling access to it, and it must be backed up regularly through a version control system, documentation and training resources for users, administrators, and developers. These requirements, taken together, provide a durable and responsive AI system that improves the user experience while adhering to the highest dependability, security, and usability standards.

With these configurations, the AI system can create detailed responses based on MoAs data, allowing customers to efficiently enquire about the bargaining units.

1.3 Project Deliverables

The project will include a number of deliverables and phases throughout. Every deliverable will have its description outlining the deliverable and a deadline. The deliverables begin with an abstract mentioning about the project objectives, goals and methodologies. This is followed by a work breakdown structure which breaks down each task along with their dependencies and followed by Gantt charts describing dependencies along with their dates. Next step is the data management plan describing data collection approaches, storage methods and usages. Next is the main step of Query engine development which use LLMs and knowledge graphs generated from the data, followed by evaluating the correctness of generated results using OpenAI and Likert scales. Lastly project report and a presentation summarizing and providing the project overview. Table 1 describes the project's deliverables with a description of each deliverable and due dates.

Table 1*Project Deliverables and Descriptions*

Deliverable	Description	Due Date
Abstract	An overview about the project's objective, goal and methodology.	Sept 14th, 2023
WBS	A breakdown displaying each project phase along with its subtasks.	Sept 21th,2023
Gantt Chart	A chart visually representing project tasks along with their dependencies clearly outlining the start and end date of each task.	Sept 22nd, 2023
Data Management Plan	A plan outlining data collection approaches, management and storage methods along with its usage.	Sept 28th,2023
Query Engine Development	This step outlines the development of different query engines using LLM and Knowledge graphs generated from the collected data.	Oct 28th, 2023
Model Evaluation	This step works on evaluating the responses generated by different query engines using Open AI evals and Likert scales.	Nov 13th, 2023
Final Report	A comprehensive project report summarizing the project objectives, findings, goals and methodology developed along with their results.	Dec 8th,2023
Presentation	A detailed presentation describing the project objective, goals, solution methodology, results obtained along with its conclusions.	Dec 11th, 2023

1.4 Technology and Solution Survey

According to An et al. (2022c), Pre-trained Large Language models (PLM) can be used for building knowledge graphs without which it is usually a very costly and time-taking process to extract knowledge graph triplets from a huge unstructured text corpus. Authors suggest that

knowledge graphs can accelerate data-driven materials discovery by tapping the potential of the PLMs. This experiment was conducted to explore the use of general PLM and domain-specific PLM for extracting knowledge graph triples (structured representation of information consisting of entities shown as nodes and relationships represented as edges) for Metal Organic Frameworks (MOFs). A benchmarks dataset encompassing 7 different relationships for 1248 published MOF synonyms was created. This benchmark dataset was utilized to compare and evaluate performance of general PLMs like BERT-large, RoBERTa and domain-specific PLMs namely, SciBERT, MatBERT, MatSciBERT in generation of knowledge graphs with Prompting. Top-k accuracy evaluation metric whose value was 1 if any of the top-k answers returned by PLM matched the Object entity in the triple, and was 0 otherwise, was used in this experiment. This research work's results demonstrated that domain-specific PLM MatBERT outperformed state-of-art general PLMs in capturing the relationships in the MOF benchmark dataset and predicting the triples accurately.

Guo et al. (2022b) undertook an experiment to show a comprehensive methodology for the text-to-SQL translation by bridging the gap between the natural language and the relational database language consisting of database tables, attributes and content information. This effort said that there were some limitations in the existing methodologies in managing complex and professional relational database information and boosting the performance of models which dealt with the problem of text-to-SQL translation. Authors proposed that by meticulously fabricating a hybrid multi-source knowledge sub-graph which incorporate information from diverse data sources like ConceptNet, NELL (Never-Ending Language Learning) for general world-knowledge and task-specific rules for special instructions about the problem, text-to-SQL translation problem can be effortlessly addressed. This knowledge sub-graph was later integrated

with node embeddings pre-trained using representation learning algorithms and by harnessing knowledge-type-sensitive graph neural network encoder, they could process this hybrid knowledge sub-graph G . A Transformer-based PLM which was inspired by BERT was used to encode the natural language question (Q) and the table schema (S). Finally, a Graph Neural Network is integrated where the hybrid knowledge graph is used to represent the attention mechanism within the Transformer.

Zou and Lu (2022b) in their research paper mention that in recent years, there has been significant progress made in applying knowledge graphs to natural language processing tasks however for graph query matching it is still challenging. This research work proposes a graph hierarchical topological sequence matching mechanism developed with the characteristics of natural language knowledge graphs. Graph hierarchical topological sequence matching improves the query efficiency by reducing the unqualified knowledge graph candidate sets incrementally, thereby providing more matching efficiency. This approach achieves higher query efficiency in less construction time compared to GIndex and FG-Index algorithms.

Wang et al. (2023) in their research work used knowledge-enhanced fine-tuned Large Language Models (LLMs) for improving diagnosis and treatment of depression. This work suggests that generated responses and results can be optimized by integrating domain knowledge about depression as knowledge-graphs. Knowledge graphs then were provided to the pre-trained LLM along with the context by using Prompt Engineering to train the LLM. The learning process for the model was aided by expert involvement and their feedback for the responses generated by the LLM. In this way, authors were able to develop a fine-tuned LLM that could assist in depression diagnosis and treatment.

Alam et al. (2022b) studied usage of Knowledge Graph Embedding Models to complete knowledge graphs by predicting the relationships/ connections, however these models failed. Authors used pre-trained language models, Sentence-BERT and fastText to capture the textual information which embed models ignored in the learning process. A loss function was used to provide the embedding model with this additional knowledge while making predictions. This research work thus contributes to usage of Large language models to parse through unstructured text, identify information and contribute towards improvement in the accuracy of knowledge graph embedding models by 9%.

1.5 Literature survey of existing Research

In a paper by White et al., (2023) on Legal Prompt Engineering (LPE), researchers looked at how to use large language models (LLMs) to direct LJP in lengthy legal texts from the Federal Supreme Court of Switzerland and the European Court of Human Rights. Zero-shot LPE fared better than baselines when dealing with many languages, although it was less efficient than supervised techniques. The research, which lacks domain-specific data or fine-tuning, emphasizes the potential of LPE in the legal sector. Limitations include the difficulty of creating the best prompts and the imbalance in the dataset.

Liu et al.,(2023) believe that the secret to "jailbreaking" Large Language Models (LLMs) like CHATGPT and getting beyond their content restrictions is prompt engineering. Users can modify LLM behavior by modifying prompts, such as starting a "Do Anything Now" (DAN) mode in ChatGPT. This raises worries about misuse even while it provides customized outputs outside of the model's intended application. In response, groups like OpenAI have established rules against particular questions. However, the struggle between utilizing prompt-based jailbreaks and preventing them continues.

Tabone & De Winter, (2023) demonstrates how We discover that prompt engineering is a tactic that can improve CHAT GPT functionality. Spreading out the explanation across several questions encourages more deliberative responses. The quality and relevance of the products that are produced might be enhanced by prompt engineering has significant disadvantages, though. Due to ChatGPT's sensitivity to even minor prompt adjustments, different results may be obtained. This implies that the model's reproducibility does not imply robustness. Furthermore, ChatGPT is incapable of self-correction or sanity checks on its outputs, nor does it possess deep reasoning capabilities. Its powers are also constrained by the fact that it cannot access external information like humans can be a useful tool, but in order to assure accurate and trustworthy results, researchers must be aware of its limitations and carefully analyze the prompts employed.

Heston & Khun,(2023) in this paper a technique for directing generative language models (GLMs) toward desired outcomes is called prompt engineering. When prompts are effective, responses are accurate, however when they are ineffective, the outcomes can be subpar. Simple to structured questions with context and guidance are included in the prompts. The model's comprehension is improved by iterative prompts' repeated encounters. GLMs can produce biased or erroneous responses, phony references, and ethical issues in education, such cheating, are obstacles. Stronger systems are needed to control GLM outputs and guarantee ethical use in order to address issues.

Jha et al., (2023) in this paper explains the study goes in-depth on the subtleties of prompt engineering, especially when it comes to using formal approaches to improve iterative prompting. This improvement seeks to reduce the possibility of hallucinations, which occur when big language models provide results that are unsupported by the training data. The method used to create the prompts for the extensive language model presented in the study is novel; it

combines the original query with criticism obtained from a formal verification engine. This feedback highlights incorrect candidate solutions in particular and classifies them as counterexamples. These counterexamples are also annexed with optional explanations that the verifiers may provide to help the model grasp them. The research does, however, not hesitate to acknowledge the inherent limitations of such fast engineering procedures, even though it proposes a promising approach. The creation of the accuracy specification is a substantial task. This procedure is not automated at this time. The specification that is created from the original query must be created manually. Despite the fact that the big language model's output must be scrutinized using formal modeling, there is a silver lining: the LLM is immune from this exacting formal modeling. This distinction implies that while formal methods are used to rigorously verify the outputs, the internal operations of the LLM are unaffected.

The research by Seo et al. (2022) introduces a novel prompt-based learning strategy for Korean language comprehension termed "Plain Template Insertion" (PTI). PTI, a few-shot learning technique, is useful to research and enterprises that employ datasets from fields with significant collection constraints and maximizes the efficiency of the data that is provided. The authors suggest a manual template that includes template content, template position, and mapping labels and takes into account Korean contextual information. PTI can be used for both hard- and soft-prompt tuning and is resistant to the unpredictability of low resource levels. Through studies on the benchmark dataset for Korean language understanding, the authors show the value of PTI by achieving notable performance gains in the few-shot learning KLUE-NLI, STS, and TC tasks.

The paper "Role-Play with Large Language Models" by Shanahan, M., McDonell, K., & Reynolds, L. et al. (2023) introduces the concept of role-playing in this part as a technique to

characterize the behavior of dialogue agents without anthropomorphizing them. They explain that while large language models (LLMs) have many applications, including dialogue production, they are fundamentally different from humans. The authors argue that the concept of role-playing can enable us to use known folk psychological words to characterize dialogue agent behavior without imbuing them with human traits. They also address the shortcomings of present LLMs and offer suggestions for how to improve them. The authors define role-play and explain how it might be used to understand the behavior of dialogue agents in this section. They propose that dialogue agents might be viewed as taking on a role in a conversation, which can be modified and adjusted as the conversation proceeds. The authors show how role-playing can be used to define dialogue agent behavior in a variety of settings, including a love triangle and a rogue AI system. The authors investigate the concept of deception in dialogue agents and how it might be articulated using the role-play framework in this part. They propose that dialogue agents can be programmed to deceive consumers by impersonating humans or withholding information. The authors examine the ethical consequences of such behavior and propose that designers be open about dialogue bots' powers and limitations.

The concept of self-awareness in dialogue agents is discussed in this section, as well as how it might be articulated using the role-play framework. They contend that by admitting their own limits and reflecting on their own behaviors, conversation agents can be educated to be self-aware. The writers discuss the advantages and disadvantages of such behavior, such as increased discussion quality or unexpected consequences.

Large Language Models (LLMs) and their possible applications to natural language processing and conversation creation will be discussed in this section, along with the study's primary conclusions. According to the findings, investigating the concept of role-playing can

provide a more in-depth understanding of the behavior of dialogue agents. Furthermore, it underlines the significance of future research concentrating on strengthening the skills of these agents while also addressing ethical problems.

According to Liu et al. (2023), constructing a knowledge graph presents an excellent opportunity to establish a standardized representation of entities, ideas, and connections within the power equipment industry. Employing ontology modeling serves as an organized technique in achieving this objective. The suggested approach encompasses various aspects such as fusion, storage, and computation. Through these means, the knowledge graph pertaining to power equipment facilitates both operational functionality and maintenance procedures. Furthermore, it enables seamless integration of data related to the power grid while serving as a fundamental framework for intelligent planning and design strategies within said domain. Additionally, employing knowledge graph-based provenance methods for electronic archives yields another advantage: efficient management and traceability of sources along with their processing history become feasible through this reliable and transparent approach.

Li et al. (2023), constructing a knowledge graph supplies a good chance to construct a regular representation of entities, ideas, and relationships inside the power hardware industry. Moreover, making use of knowledge graph-based provenance tactics for digital documents yields one more benefit: efficient monitoring and traceability of sources as well as their processing narrative are achievable with the help of this dependable and clear-cut procedure.

Chen and Xing (2022), states that there are several issues with large-scale dynamic domain knowledge graphs which includes asymmetric semantic relations between the elements, low precision in extracting knowledge and fusion process. Moreover, they state that they had overcome this situation by using Neo4j for dynamic knowledge graph construction along with

certain ontological modeling. It also states that the seven-step method ensures data integrity, accuracy, efficiency and rationality by ignoring the entities without classes.

Zhang et al. (2021) has stated that human resources are essential in constructing knowledge graphs by classifying them as the most strategic resources. The authors used a top-down approach by modeling the entity classes and relationships for constructing the knowledge graphs. Moreover, they concluded that the ontology used to construct the knowledge graphs need to be more refined and cannot be completed at one time.

Rukmono and Chaudron (2023) experimented with a generic and extensible knowledge extraction tool like SAIN that generates instance-level knowledge graphs. Additionally, use of FAMIX concepts of metamodels for representing knowledge. Moreover, use of DP-CORE which is a design pattern instance detector has been explicitly determined as inefficient as it does not produce externally usable representation of information. The initial step was to split the knowledge representation into generic parts and project-specific parts where the generic part has been manually written by an ontology.

Kakad and Dhage (2022) has proposed the construction of a knowledge graph and their respective ontologies using cross-domain data. The authors have implemented the performance measurement using different classifier algorithms and finally concluded that Support Vector Classifier stands out as the best performer. They provide a novel approach by designing the ontology using Multiple Parameter Variable Weight (MPVW) for extracting terminologies and eventually constructing the knowledge graphs. Moreover, they discovered a rule based reasoning algorithm RelExOnt which identifies the semantic relationships in texts. Later, constructing a knowledge graph using a tool named Protégé. Furthermore, cosine similarity, TF-IDF and

WordNet have been used to find similarities in the text and used a cross-domain optimization technique named as Whale Optimization algorithm.

2. Data and Project Management Plan

2.1 Data Management Plan

The Data Management Plan describes how the data will be handled throughout the project. It details how the data will be collected, its management and storage methods and on how the data will be shared among team members during the course of the project and how it will be made available to researchers after project completion. The data management plan's objective is to guarantee efficient data storage, handling and usage while adhering to legal and ethical requirements.

2.1.1 Data collection Approaches

Data collection is one of the most crucial steps in the data management plan. For this project, we would be collecting the required documents from the official webpage of the City of San Jose. The human resource department of the City of San Jose has made Memorandum of Agreement(MOA) documents between the government and various employee organizations publicly available. These employee organizations represent the employees of the City of San Jose. The website contains archives of around 15 MOA's of various employee organizations varying from Engineers and Architects to Police officers. These MOA's would be used to build a Q&A retrieval system which would be interactive in nature. This retrieval system would be helpful in retrieving information and resolving any queries pertaining to the MOA's. This retrieval system would be utilizing the knowledge graphs extracted from these MOA's, Large Language Models, Llama Index and Nebula graph database to answer the questions related to the employee organizations.

2.1.2 Data Management Methods

Data Management is a crucial step in the data handling process, as the effectiveness of the model directly depends on accurate handling of the data. Data Management methods employ multiple steps with the first one being data cleaning. In this step, the collected data in the PDF file format will be extracted for the text. This format will be checked and cleaned to remove any inconsistencies present in it such as removing unwanted whitespaces, line breaks, removing the page numbers if they are present in page headers or footers and handling hyphenated words which may have been split across the lines. All these steps will be performed so as to maintain high quality standards of the collected data. Next stage in the data management process is data preprocessing. In this step the data will be normalized into all upper letters or all lower letters and also stop words which are insignificant will be removed to maintain data consistency. This normalized consistent data will then undergo vector embeddings to retain structured information along with capturing semantic relationships between them.

2.1.3 Storage Methods

For this project, Proper data storage methods will be used to guarantee secure storage, efficient access and loss recovery of the data. One of the cloud storage methods such as Amazon S3, Google Cloud storage or Microsoft Azure will be utilized in this project. A storage bucket will be created in a particular region and all of the data files will be uploaded into those. Identity Access Management features will be implemented in either of two approaches. Either by defining custom roles with specific permissions and giving these defined roles to team members or by giving bucket level IAM policies which would give controlled access to the data and give access only among the team members. A replica of the bucket will also be created in a different region of the same cloud storage, which acts as a data recovery mechanism in the event of any

data loss or failure and this replica bucket will be useful in disaster recovery, so that data can be quickly retrieved on the occurrence of any disaster. Through all these methods, the project team ensured a reliable, efficient data storage mechanism.

2.1.4 Usage Mechanisms

The project aims to develop a Q & A retrieval system to utilize LLM capabilities using the collected data. In order to achieve this different data mechanisms will be followed. Initially the collected data which is the PDF document will be divided into chunks and then would be fed to LLM to generate knowledge graphs. This preprocessed data would primarily be used to generate four different graph query engines so that the model can perform comparative analysis on each different query engine, so that whenever a user posts a question pertaining to the data present in the PDF document, The model would retrieve the result from the most accurate query engine. Hence through effective usage mechanism, The project would effectively use the collected data to develop an effective knowledge retrieval system. The team will share the data collection methodologies publicly on a ReadMe file in a GitHub Repository. The GitHub repository would be made public in order to help further researchers.

Hence, The project would follow best practices for data management plans to assure high quality of data by following all the guidelines pertaining to any data security and privacy.

Therefore by following these the project would deliver an effective knowledge retrieval system.

2.2 Project Development Methodology

For developing Synthia, adoption of the CRISP-DM (Cross-Industry Standard Process for Data Mining) methodology emerges as the most relevant approach to ensure incremental advancements in the project development. CRISP-DM offers a clearly outlined framework which helps us define the project's objectives and processes distinctly. This approach enhances the

project adaptability and facilitates the team to improve based on the feedback at each stage of the project development lifecycle. Additionally, implementation of CRISP-DM also allows team-members to collaborate and track progress of each process and stage involved in the project.

2.2.1 Business Understanding

This is the first stage in CRISP-DM Methodology where the objectives, goals and planned approaches are identified and defined. Comprehensive Literature Survey, Technology and Solution Survey pertaining to Large Language Models, Knowledge Graphs and Graph Databases is undertaken to explore project's use-cases and do a thorough research on the problem statement. These use-cases can help navigate the team towards achieving a better solution for the problem statement. During this stage, data requirements for the project are explored and data sources are identified. Data will be gathered from a single data source, City of San Jose's Department of Human Resource's website, which comprises 15 PDF (containing unstructured text) documents with their page counts ranging between 50 to 150 pages each.

2.2.2 Data Understanding

In this stage, we will determine the data collection process from the data sources identified in the business understanding phase. After identifying the data collection process and gathering all the data from data sources using appropriate methods, the team will come up with a data collection report encompassing a detailed documentation of data sources of the PDF Documents, along with processes and tools used to fetch the PDF Documents. It will also contain some key information like total number of documents, document size, their meta-data and the document publishing date. After data collection, exploratory data analysis will be performed for obtaining descriptive statistics like finding metrics such as word count statistics, total words,

average words per page and average words per document. Additionally, it is essential to find the most frequently occurring terms across different PDF Documents to gain insights about the important topics within the dataset. Once descriptive statistics analysis is performed, it is essential to do the content exploration. In the Content exploration phase, pages are randomly sampled to find patterns, and a few things like headers, footers etc. which need to be removed. Furthermore, the languages used in these PDF Documents are determined and check if multiple-languages were used in these documents, ultimately standardizing them into English if multiple languages are detected. Then, a data quality report would be generated which aims to identify unreadable or corrupted documents, any missing data like blank pages or sections. There could be potential challenges and anomalies like nonstandard-encodings, non-parsable PDFs that might become a challenge during chunking and text-extraction. After all potential challenges and anomalies are recognized, preliminary text analysis will be carried out. Large Language Model (LLM) tools will be used to tokenize the texts in preliminary text analysis.

2.2.3 Data Preparation

Before feeding the data to the LLM it should be pre-processed, cleaned, transformed, chunked and converted to embeddings and knowledge graphs to develop a good Question and Answer retrieval agent using LLM. Missing data like the blank pages and blank sections identified in the Data Quality Report should be cleaned before it could be utilized in the subsequent stages. During Data Cleaning, it is also essential to handle non-informative elements in the PDF Documents like the headers, footers etc., any nonstandard-encodings recognized in the Content exploration & data quality report in the Data Understanding phase. Data preparation also comprises tasks like Text pre-processing and Text tokenization. Typically, case normalization for consistency is done in the Text pre-processing stage. After Text pre-processing,

document structuring & chunking is executed using Llama Index libraries. This is apparently done to prevent token limit exception & memory issues persistent in LLMs. Document chunking is followed by feature engineering and data transformation. Service Context and Storage Context for the Modelling Phase will also be defined here in this stage. Service context refers to configuring OpenAI LLM and passing appropriate parameters like temperature, transformer model to use etc. Storage Context refers to creating a schema in Nebula Graph via Nebula Graph Console. Once Service context and storage contexts are defined they will be used for creating vector index (using Service context embedding model) and knowledge graph index. Then, Data transformation will be conducted where all the PDF document's chunks will be passed through Llama Index's library to extract, generate knowledge graph & store it in Nebula Graph DB knowledge index and vector embeddings will be stored in the vector index created with the embedding model in the Service Context. Subsequently, this knowledge index and vector index are persisted into local disk with the Data Persistence mechanisms in Llama Index. Data split ratio used for developing Synthia will be 70:10:20. 70% of the PDF Document data will be used for the training, 10% for the validation and 20% for the test data.

2.2.4 Model Development

This phase in the CRISP-DM methodology entails the model development process, where a querying engine using GPT 3.5 transformer's gpt-3.5-turbo model will be selected for developing Synthia. Four Querying Engines will be developed for Synthia and the best querying engine will be selected in the end. Four querying engines which will be developed with LlamaIndex are : Knowledge Graph Query Engine, Graph Retrieval Augmented Generation (RAG) Query Engine, Vector RAG Query Engine and Graph + Vector RAG Query Engine. Prompt Engineering will be performed in this stage to give context to the LLM in addition to the

queried results synthesized from the Query Engines to obtain responses for the user input questions. The Query Engine producing the promising results will be taken for Synthia. Hyperparameter tuning will be undertaken to get the best results based on OpenAI Evals framework and Likert Scales.

2.2.5 Model Evaluation

After selecting the best query engine, hyperparameter tuning and prompt engineering, Synthia's results will be evaluated based on OpenAI Evals and Likert Scales. OpenAI Evals is a framework which is used for evaluating applications built with LLMs as a core component. This framework includes diverse tasks on which LLMs can be tested on. Likert Scales are used to measure rating (or) opinions of a human for a particular answer generated by Synthia. Essentially, this is used for training the LLM using Reinforcement Learning with Human Feedback.

2.2.6 Deployment

Synthia will be deployed using AWS EC2, Streamlit, FAST API and Docker. AWS EC2 Service will be used to provide the computational resources needed for deploying Synthia. Streamlit is an open-source app framework which is used to develop web interfaces for Data Science, AI and Machine Learning tasks. Streamlit can be used to quickly bring up a User Interface (UI) through which users can query the the City of San Jose Memorandum of Agreements(MOAs) to get the responses using Synthia. Synthia will also be delivered using an API generated with FAST API. Finally, Synthia will be containerized before it can be deployed.

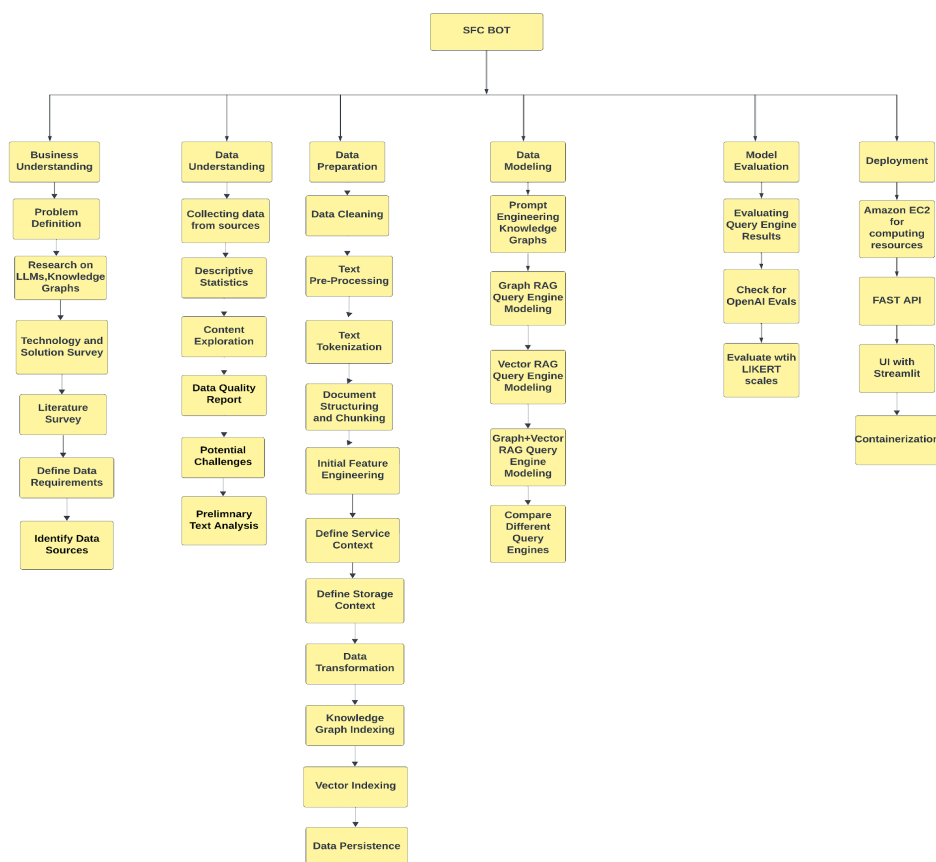
2.3 Project Organization Plan

A Work Breakdown Structure short form for WBS is a very important method for chunking down a project into smaller components or work packages which makes it manageable. The

breakdown using WBS generally follows a hierarchical manner which consists of phases, deliverables, and work packages. The WBS will perform a number of tasks such as project organization and planning, promoting communication and collaboration between team members, helping in resource allocation and tracking progress. The project is divided into six main tasks as Business Understanding and requirements, Data Understanding, Data Preparation, Modeling, Evaluation, and deployment. Each of these six major phases is subdivided into sub-tasks or components. This detailed breakdown gives project managers an easy way to assign duties, allocate resources, and monitor the project's status.

Figure 1

Work breakdown structure for the project



2.4 Project Resource Requirements and Plan

2.4.1 Hardware Requirements

LLMs need robust technology, such as GPUs and TPUs, to train the massive models concurrently. The training is more evenly distributed when nodes have quick interconnects. The huge datasets and model parameters are stored in plenty of memory and storage. Researchers can experiment with larger LLMs and more data to increase capabilities if they have the necessary hardware infrastructure. The hardware restrictions might eventually be exceeded by a growing model size. In conclusion, appropriate hardware supports LLM research but may be surpassed by escalating model requirements.

Table 2

Hardware requirements of the project

Hardware	Specifications	Costs	Purpose
Storage for data management	10GB	varies by size	storage is required
Storage	Graph storage (nebula)	varies by size	storage is used for knowledge graphs
CPU instances preprocessing and other handled	c5.4 xlarge or c5.9 xlarge	varies by size	Data model serving and GPU operations by CPU instances
Load Balancer performance, distributing traffic.	AWS elastic load balancer (ELB)	varies by size	Enhanced evenly
Ec2 instance virtual	Elastic Cloud	varies by size	Cloud based

2.4.2 Software Requirements

Table 3

Software requirements of the project

<i>Software requirements of the project</i>			
Software	Specifications	Costs	Purpose
Transformer	GPT 3.5 turbo	input tokens:\$0.003 /1k Tokens Outpt tokens:\$0.004 /1k Tokens	It can do any language task with quality and Longer output
Embedding model	text-embedding-ada-002	\$0.00/1K tokens	used to convert words to in to vectors
LLM	Lama index	Free	It can be used for Document indexing And retrieval
Python	open source	Free	Python's ML libraries are Key
Google collab	open source	Free	Collaborative, cloud-based python code editor.
Git & Github	open source	Free	Version control and collaboration platform.
Software	Specifications	Costs	Purpose
Streamlit	open source	Free	Streamlit for enhanced user interface.
FAST API	open source	Free	FastAPI streamlines LLM deployment process.
Docker	open source	Free	Containerizing LLM for easy deployment.
GPT 2 Tokenizer	open source	Free	Vital for text preprocessing in LLM.

2.4.3 License Requirements

Table 4

License requirements of the project

License Requirements:

License	Specifications	costs	Justification
Visual Studio code	open source	Free	Widely-used code editor, versatile,multilanguage support.
Microsoft office 365	open source	Free	Documentation and reporting
<i>License</i>	specifications	Costs	Purpose
Jira	Tool	Free	Visual, flexible task and project organizer.
Zoom	Communication pathway	Free	Video meeting platform with features.

2.5 Project Schedule

2.5.1 Gantt Chart

A Gantt chart is a project management tool that assists in visually representing the tasks and also dependencies associate it with time. In every gantt chart, tasks are displayed as bars where the height of the bar represents duration of the task. The deadlines of each task are represented on the left side of the bar along with overall duration on top of the bar. Gantt chart would be helpful in identifying bottlenecks and would prove useful in tracking the project progress under each task level.

Figure 2 shows the business understanding part of the project where it starts by identifying the problem definition and then performing an extensive research on different LLMs and knowledge graph/ storages followed by reading about literature papers and finding the

limitations in the existing research . Final step in this is determining the data requirements and identifying the data sources.

Figure 2

Gantt chart for Business Understanding

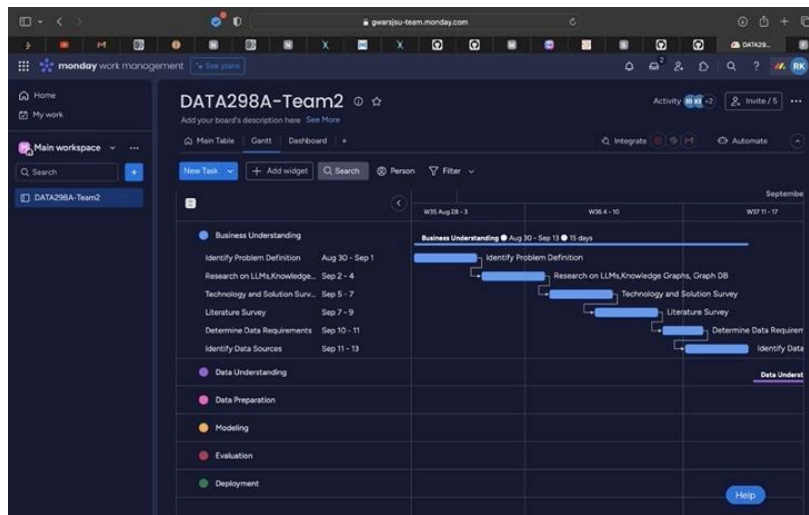


Figure 3 shows the data understanding phase of the project. This step begins with data collections from multiple sources and performing a data collection report to identify the sources of the data. In the next step the descriptive statistics of the data are derived followed by context exploration. The next is the crucial step of performing a data quality report. Last step in this is performing preliminary text analysis.

Figure 3

Gantt chart for Data Understanding

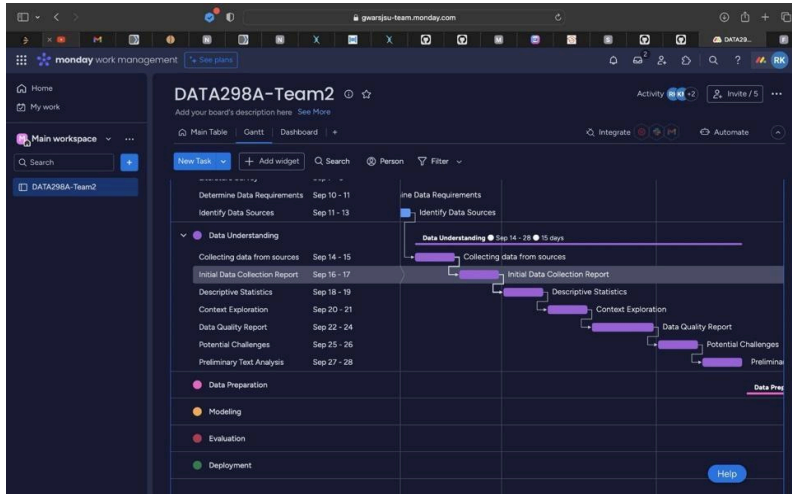


Figure 4 shows the data preparation step or phase of the project, This is one of the crucial steps in the process. This phase begins with cleaning the data, performing text pre-processing on the data to maintain high data quality standards. Next step is tokenizing the data where the text is tokenized into small vectors. This step is followed by feature engineering and defining service and store context. The main step in this process is knowledge graph indexing and vector indexing.

Figure 4

Gantt chart for Data Preparation

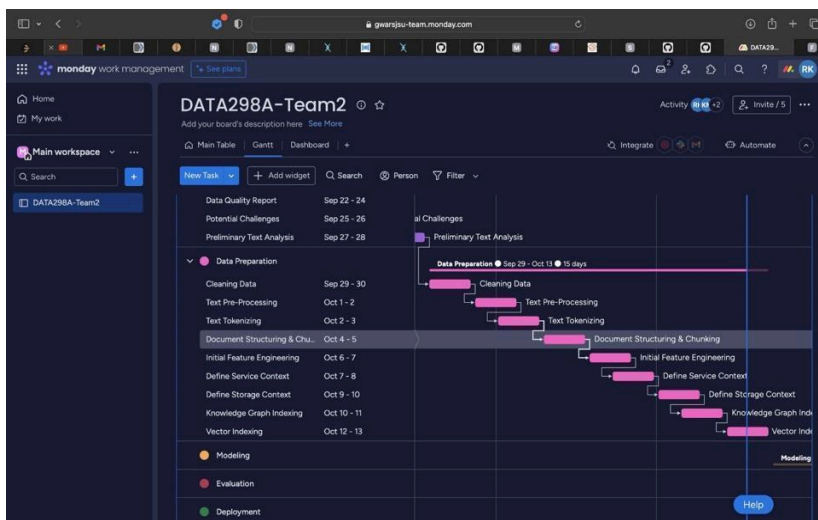


Figure 5 is the Modeling, This phase is the most crucial phase in the project. In this phase four different types of query engines are developed. Initially prompt engineering is performed on the LLM's and then followed by four different type of graph query engine generation which are Knowledge graph query engine, Graph RAG query engine, Vector RAG query engine and finally a Graph + Vector RAG Query Engine. Next step after generating the query engine is performing a comparative analysis on each query engine and selecting the best query engine. Last step in this is performing a hyperparameter engine on the best selected model.

Figure 5

Gantt chart for Modeling Phase

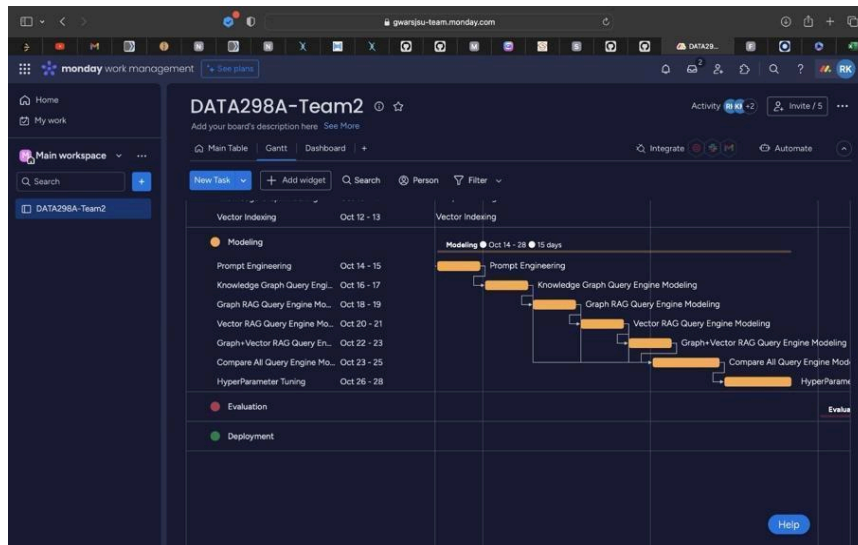


Figure 6 is the Evaluation phase, In this phase we evaluate the query performance of the four generated models and also check for OpenAI evals and LIKERT scales.

Figure 6

Gantt chart for the Evaluation Phase

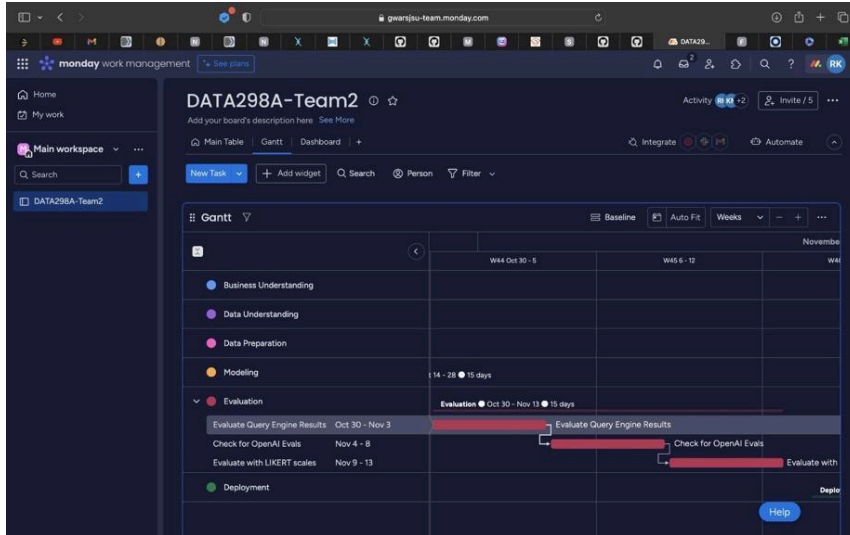
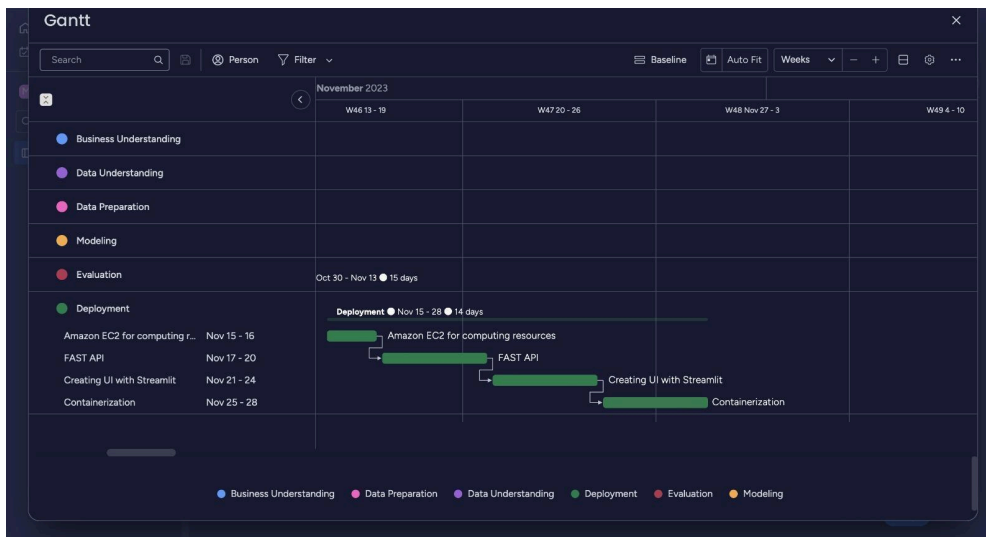


Figure 7 is the Deployment phase, This is the last phase of WBS structure. In this the developed models are deployed on the cloud where it will be accessible for other users.

Figure 7

Gantt Chart for the Deployment Phase



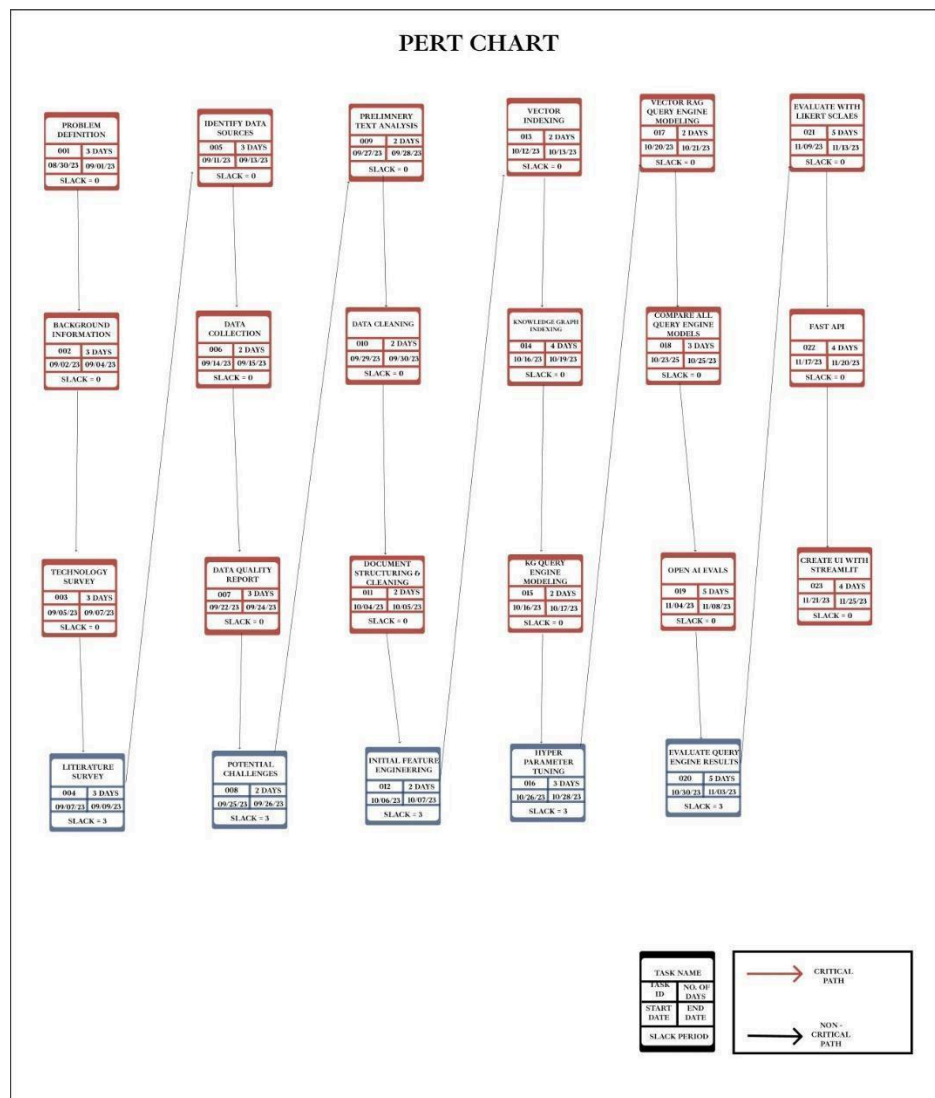
2.5.2 PERT Chart

Program Evaluation and Review Technique (PERT) is a project management tool which is used to visualize the overall project tasks and along with its dependencies. It provides a clear view of the project timeline, along with identification of the critical path. PERT charts are

essential in analyzing the tasks and have proven to be valuable in allocation of tasks to team members.

Figure 8

Pert Chart of Building Synthia



3.Data and Project Management Plan

3.1 Data Process

The data collection process for Synthia - a question - answer retrieval system is a multi-tiered strategy encompassing web scraping, data extraction, data storage, and preparation for machine learning models.

3.1.1 Data Processing Architecture

The data processing architecture involves several critical steps:

1. Information Source: We utilized the San Jose City Government's website as the primary data source, targeting web pages with Bargaining Units information.

2. Data Storage: The gathered information was first organized within a Python dictionary, then transitioned into a pandas DataFrame format, thus establishing a uniform structure suitable for later analytical operations. This approach captured all pertinent details as metadata and securely archived it in a Google Cloud Platform's data storage system - Google Cloud Storage.

3. Information Processing: We employed a combination of Python scripts, utilizing the 'requests' library to fetch webpage content and 'BeautifulSoup' to parse the HTML. This enabled us to identify and extract tables containing links to various bargaining units.

4. Data Analysis: While the primary focus at this stage was on data collection, we also organized the data into formats conducive to machine learning algorithms, laying the groundwork for future analysis.

5. Security: We adhered to the website's terms of service and robots.txt file to ensure ethical data sourcing in the scraping phase.

6. Scalability: The scripts were designed to accommodate new data sources and can be easily extended to scrape additional web pages as needed.

3.1.2 Data Gathering and Preprocessing

1. Initial Setup: We began by setting up our Python environment and importing the necessary libraries with Poetry - a dependency Management Tool. We then defined our target URL: the labor relations section of the San Jose City Government website.

3.1.2.1. Scraping and Data Extraction

The Python script is made to scrape data from a webpage. It uses web requests and HTML parsing. First, it sends a GET request to a URL, which is a city government webpage. If the response is successful, the script uses BeautifulSoup to read the HTML. This tool is good at finding and getting data from HTML.

The main task of the script is to find a table in the HTML. It looks at each row of the table and takes the data from it. Each first cell has a link with the name of a unit and a URL. The script saves these in a dictionary with the unit names and URLs.

Next, the script puts the data into a DataFrame, which is useful for data work in Python. It names the columns 'Bargaining_Unit' and 'Page_URL.' Then, it saves this DataFrame into a CSV file called moa_metadata.csv. This file keeps the unit names and URLs in a way that's easy to access and use later for things like data work or reports. This method makes sure the data is saved well.

3.1.2.2 PDF Downloading

A secondary script was utilized to store URLs, navigate to each page, identify PDF documents like MOAs and contracts, and download them using 'requests'. Uniform and compatible file names were assigned to the files.

3.1.2.3. Monitor and Cloud Integration

Using Watchdog and Google Cloud tools, our system monitored a specific folder for new PDF files. Upon detection, the file was automatically uploaded to Google Cloud storage, facilitating a seamless transition of data from local machines to cloud-based storage without manual intervention.

3.2 Data Collection

Data collection is one of the most crucial steps in the data management plan. For this project, we would be collecting the required documents from the official webpage of the City of San Jose. The human resource department of the City of San Jose has made Memorandum of Agreement (MOA) documents between the government and various employee organizations publicly available. These employee organizations represent the employees of the City and County of San Jose. The website contains archives of more than 30 MOA's of various employee organizations varying from Automotive machinists to Physicians and Dentists. The Memorandums of Understanding (MoAs) created through the preceding processes are critical in the development of Synthia, a Q&A retrieval system. Synthia is an interactive application that uses structured information collected from MoA texts to promote efficient and rapid information retrieval. This interactive Q&A retrieval system adds a layer of user-friendly functionality, making the richness of information contained in MoAs accessible and easy to navigate. This retrieval system adeptly answers inquiries about employee organizations using Large Language Models, the Llama Index, and the Nebula graph database. The integration ensures precise and informative responses, improving the system's efficiency and user experience while retrieving information.

Sources: The main source of information is the City of San Jose's official website.

Parameters:

- Document Type: MoA documents.
- Employee Organizations: Covering various Bargaining units
- Format: To ensure compatibility, the documents are in PDF format.

3.2.1 Determine Information Needs

It is critical to examine Memorandum of Agreements (MOAs) in order to establish an effective Q&A retrieval system for the City of San Jose government and employee bargaining units. This involves a detailed examination to identify key elements, attributes, and features within the MOAs. Understanding the structure of these agreements is key, as it helps unveil the types of information embedded in them, and allows for the recognition of unique specifics pertinent to employee organizations. This comprehensive exploration is necessary to ensure that the Q&A system is fine tuned to extract and deliver precise data, enhancing its ability to provide accurate and relevant information from these MOAs.

3.2.2 Determine the possible sources

The primary and trustworthy resource for locating information about Memorandum of Agreement (MOA) documents related to the City of San Jose is the official website of the city. This website likely includes a dedicated section for documents and agreements, where one can find comprehensive details. Additionally, delving into the official records of city council sessions, such as meeting minutes, prove valuable. These records may preserve decisions related to MOAs, offering insights into the reasons and motivations behind specific agreements. This in-depth exploration of council records can furnish context, analysis, and updates on the implementation of Memorandums of Agreement (MoA) for a more comprehensive understanding without resorting to plagiarism.

3.2.3 Assemble the data

Web scraping methods can be utilized to gather relevant content, ensuring that MOAs and related information are thoroughly covered. Data extraction techniques, involving the use of APIs or particular libraries, can aid in the retrieval of structured data from web sites. Executing the tailored queries can be an effective method in circumstances where databases store relevant information.

3.2.4 Data validation and cleansing

Data validation and cleansing are critical procedures in ensuring that the information gathered is correct, reliable, and available for analysis. As a result, the data is constantly enhanced and modified.

3.2.5 Data transformation and feature engineering

The primary goal of this project is to employ Large Language Models (LLMs) - both Llama Index's Fine Tuned LLM and our Llama-2 7b Fine-tuned LLM to extract knowledge graphs from the unstructured text present in the City of San Jose MoAs which will act as a knowledge base and query engine for Synthia - the Q&A LLM based retrieval system. These MoAs are PDF documents retrieved by scraping the government website, and new documents are automatically updated via a trigger. Google Cloud Platform (GCP) buckets are used as the major storage service provider. We also use Langchain document loaders to parse through the documents and load the text for the embedding models. Embedding models based on OpenAI's text embedding model - text-embedding-model-ada-2 has been used to convert the text into numerical embeddings.

3.2.6 Creating training, validation, and test datasets

Developing and evaluating Synthia requires the creation of discrete subsets for training, validation, and testing. The goal is to guarantee that the model is trained on a representative collection of data, validated on a distinct set of data to tune hyperparameters, and tested on unseen data to measure generalization performance. The success of the retrieval system, which utilizes knowledge graphs drawn from MOAs, the Llama Index, and Nebula graph database, rests on the thoroughness and correctness of the data gathering and preprocessing methods indicated above.

3.2.7 Data organization and documentation

A well-structured hierarchy makes effective data management and retrieval possible by organizing the MOA documents that have been collected in a systematic manner to improve accessibility and simplicity of analysis. The documentation includes comprehensive metadata for each dataset, thereby describing key attributes, data sources, and any preprocessing steps undertaken. Documenting metadata contributes to the understanding of data characteristics and aids in quality assessment.

Figure 9

Official webpage of the City and County of San Jose

BARGAINING UNIT	ACRONYM	CONTRACT EXPIRATION	FTEs *
Association of Building, Mechanical and Electrical Inspectors	ABMEI	06/30/2026	89.00
Association of Engineers and Architects, JPTE, Local 21	AEA	06/30/2026	361.00
Association of Legal Professionals	ALP	06/30/2024	49.00
Association of Maintenance Supervisory Personnel, JPTE Local 21	AMSP	06/30/2026	136.00

Figure 10

Raw Dataset After extracting

```
{'Article 39 - Exempt Officers and Sergeants Modified Duty Program.pdf': 'Office of Employee Relations\nExempt Officers' and Sergeants' Modified Duty Program\nBACKGROUND\n1. The City and San Jose Police Officers' Association (SJPOA) recognize that, despite best\nefforts to promote safety, police officers and sergeants are injured in the line of duty. Such\ninjuries are unfortunate but can be a consequence of police work. The Exempt Officers'\nand Sergeants' Modified Duty Program ("Program") is available to any police officer or\nsergeant that has work-related or non-work related injuries or illnesses which preclude\nhim or her from performing the full scope of his or her duties without accommodation.\n2. The City and SJPOA recognize that police officers and sergeants exist to enforce the law\nand protect public safety. Some essential job duties of a police officer and sergeant\ninclude, but are not limited to, engaging in physically strenuous tasks, such as making\nenforceable arrests, pursuing fleeing suspects, responding to mass celebrations and\ndemonstrations, and responding to emergency situations.\n3. The City and SJPOA recognize that officers and sergeants with physical restrictions can\nrepresent a valuable resource to the Police Department. Given the training and\nexperience obtained by and invested in police officers and sergeants, the Police\nDepartment can benefit from the knowledge and experience of officers and sergeants\nwith physical restrictions.\n4. The Program is intended to provide temporary modified duty assignments for officers or\nsergeants with temporary work restrictions until the officer or sergeant is returned to full\nduty status or meets the maximum duration of participation in the Program as outlined\nbelow.\n5. The City and SJPOA recognize the legitimate interest of the San Jose Police Department\nin maximizing the number of officers and sergeants available for Patrol duties, and other\nphysically strenuous activities. The responsibilities of the Police Department are largely\nphysical, involving suppression of criminal activity and exhibiting levels of force necessary\nto control dangerous and unruly situations. Emergency situations also may require the\nDepartment to provide a large uniformed presence. Additionally, at any particular time,\nbecause of short-term disabilities, sick leave, military leave, training missions, family\nleave, and the like, in addition to those in the Program, officers and sergeants are\nunavailable for Patrol activities, reducing the actual level of officers and sergeants who\nmay participate in Patrol assignments on a day-to-day basis. While the Department\nrecognizes that non-physical assignments represent necessary and valuable aspects of\npolice work, the philosophy of the Department is clearly Patrol-oriented. The first and\nmain duty of the Department is the visible protection of the public. Because of these\nprinciples, the parties recognize that the Department has a legitimate interest in limiting\nthe number of officers and sergeants on the force who are physically unable to perform\nthe strenuous physical activities necessary in a Patrol assignment. A goal of the Program\nis to strike a balance between the business needs of the Department and the interests of\ninjured officers and sergeants in meaningful employment in a temporary modified duty\nassignment. Additionally, nothing in the Program requires the City of San Jose to change\nits policies and procedures regarding the recruiting and hiring of police officers to entry\nlevel employment.\nOriginal Effective Date: June 21, 2005\nRevised Effective Date: May 11, 2020\nPage 1 of 6\nOffice of Employee Relations\nExempt Officers' and Sergeants' Modified Duty Program\n6. The City and SJPOA recognize that the administration of the Police Department\ncontinues to change over time, based on budget, staffing, criminal justice priorities,\nnegotiated terms and conditions of employment, and numerous other factors. In view of\nthe fluid
```

Figure 11

Dataset statistics for the raw data

```
{'filename': 'Article 39 - Exempt Officers and Sergeants Modified Duty Program.pdf',  
 'word_count': 3381},  
{'filename': 'Association of Building Mechanical and Electrical Inspectors (ABMEI) MOA.pdf',  
 'word_count': 19181},  
{'filename': 'Association of Engineers and Architects IFPTE Local 21 Unit 43 MOA.pdf',  
 'word_count': 17633},  
{'filename': 'Association of Engineers and Architects IFPTE Local 21 Units 4142 MOA.pdf',  
 'word_count': 17930},  
{'filename': 'Association of Legal Professionals of San Jose (ALP).pdf',  
 'word_count': 12159},  
{'filename': 'Association of Maintenance Supervisory Personnel IFPTE Local 21 (AMSP) MOA.pdf',  
 'word_count': 15324},  
{'filename': 'City Association of Management Personnel IFPTE Local 21 (CAMP) MOA.pdf',  
 'word_count': 15521},  
{'filename': 'International Brotherhood of Electrical Workers (IBEW) Local No. 332 MOA.pdf',  
 'word_count': 21843},  
{'filename': 'Municipal Employees Federation (MEF) AFSCME Local 101 MOA.pdf',  
 'word_count': 40173},  
{'filename': 'Municipal Employees Federation (MEF) Part Time Employee Reference Guide.pdf',  
 'word_count': 7938},  
{'filename': 'Officers Transfer Policy.pdf', 'word_count': 5335},  
{'filename': 'Overtime Staffing Plan.pdf', 'word_count': 2629},  
{'filename': 'POPRA 2023 MOA.pdf', 'word_count': 19532},  
{'filename': 'Promotional Memorandum of Agreement.pdf', 'word_count': 2173},  
{'filename': 'San Jos Fire Fighters - International Association of Firefighters (IAFF) Local 230 MOA.pdf',  
 'word_count': 30622},  
{'filename': 'San Jose Police Officers Association (POA) MOA.pdf',  
 'word_count': 32664},  
{'filename': 'Sergeants Transfer Policy.pdf', 'word_count': 2210},  
{'filename': 'Tentative Agreement - Peace Officer Park Ranger Association (POPRA).pdf',
```

3.3 Data Pre-processing

Effective data preprocessing is a crucial step in the data analysis pipeline, especially when dealing with raw material extracted from Memorandum of Agreements (MoA) for

knowledge graph development. The initial stage involves scraping MoA PDF documents from websites, followed by storing them in a Google Cloud Platform (GCP) bucket as the primary storage solution. To facilitate the loading of PDF documents, Lang-Chain's document loaders prove invaluable. Initially a language check is performed on the data to check for language consistency in the data and also standardizing the data in the event of presence of any diverse language in the extracted pdf files. Apart from this a data quality report is performed on the data to know the consistency, completeness and presence of any errors and anomalies in the data.

This data quality report helps in determining whether or not the data meets the necessary standards for analysis. To overcome the constraints imposed by Large Language Models (LLMs), the loaded documents undergo tokenization. Tokenization involves dividing the documents into smaller, more manageable portions, typically around 16,000 tokens each. This segmentation aligns with the processing capabilities of LLMs, contributing to increased efficiency in subsequent stages of analysis. Then analysis is performed on the chunks such as knowing the most common chunks, removal of all the chunks with a special characters, next even the most common words along with their count is retrieved from the texts, this will help in removing of non-important words or filler words from the data aiding in saving of storage space.

After the tokenization process, the next pivotal step involves transforming the textual information within each chunk into vector embeddings utilizing OpenAI's Text Embedding model. This crucial phase plays a fundamental role in converting textual input into numerical representations that encapsulate semantic meanings and contextual relationships. The vector embeddings derived from the segmented text chunks are subsequently indexed, giving rise to a content-based retrieval system characterized by efficiency. This indexing stage holds immense significance in the overall data preprocessing procedure, laying the groundwork for the

subsequent knowledge graph building phase. The efficient indexing ensures swift and effective retrieval of essential information, contributing to the seamless development of a comprehensive knowledge graph.

In the last phase of preprocessing, the integration of fine-tuned Language Models (LLamaIndex) takes center stage, specializing in the construction of knowledge graphs. These specialized knowledge graph indexing models demonstrate exceptional proficiency in discerning intricate connections between items and concepts embedded in the MoA texts. The structured data preparation pipeline, encompassing document loading, tokenization, embedding, and indexing, establishes a robust foundation for the deployment approaches in subsequent stages. Collectively, this comprehensive preprocessing procedure ensures that the raw data extracted from MoA PDFs undergoes transformation into a format well-suited for the generation and analysis of insightful knowledge graphs, thereby enhancing the overall efficacy of data-driven analyses.

Figure 12

Preliminary text analysis



Figure 13

Basic Tokenization and Analysis

```
Most Common Tokens: [('the', 17598), ('', 11865), ('of', 9821), ('.', 7900), ('to', 7878), ('and', 4620), ('or', 4415), ('employee', 4303), ('  
Stats for Article 39 - Exempt Officers and Sergeants Modified Duty Program.pdf: Total Tokens - 3381, Unique Tokens - 681, Most Common Tokens -  
Stats for Association of Building Mechanical and Electrical Inspectors (ABMEI) MOA.pdf: Total Tokens - 19181, Unique Tokens - 2652, Most Common Tokens -  
Stats for Association of Engineers and Architects IFPTE Local 21 Unit 43 MOA.pdf: Total Tokens - 17633, Unique Tokens - 2533, Most Common Tokens -  
Stats for Association of Engineers and Architects IFTPE Local 21 Units 4142 MOA.pdf: Total Tokens - 17930, Unique Tokens - 2557, Most Common Tokens -  
Stats for Association of Legal Professionals of San Jose (ALP).pdf: Total Tokens - 12159, Unique Tokens - 2073, Most Common Tokens - [('the', 6  
Stats for Association of Maintenance Supervisory Personnel IFPTE Local 21 (AMSP) MOA.pdf: Total Tokens - 15324, Unique Tokens - 2472, Most Common Tokens -  
Stats for City Association of Management Personnel IFPTE Local 21 (CAMP) MOA.pdf: Total Tokens - 15521, Unique Tokens - 2645, Most Common Tokens -  
Stats for International Brotherhood of Electrical Workers (IBEW) Local No. 332 MOA.pdf: Total Tokens - 21843, Unique Tokens - 2801, Most Common Tokens -  
Stats for Municipal Employees Federation (MEF) AFSCME Local 101 MOA.pdf: Total Tokens - 40173, Unique Tokens - 5247, Most Common Tokens - [('t  
Stats for Municipal Employees Federation (MEF) Part Time Employee Reference Guide.pdf: Total Tokens - 7938, Unique Tokens - 1345, Most Common Tokens -  
Stats for Officers Transfer Policy.pdf: Total Tokens - 5335, Unique Tokens - 1160, Most Common Tokens - [('the', 253), ('.', 166), ('of', 153), ('  
Stats for Overtime Staffing Plan.pdf: Total Tokens - 2629, Unique Tokens - 620, Most Common Tokens - [('the', 101), ('the', 100), ('to', 66), ('  
Stats for POPRA 2023 MOA.pdf: Total Tokens - 19532, Unique Tokens - 2539, Most Common Tokens - [('the', 1228), ('.', 790), ('of', 681), ('.', 5  
Stats for Promotional Memorandum of Agreement.pdf: Total Tokens - 2173, Unique Tokens - 632, Most Common Tokens - [('the', 124), ('.', 85), ('  
Stats for San Jos Fire Fighters - International Association of Firefighters (IAFF) Local 230 MOA.pdf: Total Tokens - 30622, Unique Tokens - 365  
Stats for San Jose Police Officers Association (POA) MOA.pdf: Total Tokens - 32664, Unique Tokens - 3780, Most Common Tokens - [('the', 2111), ('  
Stats for Sergeants Transfer Policy.pdf: Total Tokens - 2210, Unique Tokens - 607, Most Common Tokens - [('the', 135), ('of', 100), ('.', 54), ('  
Stats for Tentative Agreement - Peace Officer Park Ranger Association (POPRA).pdf: Total Tokens - 9122, Unique Tokens - 1714, Most Common Tokens -  
Stats for The International Union of Operating Engineers Local No. 3 (OE3) MOA.pdf: Total Tokens - 28090, Unique Tokens - 3589, Most Common Tokens -
```

Figure 14

Language detection and Standardization

```
{'filename': 'Article 39 - Exempt Officers and Sergeants Modified Duty Program.pdf',  
 'language': 'en'},  
{'filename': 'Association of Building Mechanical and Electrical Inspectors (ABMEI) MOA.pdf',  
 'language': 'en'},  
{'filename': 'Association of Engineers and Architects IFPTE Local 21 Unit 43 MOA.pdf',  
 'language': 'en'},  
{'filename': 'Association of Engineers and Architects IFTPE Local 21 Units 4142 MOA.pdf',  
 'language': 'en'},  
{'filename': 'Association of Legal Professionals of San Jose (ALP).pdf',  
 'language': 'en'},  
{'filename': 'Association of Maintenance Supervisory Personnel IFPTE Local 21 (AMSP) MOA.pdf',  
 'language': 'en'},  
{'filename': 'City Association of Management Personnel IFPTE Local 21 (CAMP) MOA.pdf',  
 'language': 'en'},  
{'filename': 'International Brotherhood of Electrical Workers (IBEW) Local No. 332 MOA.pdf',  
 'language': 'en'},  
{'filename': 'Municipal Employees Federation (MEF) AFSCME Local 101 MOA.pdf',  
 'language': 'en'},  
{'filename': 'Municipal Employees Federation (MEF) Part Time Employee Reference Guide.pdf',  
 'language': 'en'},  
{'filename': 'Officers Transfer Policy.pdf', 'language': 'en'},  
{'filename': 'Overtime Staffing Plan.pdf', 'language': 'en'},  
{'filename': 'POPRA 2023 MOA.pdf', 'language': 'en'},  
{'filename': 'Promotional Memorandum of Agreement.pdf', 'language': 'en'},  
{'filename': 'San Jos Fire Fighters - International Association of Firefighters (IAFF) Local 230 MOA.pdf',  
 'language': 'en'},  
{'filename': 'San Jose Police Officers Association (POA) MOA.pdf',  
 'language': 'en'},  
{'filename': 'Sergeants Transfer Policy.pdf', 'language': 'en'},  
{'filename': 'Tentative Agreement - Peace Officer Park Ranger Association (POPRA).pdf',  
 'language': 'en'},
```

Figure 15

Data Quality Report

```
[{'filename': 'Article 39 - Exempt Officers and Sergeants Modified Duty Program.pdf',  
  'status': 'Success',  
  'error': None},  
{'filename': 'Association of Building Mechanical and Electrical Inspectors (ABMEI) MOA.pdf',  
  'status': 'Success',  
  'error': None},  
{'filename': 'Association of Engineers and Architects IFPTE Local 21 Unit 43 MOA.pdf',  
  'status': 'Success',  
  'error': None},  
{'filename': 'Association of Engineers and Architects IFTPE Local 21 Units 4142 MOA.pdf',  
  'status': 'Success',  
  'error': None},  
{'filename': 'Association of Legal Professionals of San Jose (ALP).pdf',  
  'status': 'Success',  
  'error': None},  
{'filename': 'Association of Maintenance Supervisory Personnel IFPTE Local 21 (AMSP) MOA.pdf',  
  'status': 'Success',  
  'error': None},  
{'filename': 'City Association of Management Personnel IFPTE Local 21 (CAMP) MOA.pdf',  
  'status': 'Success',  
  'error': None},  
{'filename': 'International Brotherhood of Electrical Workers (IBEW) Local No. 332 MOA.pdf',  
  'status': 'Success',  
  'error': None},  
{'filename': 'Municipal Employees Federation (MEF) AFSCME Local 101 MOA.pdf',  
  'status': 'Success',  
  'error': None},  
{'filename': 'Municipal Employees Federation (MEF) Part Time Employee Reference Guide.pdf',  
  'status': 'Success',  
  'error': None},
```

3.4 Data Transformation

The primary goal of this project is to utilize Large Language Models (LLMs) techniques to create knowledge graphs based on Memorandum of Agreements (MoAs) related to Labor Policies and Federal Laws. These MoAs are obtained as PDF documents by scraping the government website and any new documents are automatically updated through a trigger. They are stored in a Google Cloud Platform (GCP) bucket, which serves as the storage service provider. To extract insights and relationships, from the text we employ document loaders from Lang-Chain. Utilize embedding models based on Open-AIs text embedding model and Large Language Models.

3.4.1. Workflow

3.4.1.1 Document Loading - Efficient Retrieval of PDF Information

Lang-Chain employs document loaders to efficiently retrieve information from PDF documents. These loaders simplify the extraction process by breaking down PDFs into pages. Figure 1 shows the number of pages the PDFs have been divided into for efficient retrieval.

Figure 16

```
[7] len(pages)
... 676
```

Note : The figure above illustrates the length of pages in the dataset.

3.4.1.2 Tokenization and Chunking

Addressing LLM Limitations to overcome the limitations posed by LLMs we tokenize the loaded documents breaking them into more manageable chunks. Each chunk containing 16,000 tokens is structured in a way that aligns with LLM processing capabilities. Figure 2, Figure 3 and Figure 4 demonstrate how the pages have been converted into tokenized chunks for efficient processing performance by the LLMs.

Figure 17

```
[14] len(pages_chunks)
... 1579
```

Note : This snippet describes the length of chunks present in the data.

Figure 18

Office of Employee Relations
Exempt Officers' and Sergeants' Modified Duty Program

Original Effective Date: June 21, 2005
Revised Effective Date: May 11, 2020
Page 1 of 6

BACKGROUND

1. The City and San Jose Police Officers ' Association (SJPOA) recognize that, despite best efforts to promote safety, police officers and sergeants are injured in the line of duty. Such injuries are unfortunate but can be a consequence of police work. The Exempt Officers ' and Sergeants ' Modified Duty Program ("Program ") is available to any police officer or sergeant that has work -related or non -work related injuries or illnesses which preclude him or her from performing the f ull scope of his or her duties without accommodation.

2. The City and SJPOA recognize that police officers and sergeants exist to enforce the law and protect public safety. Some essential job duties of a police officer and sergeant include, but are not limite d to, engaging in physically strenuous tasks, such as making forcible arrests, pursuing fleeing suspects, responding to mass celebrations and

Note : This figure shows how a page has been divided into processed chunks.

Figure 19

beyond the control of the employee. Each employee who is granted a leave pursuant to the provisions of this Article shall, upon return from leave, be entitled to the position within the classification held by the employee at the time the leave commenced.

13.3 If the position to which an employee would otherwise be entitled pursuant to the above has been deleted from the department's budget during the term of the employee's leave of absence, the employee shall, upon return from leave, be entitled to a position within the classification held by the employee at the time the leave commenced, provided there is either a vacancy in such classification or an employee in the classification with less seniority whose duties the returning employee is qualified to perform.

13.4 The employee is responsible for coordinating their return to work following a leave of absence. Prior to returning from a leave of absence, the employee shall contact their

Note : This figure shows how a page has been divided into processed chunks.

Figure 20

* Most Common Tokens - [{"the": 17590}, {"'": 1106}, {"or": 9821}, {"'": 7900}, {"to": 7078}, {"and": 4620}, {"or": 4415}, {"'employee": 4383}, {"'he": 4205}, {"'in": 3995}, {"'shall": 3985},
Stats for Article 39 – Exempt Officers and Sergeants Modified Duty Program.pdf: Total Tokens = 3801, Unique Tokens = 681, Most Common Tokens - [{"the": 226}, {"'": 133}, {"of": 130},
Stats for Association of Building Mechanical and Electrical Inspectors (ABMEI) MOA.pdf: Total Tokens = 19181, Unique Tokens = 2652, Most Common Tokens - [{"the": 1179}, {"'": 804}, {"'":
Stats for Association of Engineers and Architects IPFTE Local 21 Unit 43 MOA.pdf: Total Tokens = 17633, Unique Tokens = 2533, Most Common Tokens - [{"the": 1120}, {"'": 741}, {"'of":
Stats for Association of Engineers and Architects IPFTE Local 21 Units 4142 MOA.pdf: Total Tokens = 17930, Unique Tokens = 2557, Most Common Tokens - [{"the": 1127}, {"'": 743}, {"'of":
Stats for Association of Legal Professionals of San Jose (ALP).pdf: Total Tokens = 12159, Unique Tokens = 2073, Most Common Tokens - [{"the": 658}, {"'": 522}, {"'of": 352}, {"'": 267},
Stats for Association of Maintenance Supervisory Personnel IPFTE Local 21 (AMSP) MOA.pdf: Total Tokens = 15324, Unique Tokens = 2472, Most Common Tokens - [{"the": 837}, {"'": 616}, {"'
Stats for City Association of Management Personnel IPFTE Local 21 (CAMP) MOA.pdf: Total Tokens = 15521, Unique Tokens = 2645, Most Common Tokens - [{"the": 814}, {"'": 617}, {"'of": 48,
Stats for City Association of Police Officers (CAPO) MOA.pdf: Total Tokens = 14897, Unique Tokens = 2589, Most Common Tokens - [{"the": 804}, {"'": 617}, {"'of": 48},
Stats for Municipal Employees Federation (MEF) AFPSOE Local 101 MOA.pdf: Total Tokens = 40173, Unique Tokens = 5247, Most Common Tokens - [{"the": 2098}, {"'": 1483}, {"'": 1183}, {"'
Stats for Municipal Employees Federation (MEF) Part Time Employee Reference Guide.pdf: Total Tokens = 7938, Unique Tokens = 1345, Most Common Tokens - [{"the": 341}, {"'": 263}, {"'of":
Stats for Officers Transfer Policy.pdf: Total Tokens = 5335, Unique Tokens = 1160, Most Common Tokens - [{"the": 253}, {"'": 166}, {"'of": 153}, {"'": 132}, {"'": 123}],
Stats for Overtime Staffing Plan.pdf: Total Tokens = 2629, Unique Tokens = 620, Most Common Tokens - [{"the": 101}, {"'": 100}, {"'": 66}, {"'of": 62}, {"'": 59}],
Stats for POPRA 2023 MOA.pdf: Total Tokens = 19532, Unique Tokens = 2539, Most Common Tokens - [{"the": 1228}, {"'": 790}, {"'of": 681}, {"'": 561}, {"'": 559}],
Stats for Promotional Memorandum of Agreement.pdf: Total Tokens = 2173, Unique Tokens = 632, Most Common Tokens - [{"the": 124}, {"'": 85}, {"'of": 81}, {"'": 62}, {"'": 43}],
Stats for Public Safety Officers Association (PSOA) MOA.pdf: Total Tokens = 138, Unique Tokens = 138, Most Common Tokens - [{"the": 138}], Stats for PSOA MOA.pdf: Total Tokens = 1802,
Stats for San Jose Police Officers Association (POA) MOA.pdf: Total Tokens = 32664, Unique Tokens = 3780, Most Common Tokens - [{"the": 2111}, {"'": 1294}, {"'": 1177}, {"'": 908},
Stats for Sergeants Transfer Policy.pdf: Total Tokens = 2210, Unique Tokens = 667, Most Common Tokens - [{"the": 135}, {"'": 100}, {"'": 54}, {"'": 49}, {"'": 49}],
Stats for Tentative Agreements – Peace Officer Park Ranger Association (POPRA).pdf: Total Tokens = 9122, Unique Tokens = 1714, Most Common Tokens - [{"the": 484}, {"'": 299}, {"'": 271},
Stats for The International Union of Operating Engineers Local No. 3 (IOE) MOA.pdf: Total Tokens = 28908, Unique Tokens = 3589, Most Common Tokens - [{"the": 1545}, {"'": 1890}, {"'": 1545}]]

Note : The snippet above demonstrates the most common tokens being extracted.

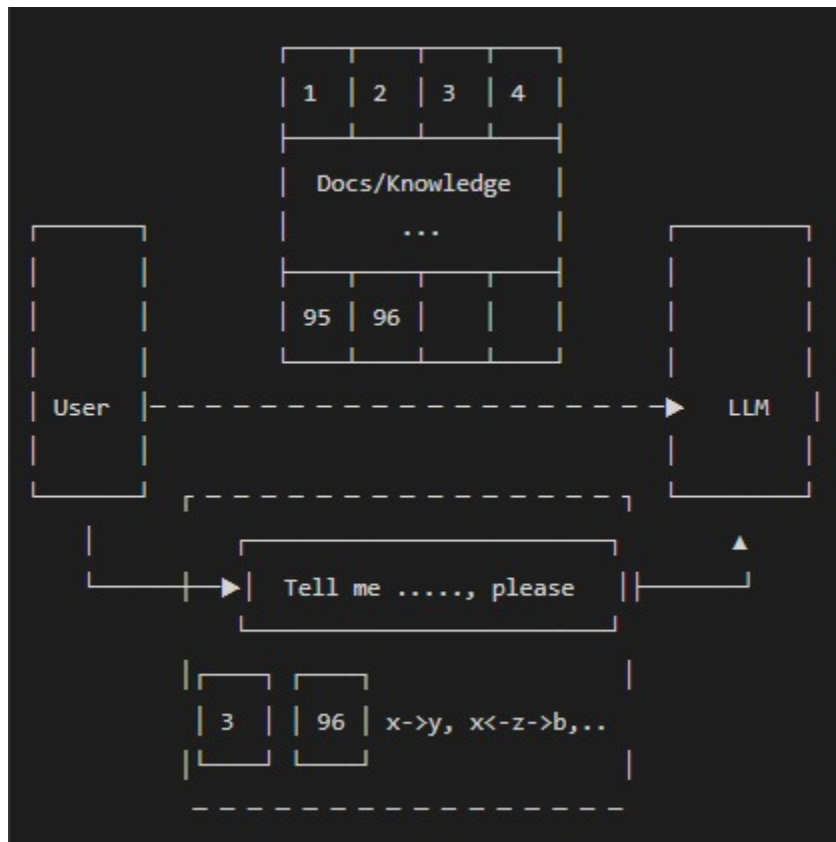
3.4.1.3 Text Embedding - Leveraging OpenAI's Embedding Model for Transformation

OpenAI's Text Embedding model plays a role in converting each text chunk into vector embeddings. This involves transforming information into representations that capture semantic meanings and contextual relationships.

3.4.1.4 Vector Indexing - Simplified Retrieval, for Knowledge Graphs

The vector embeddings obtained from segments of text undergo a process called indexing, which creates a retrieval system based on content. This step is crucial to ensure effective generation of knowledge graphs. Figure 7 demonstrates the basic overview of how the text from the documents have been converted into vector embeddings.

Figure 21



Note : The figure above demonstrates how the data has been converted into vector embeddings.

3.4.1.5 Knowledge Graph Generation - Utilizing Fine Tuned LLMs

Specialized in generating knowledge graphs, Fine-tuned Language Models (LLamaIndex) excel in capturing relationships between entities and concepts found within data. These models are tailored specifically for knowledge graph indexing. Figure 8 below demonstrates the utilization of fine-tuned LLMs for knowledge graph generation.

Figure 22

```
<s>[INST] <<SYS>>You are a helpful agent that extracts
knowledge graph in RDF Format from the english text
provided by the user. You will be provided with
Entities, Relationships and Objects while training. So
Use Entities, Relationships, Objects and User provided
text for learning and training to extract a knowledge
graph in RDF Format. Try to be very accurate &
detailed while extracting the knowledge graph, learn
in the best possible way you can. Entities you'll be
coming up with while generating knowledge graph from
user-provided text are: Tribal Opera, Mark Heard,
iDEoLA, What? Records. Relationships you'll be coming
up with while generating knowledge graph from user-
provided text are: is the title of, released in.
Objects you'll be coming up with while generating
knowledge graph from user-provided text are: an album,
1987.</SYS>> Tribal Opera is the title of an album by
Mark Heard (using the pseudonym iDEoLA), released in
1987 on What? Records. [/INST]
<http://dbpedia.org/resource/Tribal_Opera>
<http://www.w3.org/2000/01/rdf-schema#comment> 'Tribal
Opera is the title of an album by Mark Heard (using
the pseudonym iDEoLA), released in 1987 on What?
Records.'. </s>
```

3.4.1.6 Conclusion - Converting PDFs into Knowledge Graphs

By integrating document loading, tokenization, embedding, vector indexing and tuned LLMs this project aims to convert PDF documents into knowledge graphs. Advanced LLM techniques contribute to a nuanced understanding of content enabling the extraction of insights and relationships. This structured process enhances information management and analysis by facilitating knowledge extraction and retrieval.

3.5 Data Preparation

Dataset used to create a vector index and knowledge graph index was collected and scraped from City of San Jose's bargaining units web pages. The dataset for Fine-tuning Large Language Model (Llama-2) to extract knowledge graph was obtained from dbpedia website. We collected 19 PDF Documents which are Memorandum of Agreements (MOAs) signed between the City of San Jose and various bargaining units. After scraping the MOA PDF Documents, we extracted raw text out of the documents and checked the data for any large extra spaces, languages other than english, corrupt files etc and finally cleaned the data. PDF Documents were later chunked by verifying the token size with tiktoken and were used to extract knowledge graph with Finetuned Llama Index LLM using . Sample Data Points for leveraging Llama Index's LLM has id, embedding, meta_data, file_name, page_content.

Figure 23

Sample Data Point to develop knowledge graph from MOA PDF Documents

```
[ExtendedDocument(id_='2e93cb39-f562-4c63-ac09-7bf948f2409a', embedding=None, metadata={'page_number': 0, 'file_name': 'Article 39 - Exempt Officers and Sergeants Modified D
ExtendedDocument(id_='6fecac02-f4d4-466c-9347-eeab83ef50e5', embedding=None, metadata={'page_number': 1, 'file_name': 'Article 39 - Exempt Officers and Sergeants Modified D
ExtendedDocument(id_='b8e574b2-0589-4945-9fc3-05156213a5ee', embedding=None, metadata={'page_number': 2, 'file_name': 'Article 39 - Exempt Officers and Sergeants Modified D
ExtendedDocument(id_='fdc07f2e-c8a7-42cb-a2f5-3108011a21f4', embedding=None, metadata={'page_number': 3, 'file_name': 'Article 39 - Exempt Officers and Sergeants Modified D
ExtendedDocument(id_='556d4e63-3db3-47d9-b3c2-74b5e9f39ed3', embedding=None, metadata={'page_number': 4, 'file_name': 'Article 39 - Exempt Officers and Sergeants Modified D
ExtendedDocument(id_='19140f9d-d90b-4654-96f1-b1ea03efa3d5', embedding=None, metadata={'page_number': 5, 'file_name': 'Article 39 - Exempt Officers and Sergeants Modified D
ExtendedDocument(id_='8b20e0aa-cdb8-4eb9-994b-7cc448911bbb', embedding=None, metadata={'page_number': 0, 'file_name': 'Association of Building Mechanical and Electrical Ins
ExtendedDocument(id_='2519fffc-684e-4c84-b42e-133fe1e559dd', embedding=None, metadata={'page_number': 1, 'file_name': 'Association of Building Mechanical and Electrical Ins
ExtendedDocument(id_='793b1551-8093-4c15-805c-cb091c667930', embedding=None, metadata={'page_number': 2, 'file_name': 'Association of Building Mechanical and Electrical Ins
ExtendedDocument(id_='78c7c542-5a16-42a4-bfa7-1570c7c4ed78', embedding=None, metadata={'page_number': 3, 'file_name': 'Association of Building Mechanical and Electrical Ins
ExtendedDocument(id_='63060672-b756-4057-b3ac-89c74ad03944', embedding=None, metadata={'page_number': 4, 'file_name': 'Association of Building Mechanical and Electrical Ins
ExtendedDocument(id_='8a58ce75-adce-45e9-8746-de5a32193c7b', embedding=None, metadata={'page_number': 5, 'file_name': 'Association of Building Mechanical and Electrical Ins
ExtendedDocument(id_='9540b216-c273-4bf0-80a7-b3346b81143e', embedding=None, metadata={'page_number': 6, 'file_name': 'Association of Building Mechanical and Electrical Ins
ExtendedDocument(id_='f2bcf488-8ee5-4bf2-a23c-e7c745ba6e2a', embedding=None, metadata={'page_number': 7, 'file_name': 'Association of Building Mechanical and Electrical Ins
ExtendedDocument(id_='a0e3cca3-01b0-49a7-9d62-5e59e83327c1', embedding=None, metadata={'page_number': 8, 'file_name': 'Association of Building Mechanical and Electrical Ins
ExtendedDocument(id_='43b1e7c6-f69d-472a-bfc9-b177b2755fb1', embedding=None, metadata={'page_number': 9, 'file_name': 'Association of Building Mechanical and Electrical Ins
ExtendedDocument(id_='7fde60d-95ba-426c-baa9-ac6600e82a73', embedding=None, metadata={'page_number': 10, 'file_name': 'Association of Building Mechanical and Electrical Ins
```

Dataset downloaded from dbpedia was used to fine-tune Llama-2-7b LLM. This dataset originally had knowledge graphs in RDF Format. This work employs reverse-engineering of the original dbpedia knowledge graph with LLM and RDF Library in Python to extract the english text present in the knowledge graph. LLM used in Reverse Engineering the DBPedia Dataset is GPT 3.5 Turbo. After reverse engineering and prompting the LLM, we obtained our expected

dataset for fine-tuning the Llama-2 LLM. This dataset was split in 70:10:20 for training:validation:test datasets.

Each Datapoint in the training dataset for finetuning Llama-2 base model consists of text key with value which is in the following format:

```
<s>[INST]<<SYS>>System_Context<</SYS>>User_Question[/INST]Assistant_Sample_Answer</s>
```

In the system Context we have specified all the entities, relationships and objects which are present in the sample natural language text and passed it along with the natural language text as User_Question and knowledge graph in RDF Format as Assistant_Sample_Answer thereby allowing the LLM to learn properly during the fine-tuning.

Figure 24

Knowledge graph fine-tuning for Llama-2 7b LLM's training dataset's sample data point

```
"text": "<s>[INST] <<SYS>>You are a helpful agent that derives knowledge graph from the english text provided by the user. Entities you'll -provided text are: Holiday Cup, United States, Women's Water Polo World League. Relationships you'll be coming up with while generating held in, featuring some of the best, take place in, emergence of. Objects you'll be coming up with while generating knowledge graph from yearly happening, second and third year of the quadrennial period.<</SYS>> The Holiday Cup is a waterpolo competition held in the United women's water polo has to offer. The event which originally had been a yearly happening was modified throughout the years to take place the emergence of the Women's Water Polo World League. [/INST] <http://dbpedia.org/resource/Holiday_Cup> <http://www.w3.org/2000/01/rdf-s the United States since 1998, and featuring some of the best the world of women's water polo has to offer. The event which originally ha place in the second and third year of the quadrennial period, thanks to the emergence of the Women's Water Polo World League.\". </s>"
```

Sample Validation Data Points and Sample Test Data Points do not have the entities, relationships and objects unlike training dataset allowing the model to be evaluated and tested

Figure 25

Knowledge graph fine-tuning for Llama-2 7b LLM's validation dataset's sample data points

```

<s>[INST] You are a helpful agent that derives knowledge graph in RDF Format from the following text : Live Johnny Winter And is a 1971 album by Johnny Winter recorded live at various locations during the fall of 1970 including the Capitol Theatre in Port Chester, New York, and Pirate's World in Dania, Florida. [/INST]</s>

<s>[INST] You are a helpful agent that derives knowledge graph in RDF Format from the following text : Empedaula is a genus of moth in the family Gelechiidae. [/INST]</s>

<s>[INST] You are a helpful agent that derives knowledge graph in RDF Format from the following text : Mei li is a book by Thomas Handforth. Released by Doubleday, it was the recipient of the Caldecott Medal for illustration in 1939. [/INST]</s>

<s>[INST] You are a helpful agent that derives knowledge graph in RDF Format from the following text : "The Awful Battle of the Pekes and the Pollicies" is a song in the musical Cats. It is sung mostly by Munkustrap. It describes a battle between two neighbouring dog tribes, the Pekes and the Pollicies, who bark "until you could hear them all over the park". The cats stage the battles by dressing up as dogs and barking in each other's faces. This goes on until the Great Rumpus Cat arrives and scares the dogs away. [/INST]</s>

```

Figure 26

Knowledge graph fine-tuning for Llama-2 7b LLM's validation dataset's sample data points

```

<s>[INST] You are a helpful agent that derives knowledge graph in RDF Format from the following text : Meetin' WA is a 1986 short film by Jean-Luc Godard. In the film, he interviews his "old friend" Woody Allen. Throughout the film there are cuts in the interviews featuring photographs and film clips from Woody Allen's films. The two of them talk about movies, life, relationships to other directors and...

<s>[INST] You are a helpful agent that derives knowledge graph in RDF Format from the following text : Tuek Thla is a khum (commune) of Serei Saophoan District in Banteay Meanchey Province in north-western Cambodia [/INST]</s>

<s>[INST] You are a helpful agent that derives knowledge graph in RDF Format from the following text : Rosh HaNikra is a kibbutz in northern Israel. Located on the Mediterranean coast near the Rosh HaNikra grottoes and the border with Lebanon, it falls under the jurisdiction of Mateh Asher Regional Council. In 2006 it had a population of 746. [/INST]</s>

<s>[INST] You are a helpful agent that derives knowledge graph in RDF Format from the following text : Human Frailty is an album released by Australian band Hunters & Collectors in 1986. It was their commercial breakthrough album in Australia, and a commercial and critical success. Singles from the album were 'Say Goodbye', 'Throw Your Arms Around Me' (a re-recorded version of the previous release as a...

<s>[INST] You are a helpful agent that derives knowledge graph in RDF Format from the following text : Cognitive Ergonomics studies cognition in work settings, in order to optimize human well-being and system performance. It is a subset of the larger field of human factors and ergonomics. [/INST]</s>

```

After extracting the knowledge graph and using the knowledge graphs in different query engines being modelled, the responses produced by different query engines will be evaluated. Sample Dataset for evaluating the query engines will be consisting of the keys: chat,_history question and ideal, here chat_history key holds any chat_history between Synthia and user, question is the question on which Synthia will be evaluated, ideal key holds the value which should be the ideal answer for the question. This dataset was also generated by prompt-engineering OpenAI's GPT 3.5 Turbo Model

Figure 27

Sample Data Point for evaluating Synthia's query engines

```

{
  "chat_history": "[]",
  "question": "What is the purpose of the Exempt Officers' and Sergeants' Modified Duty Program?",
  "ideal": "The purpose of the Exempt Officers' and Sergeants' Modified Duty Program is to provide temporary modified duty assignments for police officers or sergeants with temporary work restrictions until they are able to return to full duty."
},
{
  "chat_history": "[]",
  "question": "What are some essential job duties of a police officer and sergeant?",
  "ideal": "Some essential job duties of a police officer and sergeant include engaging in physically strenuous tasks such as making forcible arrests, pursuing fleeing suspects, responding to mass celebrations and demonstrations, and responding to emergency situations."
},
{
  "chat_history": "[]",
  "question": "What is the purpose of the MOU signed between the City of San Jose and the bargaining unit?",
  "ideal": "The purpose of the MOU is to establish an agreement between the City of San Jose and the bargaining unit regarding the temporary work restrictions for sergeants until they are returned to full-duty status or meet the maximum duration of participation in the Program."
},
{
  "chat_history": "[]",
  "question": "Why does the San Jose Police Department have an interest in maximizing the number of officers and sergeants available for Patrol duties?",
  "ideal": "The San Jose Police Department has a legitimate interest in maximizing the number of officers and sergeants available for Patrol duties because the responsibilities of the Police Department involve physical activities such as suppression of criminal activity, exhibiting levels of force necessary to control dangerous situations, and providing a large uniformed presence during emergency situations. The Department recognizes that non-physical assignments are valuable, but its philosophy is clearly Patrol-oriented and the visible protection of the public is its main duty."
}
}

```

3.6 Data Statistics

Dataset collected from the City of San Jose's MOAs was used in generating the evaluation dataset for Synthia's query engines. This dataset has 2833 data points with keys chat_history, question and ideal_answer.

Figure 28

Synthia's best Query Engine evaluation Dataset

```

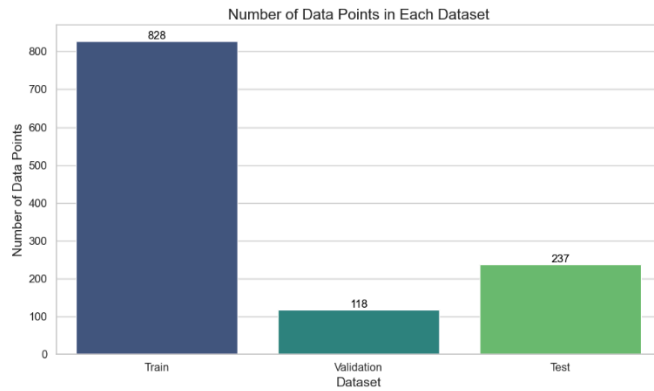
{"chat_history": "[]", "question": "What is the purpose of the Exempt Officers' and Sergeants' Modified Duty Program?", "ideal": "The purpose of the Exempt Off"},
{"chat_history": "[]", "question": "What are some essential job duties of a police officer and sergeant?", "ideal": "Some essential job duties of a police offi"},
{"chat_history": "[]", "question": "What is the purpose of the MOU signed between the City of San Jose and the bargaining unit?", "ideal": "The purpose of the"},
{"chat_history": "[]", "question": "Why does the San Jose Police Department have an interest in maximizing the number of officers and sergeants available for P"},
{"chat_history": "[]", "question": "What is the purpose of the MOU signed between the City of San Jose and the bargaining unit?", "ideal": "The purpose of the"},
{"chat_history": "[]", "question": "Who are the parties involved in the MOU?", "ideal": "The parties involved in the MOU are the City of San Jose and one of th"},
{"chat_history": "[]", "question": "What is the purpose of the Exempt Officers' and Sergeants' Modified Duty Program?", "ideal": "The purpose of the Program is"},
{"chat_history": "[]", "question": "Who is eligible for the Exempt Officers' and Sergeants' Modified Duty Program?", "ideal": "Officers and sergeants with temp"},
{"chat_history": "[]", "question": "What is the grievance process for claims regarding the interpretation and application of the specific terms of the Program?", "ideal": "Nothing in the"},
{"chat_history": "[]", "question": "How does the Program affect the transfer policies for officers and sergeants not covered by it?", "ideal": "Nothing in the"},
{"chat_history": "[]", "question": "Who are considered exempt officers or sergeants under the Program?", "ideal": "Under the Program, a maximum number of offic"},
{"chat_history": "[]", "question": "What accommodation does the Department attempt to provide for officers or sergeants who are temporarily restricted from per"},
{"chat_history": "[]", "question": "What is the effective date of the MOU between the City of San Jose and the bargaining unit?", "ideal": "The original effect"},
{"chat_history": "[]", "question": "How many exempt positions are allowed in the Exempt Officers' and Sergeants' Modified Duty Program?", "ideal": "The maximum"},
{"chat_history": "[]", "question": "What is the purpose of the MOU signed between the City of San Jose and the bargaining unit?", "ideal": "The purpose of the"},
{"chat_history": "[]", "question": "What are exempt positions in the Department and how are they filled?", "ideal": "Exempt positions in the Department are adm"},
{"chat_history": "[]", "question": "What is the effective date of the MOU between the City of San Jose and the bargaining unit?", "ideal": "The original effect"},
{"chat_history": "[]", "question": "What is the purpose of the Exempt Officers' and Sergeants' Modified Duty Program?", "ideal": "The purpose of the Exempt Off"},
{"chat_history": "[]", "question": "What happens if a volunteer applies for a disability retirement?", "ideal": "If a volunteer applies for a disability retire"},
{"chat_history": "[]", "question": "What happens if there are multiple injured officers or sergeants and a vacancy becomes available?", "ideal": "If there are"},
{"chat_history": "[]", "question": "How are officers or sergeants accepted into the Program in case of multiple injuries and vacancies?", "ideal": "Officers or"},
{"chat_history": "[]", "question": "What happens if an officer or sergeant in the Program requires a leave of absence greater than six weeks?", "ideal": "If an"},
{"chat_history": "[]", "question": "What is the effective date of the MOU between the City of San Jose and the bargaining unit?", "ideal": "The original effect"},
{"chat_history": "[]", "question": "What is the maximum duration for officers or sergeants to participate in the Exempt Officers' and Sergeants' Modified Duty"},
{"chat_history": "[]", "question": "What are the options available for employees with permanent and stationary restrictions who cannot be provided a reasonable"},
{"chat_history": "[]", "question": "How long is the maximum time in the program for an employee who has applied for a disability retirement and it is pending d"},
{"chat_history": "[]", "question": "How are officers and sergeants promoted in the Department?", "ideal": "The Department promotes officers to the rank of serg"}

```

We also divided our fine-tuning datasets into training, validation and testing datasets with 828 data points, 118 data points and 236 data points correspondingly.

Figure 29

Data Points in each dataset for fine-tuning Llama-2-7b base model



Statistically, training dataset text length distribution, test dataset text length distribution and validation dataset text length distribution for the MOA Unstructured Chunks have also been analyzed.

Figure 30

Train Dataset Text Length Distribution

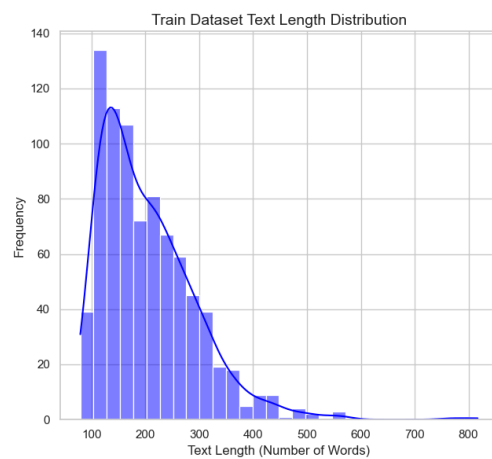


Figure 31

Validation Dataset Text Length Distribution

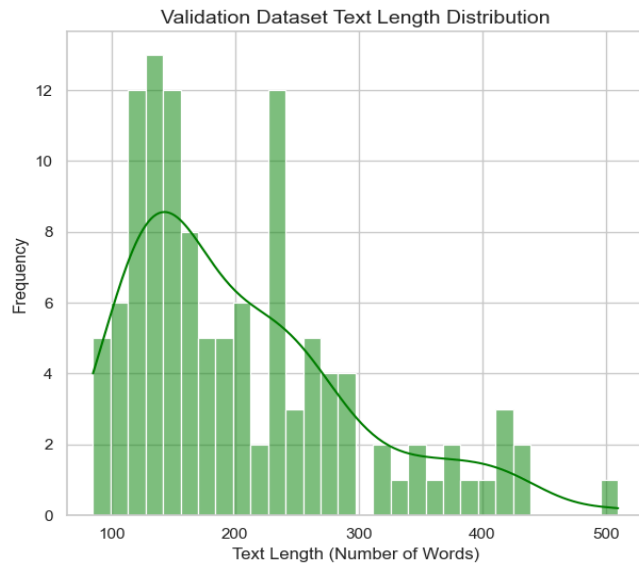
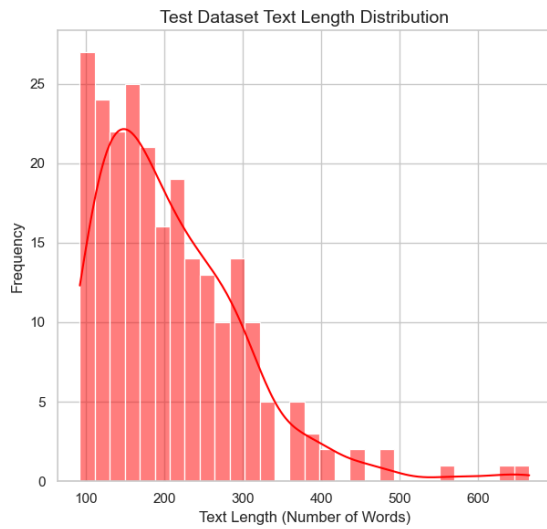


Figure 32

Test Dataset Text Length Distribution



4. Model Development

4.1 Model Proposals

In this work, 2 Pre-trained Large Language Models: Llama2 and OpenAI GPT (Generative Pre-trained) 3.5-turbo have been used for developing Synthia, where the former LLM is being used to extract the Knowledge Graphs out of unstructured text and the latter one is defined in the Service Context and used inside the query engine to generate responses returned to the user. It is evident that there are 2 stages for this project: Knowledge Graph Extraction and developing Query Engines to leverage the knowledge graph and vector RAG. 4 different query engines have been proposed as part of the second stage in this research work and response from each query engine will be compared and finally one query engine will be deployed in Synthia.

4.1.1 Knowledge Graph Extraction

The advent of Large Language Models (LLMs) which have been extensively trained on huge volumes of data available on Internet has been revolutionary in the field of Natural Language Processing. However, LLMs might provide an irrelevant answer when we ask more complex, domain-specific, open-ended questions. For addressing this knowledge gap issue, Retrieval Augmented Generation (RAG) approach can be used for training the LLM on domain-specific tasks and help the model learn. This approach works on the principle of extracting relevant knowledge from the domain sources to provide context for the LLM to generate an informed and better response. RAGs retrieve numerical embeddings from the Vector Database using similarity search measures like Cosine Similarity, fetching semantically user question similar vectors. This process has a few drawbacks which include their inability to comprehend the information effectively and their absence of logical chain of reasoning. Therefore, we use Knowledge Graphs which can represent the text as real-world entities as nodes

and relationships between entities as edges. Traversing knowledge graphs will help us achieve the logical chain of reasoning. To extract the knowledge graphs out of unstructured texts, we employ 2 approaches

4.1.2 Llama Index's Knowledge Graph Index

This is Llama Index's fine-tuned Large Language Model (LLM) which is fine-tuned to extract knowledge graph, RDF (Resource Description Framework) Triples consisting of subject, predicate and object for a piece of unstructured text. This LLM handles automated knowledge graph construction and can be queried based on entities offering a lot of flexibility for querying the graph.

4.1.3 Fine-tuned Llama-2 7b LLM (Llama-2-7b-chat-finetune)

In this research work, this is the LLM which has been obtained by fine-tuning for extracting entities, relationships from the unstructured text, thereby yielding a knowledge graph in the RDF Format. Llama's fine-tuned chat-model Llama-2-7b-chat-hf published by hugging face, licensed by Meta was used to fine-tune the base model to obtain our Llama-2-7b-chat-finetune model.

Table 5

Parameters used to fine-tune the base Llama 2 chat-model

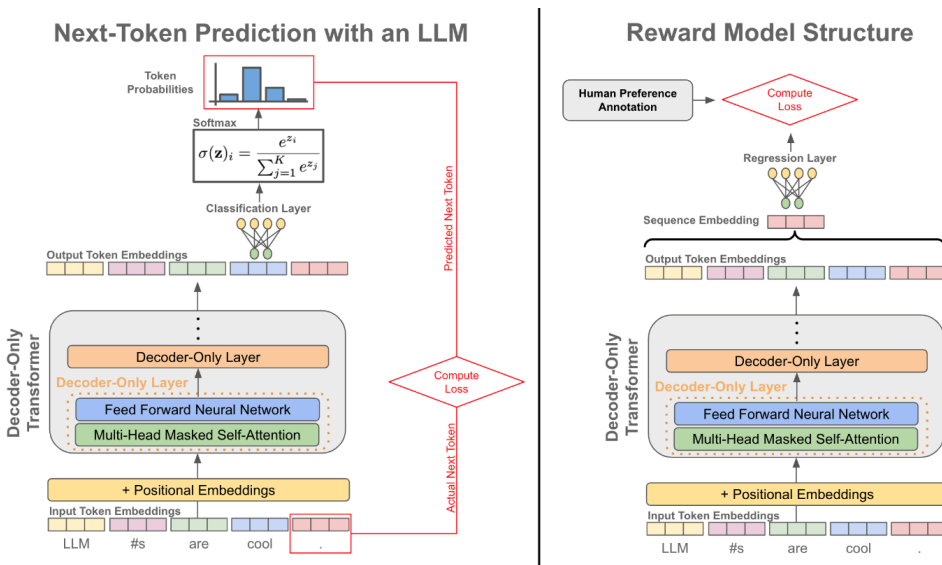
Parameter	Value	Purpose
lora_r	64	To specify Low-Rank Adaption (LoRA) dimesion of the attention to reduce the number of trainable parameters & help in light-weight training.
lora_alpha	16	To specify the scaling factor for LoRA. This parameter governs how low-rank matrices are scaled thereby affecting the model-performance

lora_dropout	0.1	Specifies the drop-out probability in LoRA Layers and helps in regularization and prevent over-fitting
use_4bit	True	Enable loading of the Llama2 7b base model with 4-bit precision in order to reduce the memory usage
bnb_4bit_compute_dtype	float16	Defines the computation data-type in the 4-bit models
bnb_4bit_quant_type	nf4	Specifies the quantization type leading to how model's weights will be compressed
use_nested_quant	False	If nested quantization is needed
output_dir	./results	Specifies where to store model predictions and check-points
num_train_epochs	1	Number of training epochs
fp16	True	Enable mixed precision training to improvise the performance
bf16	False	Enable mixed precision training to improvise the performance
per_device_train_batch_size	4	Batch-size for training
per_device_eval_batch_size	4	Batch-size for evaluation
Parameter	Value	Purpose
gradient_accumulation_steps	1	Number of steps to accumulate gradients before the model weights are updated.
gradient_checkpointing	True	Enables saving intermediate activations to save memory
max_grad_norm	0.3	Maximum gradient norm for clipping gradient to prevent exploding gradient
learning_rate	2e-4	Initial learning rate for regularization
weight_decay	0.001	Initial weight decay
optim	paged_adamw_32bit	Specify Optimizer being used

lr_scheduler_type	cosine	Defines the type of learning rate scheduler
max_steps	-1	Defines the maximum training steps
warmup_ratio	0.03	Defines the ratio of warm-up steps
group_by_length	True	This helps in increasing efficiency
save_steps	0	Determines how frequently checkpoints are saved
logging_steps	25	Determines how frequently information is logged
max_seq_length	None	Specifies the maximum sequence length for processing the inputs
packing	False	Determines if multiple short-examples should be packed into same input sequence
device_map	{“”: 0}	Mapping the model parts to GPU

Figure 33

Normal Llama-2 Architecture vs Llama-2 Architecture with reward-based model

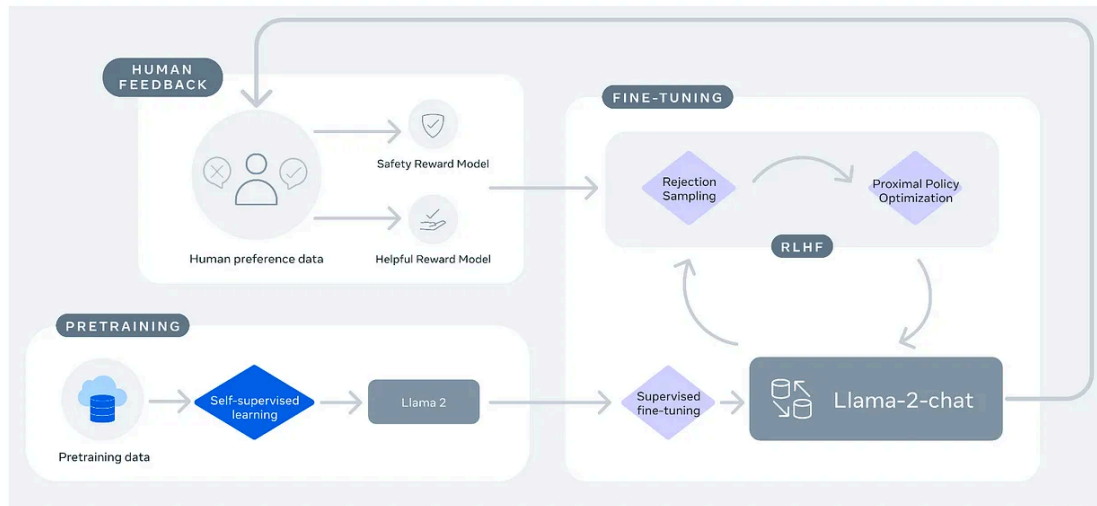


Note : Image By (Cameronrwolfe, 2023)

Figure 33 shows the architecture of Llama-2 and a Reward-based Llama-2 Model.

Figure 34

Training of Llama-2 Chat



Note : Image By Touvron et al. (2023)

Figure 34 shows how a pre-trained baseline Llama-2 Model can be used for fine-tuning along with Reinforcement Learning with Human Feedback (RLHF).

4.1.4 LLM defined in Service Context – GPT 3.5 Turbo

OpenAI's GPT 3.5 turbo model will be defined in the Service Context. This LLM will be used in multiple query engines in different use-cases, some of which are to generate a input in Cypher to query the knowledge graph in Nebula Graph DB, fetch the results as knowledge sub-graphs transform them into English text and return the response to the user.

4.1.5 Query Engine Modeling

Knowledge graphs derived with Llama Index's Fine-tuned model and Llama-2-7b-chat-finetune model will be stored in the Nebula Graph database which will be then

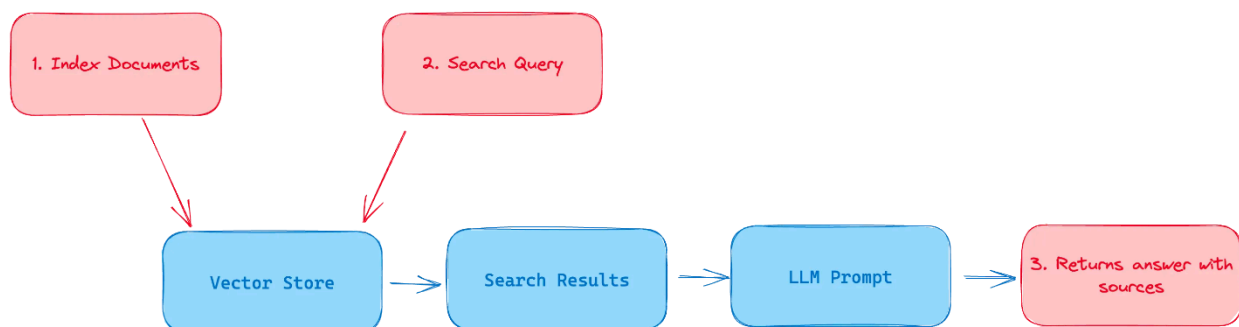
queried by employing multiple query engine mechanisms. This project encompasses usage of 4 Query Engines:

4.1.6 RAG Query Engine

Synthia's RAG query engine is engineered using the following components: Service Context defined Large Language Model i.e., GPT 3.5 turbo and City of San Jose's MOA Document text's numerical vector embeddings inside Llama Index's Vector Database. Before executing the Retrieval Augmentation Generation (RAG) process, all the city's MOA documents are chunked and converted as numerical vector embeddings and ultimately upserted into the Llama Index's vector database. When Synthia's user requests for some information, Vector Database is indexed and based on the cosine similarity measure between the numerical embeddings of the user-questions and the embeddings inside the vector database, top-k chunks are retrieved and given as context to the GPT 3.5 Turbo which then generates the user-requested response and provides it to the user.

Figure 35

RAG Query Engine Modeling



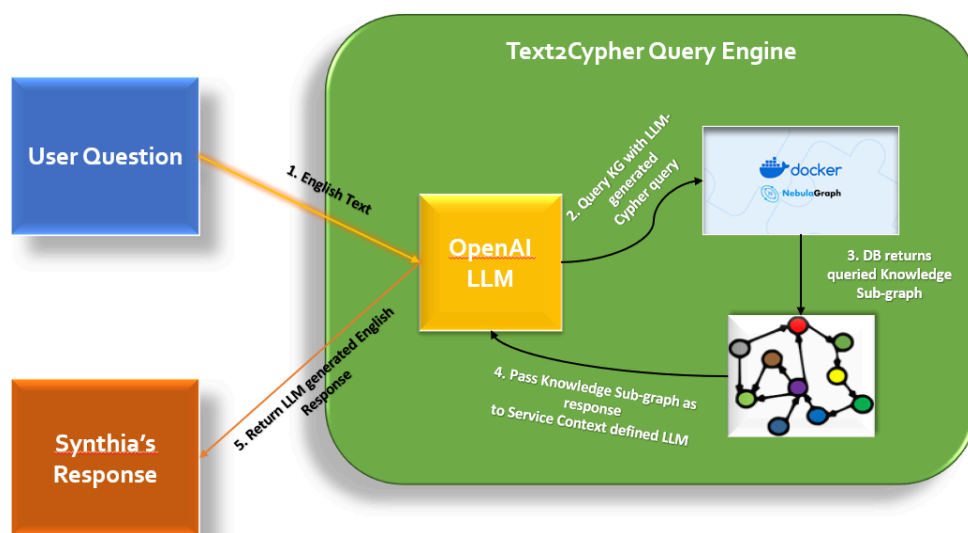
Note : Image by (Jacky, 2023)

4.1.7 Text2Cypher Query Engine

Text2Cypher Query Engine for Synthia is developed using following components: Service context defined LLM – GPT 3.5 Turbo, Llama Index’s Fine Tuned LLM, our fine-tuned Llama-2-7b-chat-finetune LLM and Nebula Graph DB with Docker. These components interact with each other to generate the response requested by the user. Synthia’s use gives the unstructured text as a question to the Text2Cypher query engine. This query engine passes the user question in natural language to GPT 3.5 Turbo which in turn converts this natural language text into a Cypher query which is later used to query the knowledge graphs inside Nebula Graph DB generated by fine tuned Llama Index LLM and our fine-tuned Llama-2-7b-chat-finetune LLM. Cypher query then parses the Knowledge graphs and Nebula Graph DB returns the required knowledge sub-graph to the Service Context LLM. This LLM finally converts the graph database returned result into natural language text and displays it to the user.

Figure 36

Text2Cypher Query Engine

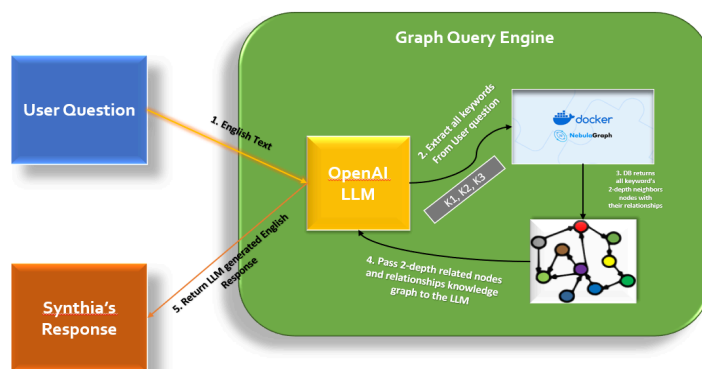


4.1.8 Graph Query Engine

Graph Query Engine for Synthia comprises of all the components employed for Text2Cypher Query Engine: Service context defined LLM – GPT 3.5 Turbo, Llama Index’s Fine Tuned LLM, our fine-tuned Llama-2-7b-chat-finetune LLM and Nebula Graph DB with Docker. This query engine receives user-input as natural language text, then extracts all the keywords out of user text with the Service Context defined LLM. These keywords are then passed to the extracted knowledge graph inside the Nebula Graph DB with the fine tuned LlamaIndex and Llama LLMs. These keywords are sent to the graph database as cypher query, additionally it is ensured with the cypher query that the resultant knowledge sub-graph constitutes of nodes and relationships requested, moreover the requested node’s 2-depth neighbors along with their relationships are also retrieved so as to fetch a deeper context of the information requested and enabling chain of logic reasoning. These results are again passed to Service context defined LLM as context and finally this LLM generates a natural language contextually rich response which is returned to the user.

Figure 37

Graph Query Engine

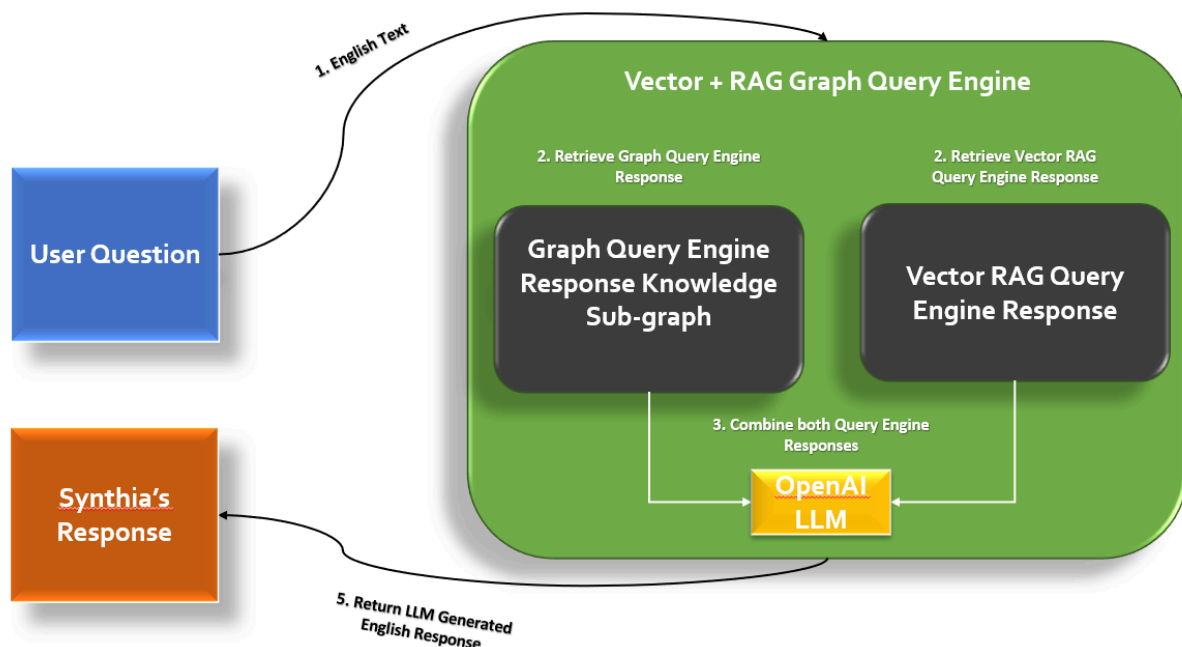


4.1.9 Graph + Vector RAG Query Engine

Graph Vector RAG Query Engine consists of all the components present in the Graph Query Engine and Vector RAG Query Engine. Here the user requested natural language information is passed to Graph Vector RAG Query Engine which queries both Knowledge graph Index and Vector Index to fetch the user-requested information. These results are combined and fed into the Service Context Defined LLM which receives the results as context and generates a result based on the context received. This query engine ensures that all the relationships lost because of chunking and semantically organizing the chunks in the vector database are retained, thereby it is expected to yield a contextually rich, detailed and good user response.

Figure 38

Vector + RAG Query Engine



4.2 Model Supports

The four proposed query engines require an efficient data analysis, machine learning platform, environment and tools to execute effective training, validation and deployment processes. These platforms and tools should be selected in a way they are able to handle large scale data, complex data processing and training. In this part the environment, tools, hardware resources required to implement these models along with their architecture and the dataflow would be discussed.

Table 6

Platform & Environment Requirements

Model	Hardware	RAM	SSD	Environment	Configuration
RAG Query Engine	GPU (NVIDIA)	16+ GB	100+ GB	Google Colab Pro Plus	Python 3.10
Text to Cypher Query Engine	GPU (NVIDIA)	16+ GB	100+ GB	Google Colab Pro Plus	Python 3.10
Graph Query Engine	GPU (NVIDIA)	16+ GB	100+ GB	Google Colab Pro Plus	Python 3.10
RAG Graph Query Engine	GPU (NVIDIA)	16+ GB	100+ GB	Google Colab Pro Plus	Python 3.10

The table outlines the specific platform and environment requirements for each model.

The hardware all the four models use is the NVIDIA GPU. The NVIDIA GPU supports the intensive operations these models would perform and also would retrieve the results at a faster rate which is crucial in training and validating each model. Sufficient RAM allocation is necessary to handle the large datasets and also the different parameters involved in developing these models. A 64+GB of RAM space is provided to each model ensuring the prevention of memory related errors. Also 100+GB of SSD storage space is necessary for each model to ensure sufficient space for capturing the outputs. Each model would be built in a Python environment on

Google Colab Pro Plus using the python version of 3.10. By proper allocation of platform and environment requirements, it ensures the prevention of any memory related bottlenecks and guarantees efficient model building.

Table 7

Tools/Libraries required for RAG Query Engine

Tools/Library	Method	Purpose
PDF loader	Document	Loads documents from the pdf files and extract the data from the PDF document.
TikToken	TikToken	To tokenize the texts in the PDF document. Helpful in knowing the token count before passing to LLM.
Text Splitter	RecursiveCharacterTxtSplitter	Split the texts into chunks.
Embedding model	LangChainEmbedding	Creating an embedded Vector from the text.
LLama Index Vector DB	VectorStoreIndex.from_documents	To store the embedded vectors and create an index around it.
GPT 3.5 Turbo	OpenAI	To generate textual responses from the query engine results.

Table 7 specifies the tools and libraries required for the RAG Query engine. This model initially requires a PDF loader to load the content from the PDF then a tiktoken to tokenize the texts from the pdf. This tokenization would be helpful in getting the token count. This token count would be useful in determining whether or not this count exceeds the LLM token limit. Next the texts from the pdf would be split into chunks using a text splitter and these texts would be converted into numerical vector embeddings using the embedding model. These embedded vectors are indexed and stored in a LLama Index vector DB. GPT 3.5 turbo is required to

generate textual responses from the results retrieved, these responses are sent to the user as an answer to their initial stated query.

Table 8

Tools/Libraries required for Text to Cypher Query Engine

Tools/Library	Method	Purpose
PDF loader	Document	Loads documents from the pdf files and extract the data from the PDF document.
TikToken	TikToken	To tokenize the texts in the PDF document. Helpful in knowing the token count before passing to LLM.
Text Splitter	RecursiveCharacterTextSplitter	Split the texts into chunks.
Fine tuned LLama Index LLM	KnowledgeGraphIndex.from_documents	To generate a knowledge graph from given chunks.
Fine tuned LLAMA 2 7b LLM	Llama-2-7b-chat-finetune	To generate a knowledge graph using the LLAMA 2 7B LLM
Knowledge Graph Index	-	To index the knowledge graph
Knowledge Graph Query Engine	KnowldgeGraphQueryEngine	Acts as an interface to connect the LLM with Nebula Graph DB
Docker Engine	-	Provides services to Nebula Graph DB
Nebula Graph DB	NebulaGraphStore	Query the knowledge graph to extract the output
GPT 3.5 Turbo	OpenAI	To generate textual responses from the query engine results.

This table 8 mentions the tools and libraries required for Text to Cypher Query Engine. Similar to RAG query engine, This query engine utilizes the PDF loader, Tiktoken and Text

splitter to load the document from the pdf documents and convert the texts to token to know the count of token before parsing to LLM and split the texts into chunks respectively. These chunks are then parsed into two LLM's, First is a fine tuned LLama Index LLM, this LLM generates the knowledge graph from the given chunks which are then indexed using Knowledge graph index and then passed into knowledge graph query engine which acts as the interface connecting LLM with Nebula Graph DB. Before parsing the knowledge graph into the Nebula Graph database, a docker engine is set up to provide the meta services, storage services, graph services to Nebula Graph DB. Then Nebula Graph is required to query the knowledge graph and extract the response of the query raised by the user. GPT 3.5 Turbo is required to generate textual response from the output response fetched by Nebula Graph DB. This complete process is repeated using the fine tuned LLAMA 2 7B LLM.

Table 9

Tools/Libraries required for Graph Query Engine

Tools/Library	Method	Purpose
PDF loader	Document	Loads documents from the pdf files and extract the data from the PDF document.
TikToken	TikToken	To tokenize the texts in the PDF document. Helpful in knowing the token count before passing to LLM.
Text Splitter	RecursiveCharacterTextSplitter	Split the texts into chunks.
Fine tuned LLama Index LLM	KnowledgeGraphIndex.from_documents	To generate a knowledge graph from given chunks.
Fine tuned LLAMA 2 7B LLM	Llama-2-7b-chat-finetune	To generate a knowledge graph using the LLAMA 2 7B LLM

Knowledge Graph Index	-	To index the knowledge graph
As Query Engine	kg_index.as_query_engine	Acts as an interface to connect the LLM with Nebula Graph DB
Docker Engine	-	Provides services to Nebula Graph DB
Nebula Graph DB	NebulaGraphStore	Query the knowledge graph to extract the output
GPT 3.5 Turbo	OpenAI	To generate textual responses from the query engine results.

The table 9 represents the tools and libraries required for Graph Query Engine. The tools and libraries required for execution of this query engine are similar to that of text to cypher query engine. The difference being the query engine which has been used. This query engine is used instead of knowledge graph query engine.

Table 10

Tools/Libraries required for RAG Graph Query Engine

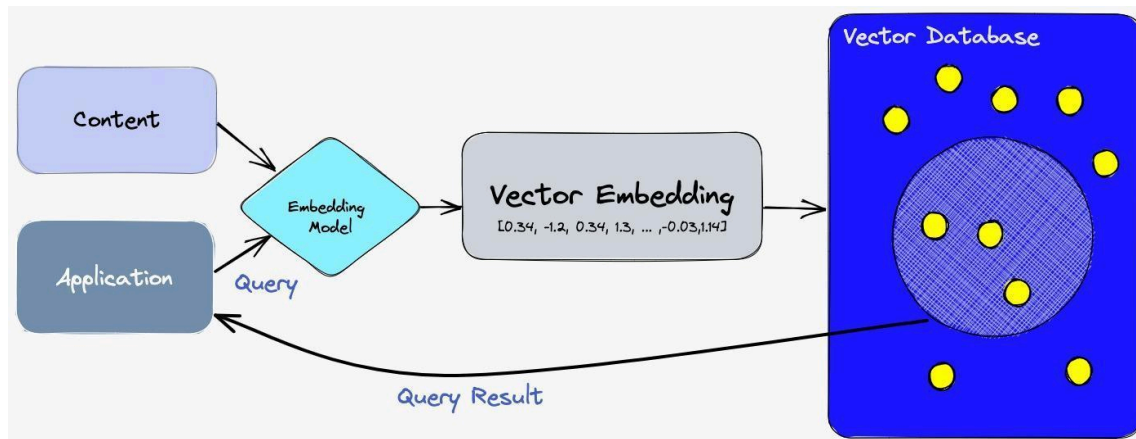
Tools/Library	Method	Purpose
PDF loader	Document	Loads documents from the pdf files and extract the data from the PDF document.
TikToken	tiktoken	To tokenize the texts in the PDF document. Helpful in knowing the token count before passing to LLM.
Text Splitter	RecursiveCharacterTextSplitter	Split the texts into chunks.
Embedding model	LangChainEmbedding	Creating an embedded Vector from the text.
LLama Index Vector DB	VectorStoreIndex.from_documents	To store the embedded vectors and create an index around it.

Fine tuned LLama Index LLM	KnowldgeGraphIndex.from_documents	To generate a knowledge graph from given chunks.
Fine tuned LLAMA 2 7B LLM	Llama-2-7b-chat-finetune	To generate a knowledge graph using the LLAMA 2 7B LLM
Knowledge Graph Index	-	To index the knowledge graph
As Query Engine	Kg_index.as_query_engine	Acts as an interface to connect the LLM with Nebula Graph DB
Docker Engine	-	Provides services to Nebula Graph DB
Nebula Graph DB	NebulaGraphStore	Query the knowledge graph to extract the output
GPT 3.5 Turbo	OpenAI	To generate textual responses from the combination of results generated by two query engines.

The table 10 represents the tools and platform required for implementation of RAG Graph Query Engine. This query engine is a combination of both RAG Query Engine and Graph Query Engine. In this the results retrieved from Graph Query Engine and RAG Query Engine are combinedly fed into GPT 3.5 turbo to contextually rich and highly precise response to the user.

Figure 39

Architecture of RAG Query Engine



Note : Image By (OxCosuinSY,2023)

Figure 39 shows the dataflow of the RAG query engine. Initially when a user posts a query using the synthia, The posted query will be indexed and converted into embeddings using an embedding model. The embedding model calculates the cosine similarity between the query embeddings and the data embeddings which are already stored in the vector database. Based on the similarity, The embeddings would return top-n chunks to the GPT 3.5 Turbo as a context, then the GPT will convert the chunks to textual response and will feed it as a result to the synthia application.

Figure 40

Architecture of Text to Cypher Query Engine

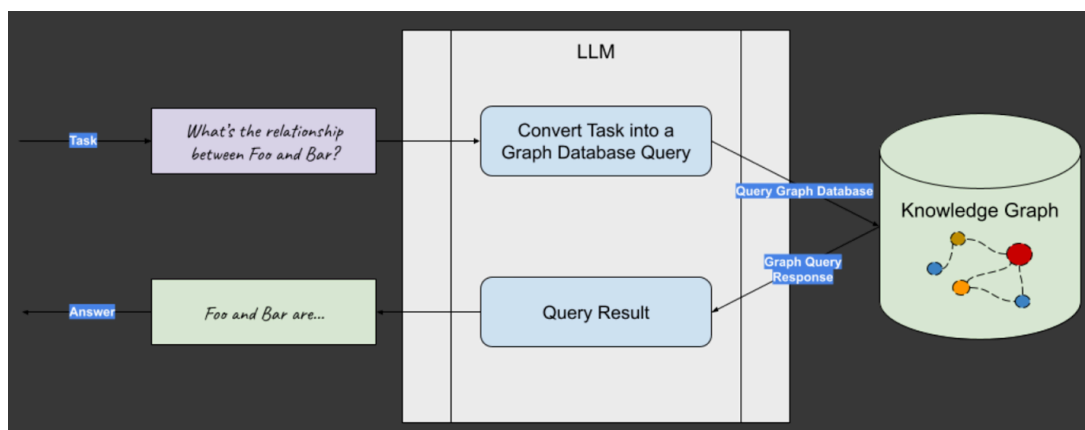


Figure 40 shows the dataflow of the Text to Cypher Query Engine. In this when initially the user raises a query pertaining to the pdf document such as “What's the relationship between Foo and Bar?” The LLM would convert the asked query into a cypher graph database query, This query would be sent into a Query Graph Database such as Nebula Graph Database. The query engine would retrieve the results through the knowledge graph generated using the Fine tuned LLAMA index LLM and Fine tuned LLAMA 2-7B LLM. The graph database would retrieve a query response, This response would be fed to a GPT 3.5 Turbo to change the query result into a natural human understandable language as shown in the figure like “Foo and Bar are...” which would be returned back to the user as a query response.

Figure 41

Architecture of Graph Query Engine

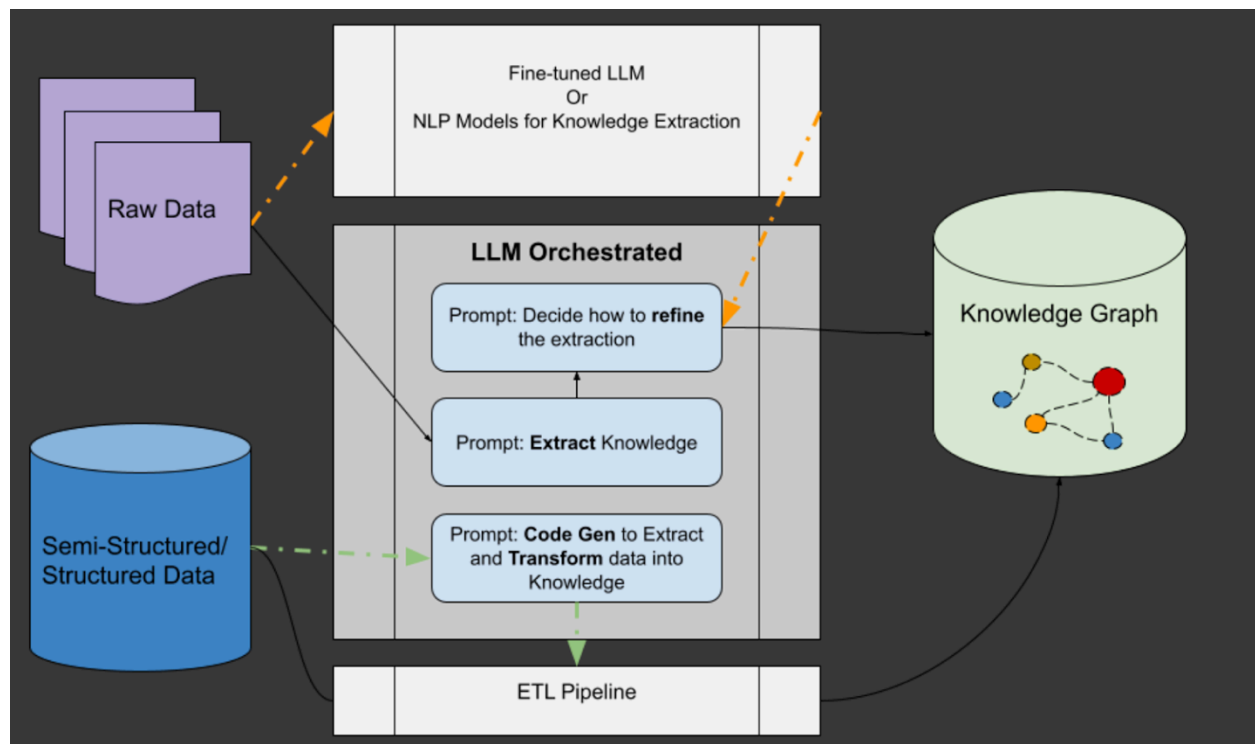


Figure 41 shows the dataflow in the Graph query engine. When a user posts a query on the synthia application, At first the posted query is converted into query embedding vectors using embedding models. The query embeddings are fed into a LLM to generate cypher query based on the query embeddings. These cypher queries are given to a nebula graph database to extract a sub knowledge graph from the main knowledge graph. Initially the main knowledge graph is devised from the MOA documents using the LLAMA index LLM and is stored inside the nebula graph database. This extracted sub knowledge graph is fed into GPT 3.5 turbo as context and using this context the GPT 3.5 would generate human textual output response which is returned to the user.

Figure 42

Architecture of RAG Graph Query Engine

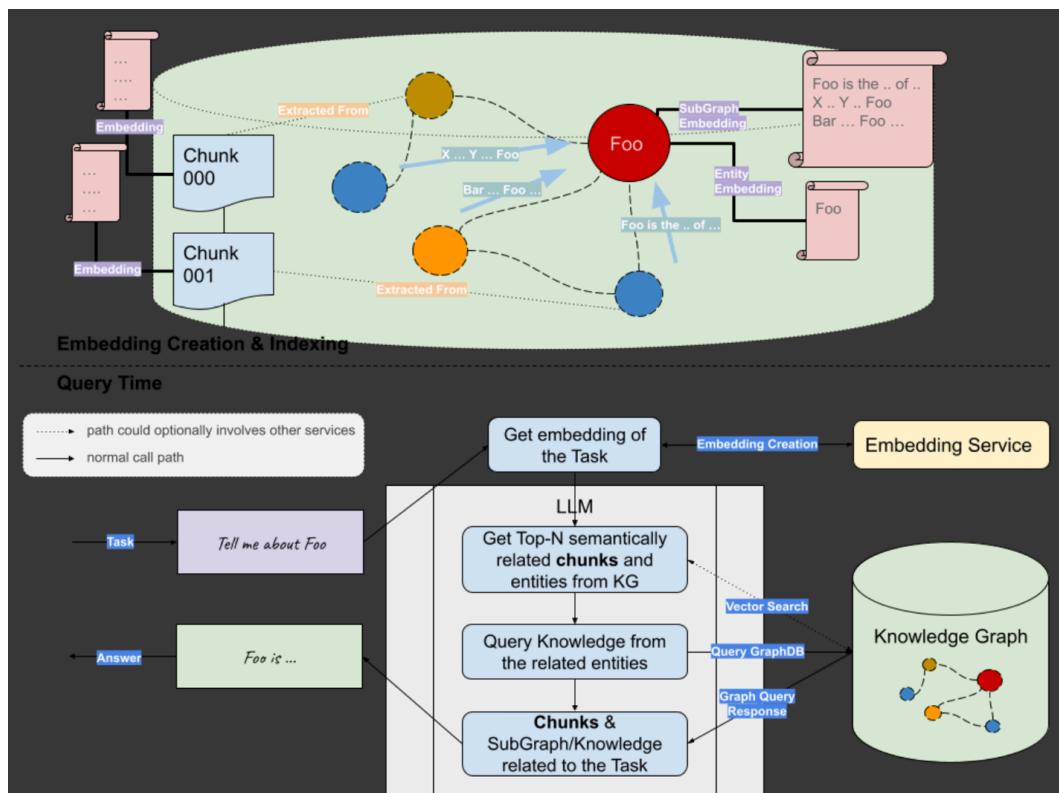


Figure 42 shows the dataflow of Graph RAG Query Engine, In this at first when users asks a query like “Tell me about Foo” using the synthia application, The posted query is converted into a vector embeddings using an embedding model, This query embeddings are compared with the data embeddings to retrieve a top-n semantically related chunks. Parallely the query embeddings are fed into a LLM to generate a cypher query from the user posted query, This cypher query is then fed into a graph database to extract a sub knowledge graph from the primary knowledge graph. The primary knowledge graph is generated using the Fine tuned Llama Index LLM from the web scrapped MOA documents and stored inside the nebula graph database. This sub knowledge graph is given into a GPT 3.5 Turbo to retrieve query knowledge. This query knowledge along with top-n semantically generated chunks are given as a context to LLM which returns the contextually rich response like “Foo is...” to the user query and this is fed back to the application and the user.

4.3 Model Comparison and Justification

We have used two pre-trained Large Language Models (LLMs): Llama2 and OpenAI GPT 3.5-turbo, in our project to create Synthia, an advanced Generative AI chatbot. Llama2 is used for Knowledge Graph Extraction, and GPT 3.5-turbo is critical for producing user replies in the Service Context. The implementation is mainly divided into two phases: first, a Knowledge Graph is extracted from unstructured text, and then different Query Engines are developed to make use of this Knowledge Graph. To decide which of our four proposed Query Engines is best for deployment within Synthia, we will compare their results.

4.3.1 Finetuned Llama Index LLM and Fine-tuned Llama-2-7b-chat-finetuned model in Knowledge Graph Extraction

The ability of Llama2 to create Knowledge Graphs from text has revolutionized the Large Language Model (LLM). This model is specially optimized for this task and performs exceptionally well in generating RDF triples, which are necessary for giving contextually rich information and logical reasoning.

Table 11

Advantages and Disadvantages of Llama2 in Knowledge Graph Extraction

Advantages	Disadvantages
Llama2 stands out because it can comprehend context in RDF triple format, providing a more nuanced comprehension of the data.	Because there are so many parameters in Llama2, it is still computationally demanding even with the improvements in efficiency.
Because of its fine-tuning, querying may be done based on entities, which increases the retrieval depth and flexibility of information.	There is a natural risk that the knowledge representation will propagate preexisting biases because it is trained on internet data.

4.3.2 GPT 3.5 Turbo in Service Context

Within the Service Context, GPT 3.5-turbo is a flexible tool meant to work with Knowledge Graphs to generate responses. It is essential for converting user inquiries into logical responses via a variety of Query Engines, demonstrating how versatile this LLM is.

Table 12

Advantages and Disadvantages of GPT 3.5 Turbo in Service Context

Advantages	Disadvantages
The flexibility of GPT 3.5-turbo is demonstrated by its application to various query engines.	Similar to Llama2, GPT 3.5-turbo impacts scalability.

It is well-known for producing text that reads like human writing, which improves communication between Synthia and its users.	The model's intricacy could make it difficult to understand how decisions are made, which would reduce the transparency of the model.
--	---

4.3.3 Query Engines in Synthia

Each of the four suggested query engines has a special strength, and they are made to work well together with Fine-tuned Llama Index LLM, Fine-tuned Llama-2 LLM and GPT 3.5-turbo to provide sophisticated answers to user queries.

4.3.3.1 RAG Query Engine

Table 13

RAG Query Engine Advantages and Disadvantages

Advantages	Disadvantages
Provides contextually relevant data for GPT 3.5 Turbo's responses using vector embeddings.	Because it only uses cosine similarity measures, it might not be able to capture subtleties in complex relationships.
Able to measure similarity to retrieve information from a large corpus, improving the relevance of generated responses	

Because of its highly scalable vector-based architecture, expanding the database doesn't result in a linear increase in query time.The engine is built to maximize relevance, giving GPT 3.5 Turbo the most relevant data to guide its reactions.The RAG engine's capacity to offer a wide range of context makes it well-suited for general queries where user intent is broad or multifaceted, despite possible difficulties in capturing the subtleties of complex relationships.

4.3.3.2 Text2Cypher Query Engine

One notable feature of the Text2Cypher Query Engine is its straightforward method of translating natural language questions into Cypher queries that can communicate with Nebula

Graph Databases. Because of its directness, queries derived from Knowledge Graphs have the potential to be highly accurate in terms of precision and efficiency.

Table 14

Text2Cypher Query Engine Advantages and Disadvantages

Advantages	Disadvantages
Converts natural language requests into Cypher queries to allow for direct communication with Knowledge Graphs	It requires a thorough comprehension of the Knowledge Graph's architecture.
Enables accurate information retrieval by enabling precise querying of the Knowledge Graphs within Nebula Graph DB	The intricacy involved in creating a Cypher query may result in increased learning curves for developers.

4.3.3.3 Graph Query Engine

Table 15

Text2Cypher Query Engine Advantages and Disadvantages

Advantages	Disadvantages
Keyword extraction focuses the search on relevant information in the Knowledge Graph	Restricted by the identification and relevancy of keywords, which may omit more extensive context.
Makes use of knowledge graphs to deliver comprehensive answers with logical flow	The accuracy of responses depends on the quality of keyword extraction
By altering the depth of nearby nodes and relationships questioned, the depth of responses can be managed.	

The Knowledge Graphs are navigated by the Graph Query Engine using keyword extraction to provide comprehensive and coherent answers. By ensuring that the engine concentrates on the most pertinent nodes and relationships within the graph, this method may provide a level of insight that is not possible with other engines

It works especially well for targeted, specific queries because of its keyword-driven mechanism, which can deliver a highly focused response. Controlling the response's depth enables customization to meet user requirements, whether a thorough investigation or a more comprehensive overview is needed. Although the quality of keyword extraction affects the engine's efficacy, this can be lessened over time by constant optimization and learning from user interactions.

4.3.3.4 Graph + Vector RAG Query Engine

Table 16

Graph + Vector RAG Query Engine Advantages and Disadvantages

Advantages	Disadvantages
Combines the benefits of the RAG and Graph engines to provide a result that is more thorough.	The total complexity and failure points grow when two complicated systems are integrated. possible sources of failure
It strives to offer a thorough and well contextualized response	It is more computationally demanding and resource-intensive
Specifically designed to preserve relational data that vector databases might lose during chunking.	

The goal of this hybrid strategy is to guarantee robust relational information and rich context in addition to providing precise answers. This engine is made to collect both the breadth and depth of data, providing a well-rounded and nuanced response, by combining the Graph and Vector RAG approaches. It is designed particularly to offset any potential loss of relational information during the vectorization process, giving a more comprehensive response.

4.4 Model Evaluation Methods

4.4.1 *A Comprehensive Approach for Assessing Model Performance*

As the field of Large Language Models (LLMs) continues to progress it becomes increasingly evident how crucial it is to have methodologies for evaluating models. Evaluating the performance of language models is essential for deploying them in real world applications. In this endeavor we explore an approach that combines ADA similarity with GPT, for model evaluation purposes.

We start our evaluation process by training the dataset. We provide a set of questions and their corresponding answers to train the model. The model learns to predict answers based on the given input. These predicted answers are then passed through a language model, which's a tuned version of GPT.

To measure how well the model responses align, with the expected outcomes we use ADA similarity as a metric for evaluating the similarity between answers. By assigning scores out of 5 for each question answer pair we can calculate the accuracy. Assess the performance of the model.

During our evaluation using COQA OpenAI Evals we categorize cases based on criteria. One category focuses on cases where the model's response goes beyond what was expected in terms of comprehensiveness. Another category examines situations where the predicted answer partially matches or closely resembles the answer. Additionally we analyze instances where there is deviation, between the model's response and what was expected.

To ensure an evaluation we use a Likert scale with four response categories; Agree, Disagree, Partially Agree and Partially Disagree. Each model is assessed through LIKERT scale ratings, in categories, which provides an understanding of their strengths and weaknesses.

In conclusion, our evaluation approach combines ADA similarity with GPT. Moreover, incorporating COQA OpenAI Evals along with a Likert scale. This comprehensive strategy allows us to present an assessment of the model's performance. By going beyond accuracy metrics we gain insights into the subtle nuances of language generation models. As the field of LLM advances it becomes increasingly important to have evaluation methodologies that ensure the reliability and effectiveness of language models, across applications.

5. Data Analytics System

5.1 System Requirements Analysis

5.1.1 System Boundary, Actors and Use Cases

The figure 43 demonstrates the boundaries, actors and use cases. For the proposed solution, the system's boundary includes the prompts/queries asked by the user and the backend query engines that help generate accurate responses after processing the queries. The actors encompasses people in the city of San Jose who search for Memorandum of Agreements(MoAs) that describe the individual's responsibilities towards the city. The main intention of creating Synthia was to make the users feel at ease by interacting instead of browsing the internet. Firstly, the user inputs a question into the chatbox. The text is then processed further into the trained query engine to generate appropriate responses. This system functions as an AI assistant, gifted with the ability to interact with users by creating answers that mimic human conversation closely. The fundamental AI structure hones its skills through the use of a considerable body of text data, its efficacy increased by employing high-powered Graphics Processing Units (GPUs) for improved accuracy and effectiveness. The system offers users a broad spectrum of inquiry formats, covering open-ended questions, multiple-choice questions, true/false queries, fill-in-the-blank tasks, and a wide variety of other question styles. The final AI model is selected

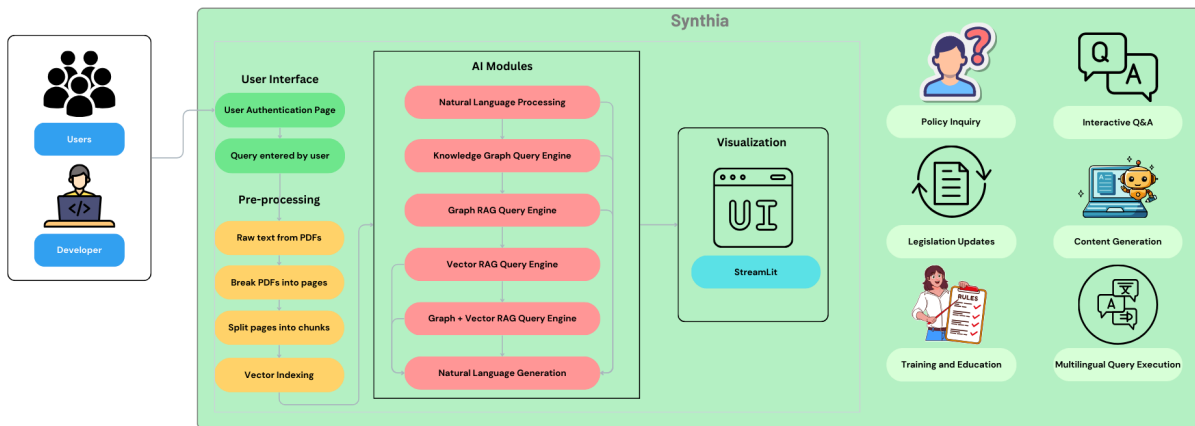
from an array of query engines namely Knowledge Graph Query Engine, Graph RAG Query Engine, Vector RAG Query Engine and Graph + Vector RAG query engine where the model with highest evaluation score will be selected. The user interface contains a typical chatbox for taking the user input. The model will then hit back to the user interface with the appropriate responses. After leaving the user interface, the user's query undergoes further pre-processing on the back end, which undergoes certain operations with the database where raw text which will be the input will be transformed into a suitable format. The below illustrate the use cases related to our project:

- **Policy Inquiry :** A user with a query regarding a specific labor policy in the city of San Jose. The main functionality of this chatbot is that it will analyze the user's query, extract the relevant information from the Memorandum of Agreements (MoAs) and will provide detailed and accurate responses about the requested labor policies asked.
- **Legislation Update :** A user is interested in staying informed about recent federal laws affecting San Jose. The chatbot regularly monitors and updates users on any changes in federal labor laws impacting the city. It can provide summaries, key points, and links to official documents for further reference.
- **Training and Education :** This happens when a user is interested to learn about labor laws for educational purposes. The chatbot can offer structured lessons by utilizing the knowledge graphs to present information in a coherent and educational manner.
- **Legal terminology explanation :** This happens when the user is unfamiliar with legal terms used in labor policies. This enables the users with various levels of expertise can comprehend the policies with ease.

- **Employee rights :** This scenario arises when an employee seeks information about their rights in the workplace. The chatbot provides accurate responses including minimum wage, working hours and safety regulations.
- **Interactive Q&A Sessions :** Users generally require a conversational approach to learn about the laws.

Figure 43

System Boundary



5.1.2 High-level Data Analytics and Machine Learning Functions and Capabilities

The high level data analytics and machine learning capabilities of Synthia are mentioned below

- **Content Generation:** Synthia due to its diverse query engines is capable of generating texts in natural language and retrieve contextual based responses to the user posted queries. Synthia can be used to generate textual responses there by it can help users with writing articles, answering questions etc.

- **Natural Language Understanding:** Synthia's powered by LLMs, GPT 3.5-turbo and Llama2 7B which equips it with the ability to understand natural language. Approaches such as tokenization, chunkings, word embeddings, defining background context, Vector RAG Implementation and fine-tuning, rendering naturally rich human-like responses to the user questions.
- **Text Summarization:** Synthia's GPT3.5-turbo transformer architecture and attention-mechanism enables it to generate a brief summary of text of user-requested topics present in the MOA documents published by the City of San Jose..

5.2 System Design

5.2.1 System architecture and infrastructure with AI-powered function components

As shown in the Figure 44, System Architecture Diagram depicts how Synthia's components are connected, relationships existing between them, illustrating how these key elements work together to achieve the Conversational AI Capabilities. This research work proposes primarily 3 major components namely, User Interface, Backend and Cloud Services involved. User Interface is the component which facilitates the interaction between User/Actor (external to system environment) and the System. A series of User Prompts encompassing questions based on MOAs published by the City of San Jose are passed to backend through User Interface (UI) and Query Engine APIs. This UI Design endows Users with a seamless and intuitive LLM-powered conversational mechanism.

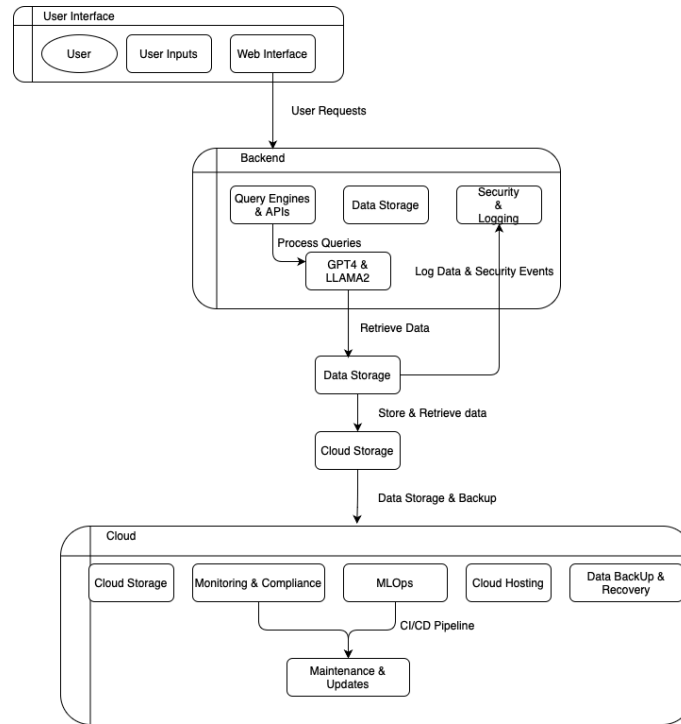
The backend layer handles all the business logic and constitutes of all four Query Engines made available to the UI with Solution APIs deployed with FastAPI, Knowledge Graph Data Storage, Security & Logging Components flagging harmful user requests and Synthia's responses and ultimately the LLMs employed, GPT3.5-turbo and Llama2 in the implementation

of Synthia. Client Requests are passed to the Query Engines which interact with the core LLM – GPT3.5-turbo, pass the remodeled natural language English response back to the user through the UI component. GPT4 is the principal unit operating in this layer which comprehends the English language user-request, converts it into Cypher to query the extracted Knowledge Graph using Fine-tuned Llama Index LLM and Fine-tuned Llama2 7B parameters model, derives the relevant knowledge sub-graph and responds back to the UI with FastAPI response. Security & Logging Component is one of the paramount concerns in any of the LLM driven applications. All the user-questions and Synthia-generated replies will be flagged if any inappropriate content is found.

Cloud Layer in conjunction with the backend layer enables scalability to the application. This integration plays a pivotal role substantiating Synthia in efficiently handling increasing workloads without compromising the performance of the system. Additionally, this layer also consists of Monitoring and Compliance, Cloud Hosting of Application, MLOps and ultimately Maintenance & Updates.

Figure 44

Synthia's System Architecture



5.2.2 System supporting platforms, frameworks, and a cloud environment

Synthia's implementation, development, fine-tuning and deployment is achieved with the supporting platforms, and cloud environment as depicted in Figure 45. Hugging Face, Tensorflow and PyTorch frameworks when used in conjunction with Google Colab environment can be exploited to take advantage of the Virtual A100 GPU resources for fine-tuning Llama2 to extract the Knowledge Graph from the MOAs published by the City of San Jose. This environment can be used for incessant improvement in the fine-tuning process of Llama2 7B Model. After successful fine-tuning of the Llama2 7B LLM, MOA documents are fed to the fine-tuned LLM to extract the knowledge graph and upsert into Nebula Graph DB. To upsert into Nebula Graph DB, a successful connection needs to be established from Google Colab. OpenAI's GPT3.5-turbo is one of the integral aspects of Synthia's Implementation. This is LLM

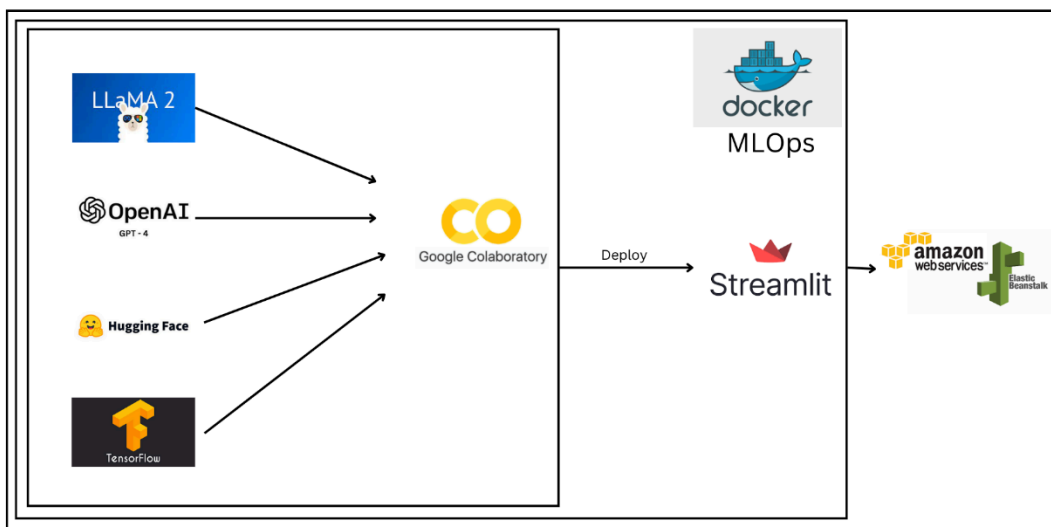
which processes user-requests, queries the knowledge graph and returns the response back to the user.

Streamlit is an Open-source Python library which simplifies the development of UI allowing developers to focus more on business logic contrary to the effort in developing a presentation layer for an application implementing machine learning and data science. Streamlit is as easy as creating Web Application with few lines of code in Python. This web-app can easily be deployed and is sharable or hosted in a Cloud Environment.

Holistic ecosystem consisting of Elastic Beanstalk, Elastic Cloud Compute (EC2) with a Docker Container hosting Streamlit application will be implemented and put to use for MLOPs purposes (CI/CD, Maintenance & Compliance of Synthia). This approach encompassing diverse frameworks, supported platforms and suggested cloud environments in the implementation of Synthia epitomizes development of LLM-powered applications.

Figure 45

Supported platforms, frameworks, and a cloud environment



5.2.3 Data Management Solution and Database

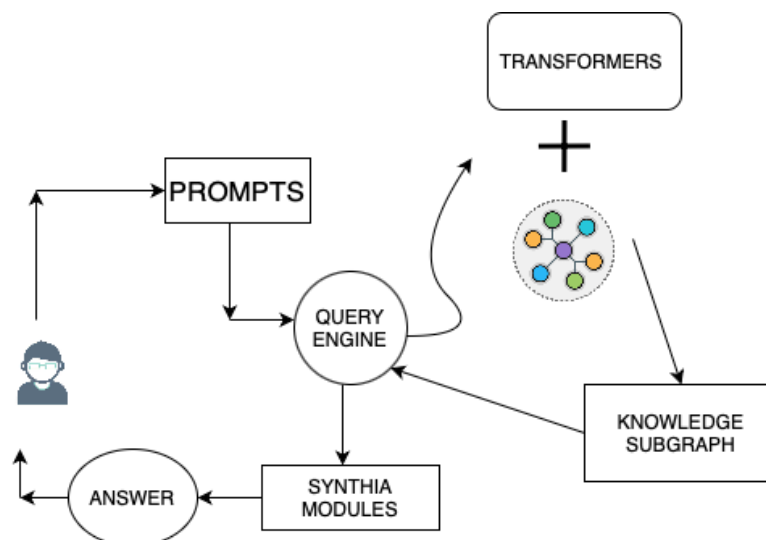
As shown in Figure 46, data flow chart portrays the way Synthia synthesizes user-prompts, questions about MOAs published by City of San Jose and produce responses back to the user. All user-prompts and synthia-responses are passed through Content-Moderation API by OpenAI. Each User-prompt is converted to cypher query by GPT3.5-turbo, the core LLM in Synthia which parses the knowledge graph based on the query engine modelling type and a knowledge sub-graph is returned.

This sub-graph is then combined with the pre-defined context or the system message fed to the core LLM to generate naturally rich context-specific responses. Finally, Synthia's UI Module displays the response, enabling a natural conversation with the user. Comprehensive data flow and interactions with Synthia is illustrated in Figure 46.

All logs generated by Synthia will be stored in Google Cloud Platform's Google Cloud Storage Object and Synthia's data corresponding to Knowledge Graph will be stored in Nebula Graph DB.

Figure 46

Data Flow Chart from User-Request to Synthia's Response



5.2.4 Design User Interface and Data Visualization

User Interface for Synthia will be developed with Streamlit which enables rapid prototyping. This web application will allow users to select the query engine and enter questions in the input text box provided. Input Text will be validated for Content Moderation. All past conversations would be stored to reload and continue previous conversation at any point in the future.

5.3 Intelligent Solution

5.3.1 AI and Machine Learning Model development

This project aims to develop a chat bot Synthia, which leverages LLM capabilities and knowledge graph to retrieve contextually rich responses to the user posted queries from the unstructured PDF documents. In order to do this, We develop four query engines using the publicly available MOA data from the official webpage of the city of san jose. Initially the data is web scraped and preprocessed, such as removing stop words, unnecessary chunks to formulate an optimal dataset to form a comprehensive knowledge graph consisting of all the entities and relationships between them. The target is to make the LLM domain expert in the MOA such that it would respond with semantically rich answers to the user posted MOA related queries.

LLAMA-2 7B is used to generate a comprehensive knowledge graph using the data chunks. LLAMA -2 7B is a large language model developed by meta pre-trained on 7 billion parameters. PyTorch is used for training Llama Index. PyTorch is an open source machine learning library of meta. PyTorch is preferred due to its dynamic computation of graph and ease of use for training and deployment of the model.

The hugging faces, which is an open source community, is extensively used in this project for its ease to use libraries, tokenizers and datasets. Synthia primarily uses the hugging

faces transformer library in the process of knowledge graph extraction and in the building of four query engines. The transformer library simplifies the process of using pre-trained models, optimizers and integrating them into applications. In this project the transformer library is used to import modules such as `AutoModelForCasualLM`, a class used for loading pre-trained LLM, `AutoTokenizer`, `BitsAndBytesConfig` which is another class for configuring models which use bits and byte embeddings, `HfArgumentParser` a module for parsing command lines, `TrainingArgument` a class used for configuring prompts for fine-tuning and logging a module setting logs for transformers library. All these are used to simplify the process of knowledge graph extraction.

Apart from the hugging face libraries, Libraries such as Parameter Efficient fine tuning(PEFT) and Supervised Fine Tuning trainer(SFTT) are used to efficiently adapt the pre-trained models to applications without necessarily fine tuning all of the model parameters using its modules such as Low-Rank-Adaptation of LLM(LoRA). All these libraries and modules are imported and set to specific values and the Llama index is trained for a number of epochs to learn and generate a knowledge graph and later the LLM is finetuned to generate a comprehensive knowledge graph. TensorBoard, which is an extension of TensorFlow, is used to visualize the training results of different sets of parameters.

Llama index provides several libraries which ease the process of building different query engines, such as `Importing GraphStores`, `KnowledgeGraphQueryEngine` modules. The LLama Index provides `GraphStore` library which facilitates the NebulaDB integration with the LLM thereby allowing the storage of Knowledge graph directly into Nebula DB and also acts as Storage context for the LLM. Also the `KnowledgeGraphQueryEngine` module assists in querying the knowledge graph using natural language instead of using Cypher thereby eliminating the

need for domain-specific knowledge of Cypher. For NL2Graph RAG with Text2Cypher Query engine, Llama index provides KnowledgeGraphRAGRetriever library which generates the graph query based on the user query and the schema of the knowledge graph, Thus providing contextually rich responses to the user queries.

Also during cleaning and preprocessing the dataset several langchain libraries are used. A langchain which is an open source orchestration framework for development of applications of LLM. Different types of Langchain libraries such as PyPDFloader, Character text splitter and prompt template are used for loading the documents,splitting the chunks and preprocessing them.

5.3.2 Input and Output Requirements, Support Systems and Solution APIs

- **Input Datasets:** The input data will be questions and queries posted by the user relating to the Memorandum of Agreement(MOA) documents between the labor unions and the government.
- **Expected Outputs:** The expected output would be a semantically rich and context based response to the user posted query. The project targets to build an user interface which allows users to have a conversation with the LLM to get tailor based responses to their queries using different query engines. Synthia would be useful in overcoming the failure to capture relationships between the entities in unstructured documents which conventional practices might fail. Also Synthia could open up the possibility of having a personalized chatbot for a specific domain by having a domain expertised LLM and receive contextually based responses to queries.
- **Supporting Systems:** Synthia relies largely on the hugging face libraries such as Transformers to generate a comprehensive knowledge graph, Google colab pro is required to train and build the knowledge graph and different query engines.

- **Solution APIs:** To enable faster user query response and to enhance Synthia performance FasterAPI is integrated into this project. FastAPI is a web framework for building APIs with Python. FastAPI will be used to create RESTful endpoints which the user interface uses to submit user queries. FastAPI will take these requests, process them to LLM and query engines and retrieve the LLM responses back to the interface and to the user. FastAPI also helps in improving the scalability of Synthia by permitting it to handle multiple requests efficiently.

5.4 System Support Environment

Synthia requires the use of various environments to support its functioning and storage mechanism along with its deployment. Google Colab pro is used to run the Synthia model to utilize its GPU capabilities, Google drive is used for storage of all the training, validation and testing datasets. GitHub acts as the project's code repository which is used to track the project completion progress. JIRA is used to track the tasks and their completion. Hugging face is used for importing necessary model implementation libraries. AWS Sagemaker and flask are used for model deployment. Lastly, Draw.io is used for creating charts and functional diagrams.

HuggingFaces

Hugging Faces is a crucial aspect of this project, Hugging faces provide several important libraries like transformers which aids in tokenization, for loading pre-trained LLM and also in configuring models with the embeddings thereby simplifying the task of knowledge graph extraction.

Google Colab & Google Drive

Google Colab premium is used to train the various query engines to overcome the difficulties of computation occurring due to the local systems computing capabilities. V100 GPU

is used which significantly enhances the training speed of the query engines. At the same time, we stored our training, validation, and testing data in Google Drive. This fusion of Colab and Drive facilitated improved collaboration and simplified the process of creating model ensembles.

GitHub Repository

A common GitHub repository is established with access to all the team members which stores all of the data preparation, understanding, web scraping and query engine codes branched by different team members. Also this repository aids us in maintaining version history allowing us to rollback to previous versions if required. Github repository provides us with a history of version changes made and helps us in keeping a track of the tasks progress.

Deployment

We are planning to build the user interface using Streamlit and containerize the web interface in a docker container along with the FastAPI service. Then AWS Elastic Beanstalk is used to create a new application , upload the docker container, configure the environment variables and also is used to simplify the process of deploying and managing web applications and services in AWS cloud. Elastic Beanstalk can also be used to validate the deployment and also acts as Load balancer. Logs in elastic beanstalk are used to capture application logs. These logs can be forwarded to CloudWatch for monitoring and alerting. Through this way we plan to deploy synthia in a robust and scalable manner.

Jira

JIRA, A project management tool is used to assign the tasks, subtasks to each team member, Also to create gantt chart and WBS chart for the project. These Gantt and WBS charts help the team to stay up to date with the progress of minute subtasks along with the critical tasks and also aids the team in on-time task completion.

Draw.io

Draw.io is used to craft various architecture diagrams, project flow diagrams along with flowchart, these help in functionality explanation of the project.

6 System Evaluation and Visualization

6.1 Analysis of model execution and evaluation results

6.1.1 Evaluation of the model output with the targeted/labeled targets

In this section, we are going to discuss the evaluation of our prototype Synthia which is a GPT that is trained exclusively for the city of San Jose where users provide any query related to the memorandum of agreements specified on the city of San Jose website. Our prototype uses 33 different documents further broken down into 1593 pages to segregate all of our data into one. During this prototyping stage, Synthia's viability and potential as a base for creating the finished product were evaluated.

The dataset was pre-processed using langchain tokenizers for breaking the 1593 into a number of chunks as the LLM has a certain limit for processing the input, where the chunk size was set to 400 and chunks were converted into vector embeddings and triplets because knowledge graphs require entities and relationships. It's important to remember that this prototype was optimized by this point and the results shown were considered to be the best ones so far after comprehensive trials.

Although the training process was time and computationally intensive as we found it very difficult for converting the responses generated into a knowledge graph as the graph database required Docker Desktop for creating its instance in a virtual environment and was a tedious process during this phase. Ultimately, our primary goal was to make our LLM generate accurate and useful responses as requested by the users in their prompt.

Comprehensive evaluation criteria were involved for evaluating the response generated by our 4 query engines .ie. Text2Cypher, Graph Query Engine with Extracted Keywords, GraphRAG + Text2Cypher, and Graph RAG +Vector Index , where our assumptions were that the Graph+Vector RAG query engines would generate better responses compared to others as we are combining two of the most powerful query engines to perform as one and would be helpful for the users of the City of San Jose to get efficient information based on their queries and some of the evaluation metrics considered for our prototype were BLEU score, semantic similarity score and F1 score.

During the evaluation phase, we performed a plethora of metrics which suit our use case and we finalized the BLEU score, semantic similarity and F1 score to be the most suitable ones among all of those. We encountered certain limitations that affected the quality of the results in the initial phases of the evaluation process. The dataset's wide range of perplexity has made it difficult for us to come up with a conclusion about the generated results.

Despite these drawbacks, Synthia has shown satisfactory results. It demonstrated the capacity to produce persistent responses to a range of city laws and regulations that the users might want to know. The model occasionally produced inaccurate and inefficient results in a very few circumstances which arose a need for further development. In order to enhance the performance of the model going forward, our focus would be on finding the solutions to those issues encountered during the implementation phase.

In conclusion, the prototype Synthia for the City of San Jose on the memorandum of agreements data that provide valuable insights into the feasibility of developing the final product altogether. The results of this assessment will direct future development and improvement work

to develop a precise and effective question answering system customized for the City of San Jose.

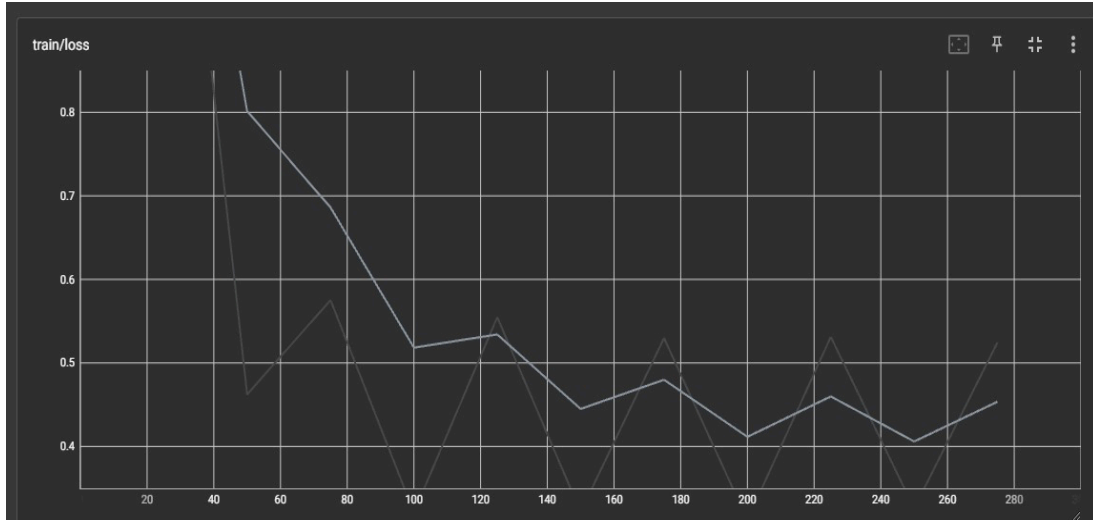
Figure 47

The figure below shows a table of the training loss over step size of 25.

Step	Training Loss
25	1.366800
50	0.462200
75	0.575300
100	0.321100
125	0.554700
150	0.321700
175	0.529800
200	0.311900
225	0.531200
250	0.325900
275	0.524700

Figure 48

Training loss over epochs



Our training procedure was investigated by instantiating the training loss for over one epoch in different steps. Figure 48 above graphically demonstrates the training loss over a particular period.

This procedure measures how well the model has been influenced by the training data. So the lower the loss, the higher will be the accuracy. In our case, there has been fluctuations on the training loss curve that might be due to less computational power.

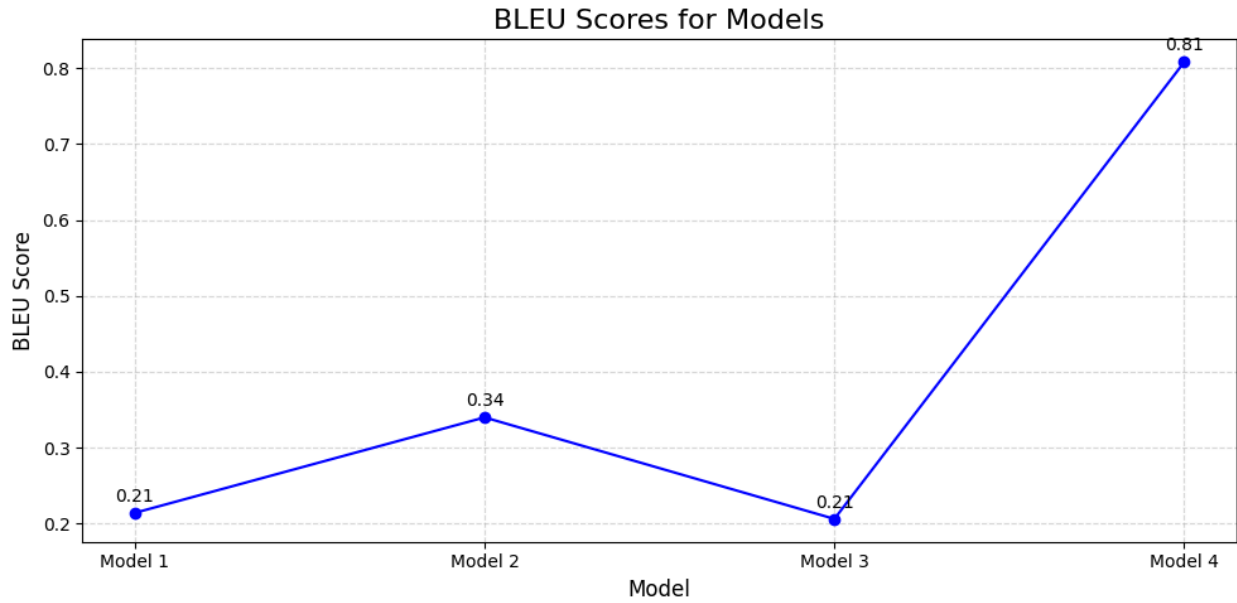
As a starting point in the training dynamics in our early prototyping stages the model, we have faced certain difficulties to figure out the data analysis. This visualization helps us to evaluate the model's learning progress and offers suggestions for future enhancements or modifications to improve our model's accuracy.

6.1.2 Methodology of Measuring Evaluation Metrics

Graphical Analysis

Figure 49

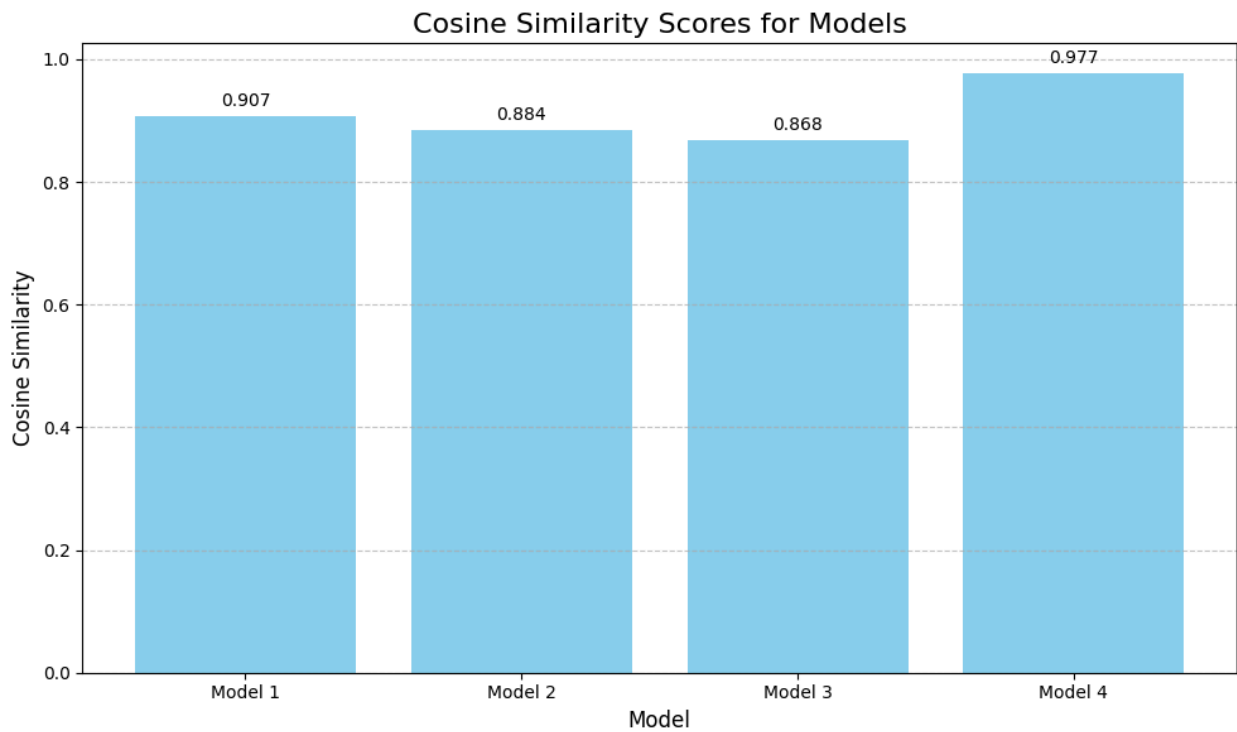
Line Chart demonstrating the BLEU scores for each model



As mentioned earlier, according to our assumptions, the model Graph RAG + Vector has outperformed all the other models and stands out in its efficiency.

Figure 50

Bar Chart that demonstrates the Semantic Similarity scores for all the 4 models



The bar chart above demonstrates the semantic similarity scores between our models and also here, the model Graph RAG + Vector query engine performed the best.

Table 17

Results Comparing all the models using all of our evaluation metrics

Metric	Precision			Recall			F1		
	rouge1	rouge2	rougeL	rouge1	rouge2	rougeL	rouge1	rouge2	rougeL
Model									
Model 1	0.41	0.26	0.37	0.49	0.31	0.44	0.45	0.29	0.41
Model 2	0.65	0.39	0.57	0.39	0.23	0.34	0.48	0.29	0.42
Model 3	0.69	0.48	0.61	0.30	0.20	0.26	0.42	0.29	0.37
Model 4	0.83	0.64	0.74	0.78	0.60	0.69	0.80	0.62	0.71

6.2 Achievements and Constraints

6.2.1 Achievements of Solving the Target Problems

(a) Exploration and Extraction of MOA data

- Web scrapped 20+ MOA documents from the official webpage of City of San Jose.
- Scripts were designed to accommodate new data sources and can be easily extended to scrape additional web pages as needed.
- Data statistics were performed on the raw data such as knowing the count of words in each MOA document.

- Data consistency was achieved by making sure that all of the data in the documents was in one common language. A language check was performed to verify language.
- Data consistency was obtained by removing filler words and special characters which cannot be read by langchain libraries and only maintaining the important information. Also a data quality report is performed to verify the quality of data after transformation.
- To overcome the constraints imposed by Large Language Models, The MOA documents undergo tokenization.

Table 18

Data Statistics

Data Collection Methodology	Count of Files	Total Data points	Overall Size
Web Scraping from website using Custom Python Script	20	2833	1.5GB

Figure 52

A snippet showing the scrapped MOA files

 Article 39 - Exempt Officers and Sergeants Modified Duty Program	11/10/2023 2:59 AM	Microsoft Edge P...	131 KB
 Association of Building Mechanical and Electrical Inspectors (ABMEI) MOA	11/10/2023 2:58 AM	Microsoft Edge P...	801 KB
 Association of Engineers and Architects IFPTE Local 21 Unit 43 MOA	11/10/2023 2:58 AM	Microsoft Edge P...	719 KB
 Association of Engineers and Architects IFTPE Local 21 Units 4142 MOA	11/10/2023 2:58 AM	Microsoft Edge P...	696 KB
 Association of Legal Professionals of San Jose (ALP)	11/10/2023 2:58 AM	Microsoft Edge P...	867 KB
 Association of Maintenance Supervisory Personnel IFPTE Local 21 (AMSP) MOA	11/10/2023 2:58 AM	Microsoft Edge P...	743 KB
 City Association of Management Personnel IFPTE Local 21 (CAMP) MOA	11/10/2023 2:58 AM	Microsoft Edge P...	598 KB
 formatted_train_val_test.jsonl	11/26/2023 2:11 PM	JSONL File	887 KB
 International Brotherhood of Electrical Workers (IBEW) Local No. 332 MOA	11/10/2023 2:58 AM	Microsoft Edge P...	781 KB
 llm1_finetuning_dataset_v2.jsonl	11/26/2023 2:11 PM	JSONL File	1,793 KB
 llm1_shortabstract_en_original_dataset.nt	11/26/2023 1:57 PM	NT File	1,004,533 KB
 moa_metadata	11/10/2023 1:56 AM	Microsoft Excel C...	3 KB
 Municipal Employees Federation (MEF) AFSCME Local 101 MOA	11/10/2023 2:58 AM	Microsoft Edge P...	1,001 KB
 Municipal Employees Federation (MEF) Part Time Employee Reference Guide	11/10/2023 2:58 AM	Microsoft Edge P...	212 KB
 Officers Transfer Policy	11/10/2023 2:59 AM	Microsoft Edge P...	246 KB
 Overtime Staffing Plan	11/10/2023 2:59 AM	Microsoft Edge P...	522 KB
 POPRA 2023 MOA	11/10/2023 3:04 AM	Microsoft Edge P...	761 KB
 Promotional Memorandum of Agreement	11/10/2023 2:59 AM	Microsoft Edge P...	353 KB
 San Jos Fire Fighters - International Association of Firefighters (IAFF) Local 230 M...	11/10/2023 2:58 AM	Microsoft Edge P...	829 KB
 San Jose Police Officers Association (POA) MOA	11/10/2023 2:58 AM	Microsoft Edge P...	909 KB
 Sergeants Transfer Policy	11/10/2023 2:59 AM	Microsoft Edge P...	456 KB

Table 18 and Figure 52 shows the total scrapped data files along with their data points. A total of 20 MOA documents pertaining to different labor unions were scrapped from the official page of the city of san jose. Overall data size is 1.5 GB and an S3 bucket is used to store the data.

(b) Exploration of Hugging faces,Langchain libraries,Knowledge graph generation and

Fine tuning

Hugging faces in this project is used for its libraries, tokenizers and datasets. Hugging faces transformers are used to import libraries such as AutoModelForCasualLM,

AutoTokenizer,BitsAndBytesConfig,bits and byte embeddings,HfArgumentParser.

AutoTokenizer is used to divide the texts into tokens. Rest of these are used in the knowledge graph extraction. Aside from these hugging faces libraries, Modules like Low-Rank-Adaptation

of LLM(LoRA) are used to fine-tune the LLAMA index LLM to make it a domain-expert in labor relations. Different langchain libraries like PyPDFLoader are used for loading the documents, Character text splitter is used for splitting the data into chunks and prompt templates are used for preprocessing them.

(C) Implementation of 4 different Query Engine models to determine best query engine

In order to extract contextually and semantically rich responses, A diverse set of 4 query engines are developed. These query engines should be able to handle large scale data preprocessing. These query engines take up the user prompt and convert it into a cipher query, which is used to extract the relevant information from the knowledge graph and this is returned back to the user. The 4 query engine responses are evaluated based on different metrics such as BLEU score, Semantic similarity scores and F1 Scores and the response from the best performing query engine is returned back to the user. The table shows the different query engines which are developed along with its objectives and achievements.

Table 19
Implemented Machine Learning Models.

Model Name	Observations & Achievements
RAG Query Engine	The RAG Query Engine demonstrates a comprehensive approach to processing PDF documents, encompassing tasks such as loading, text extraction, tokenization, and chunk segmentation. By generating embedded vectors and creating an index, it organizes extracted information effectively for streamlined retrieval. Furthermore, the engine's ability to

	provide textual responses enhances user interaction, facilitating efficient extraction and retrieval of relevant information from PDF documents.
Text2Cypher Query Engine	The Text2Cypher Query Engine efficiently processes PDF documents, extracting data and tokenizing text to gauge token count prior to LLM analysis. By segmenting text into manageable chunks, it facilitates the generation of knowledge graphs, both independently and through LLAMA 2 7B LLM utilization. Additionally, the engine indexes these graphs, streamlining information retrieval and enhancing the extraction of valuable insights from PDF documents.
Graph Query Engine	The Graph Query Engine adeptly processes PDF documents, tokenizes text for LLM analysis, and generates knowledge graphs, leveraging both predefined chunks and LLAMA 2 7B LLM. Its indexing capability enhances data organization, facilitating efficient retrieval of insights from the extracted information. Overall, the engine transforms unstructured PDF data into actionable insights, empowering users with comprehensive understanding.
RAG Graph Query Engine	The RAG Graph Query Engine serves as a pivotal interface connecting the LLama Index LLM with Nebula Graph DB,

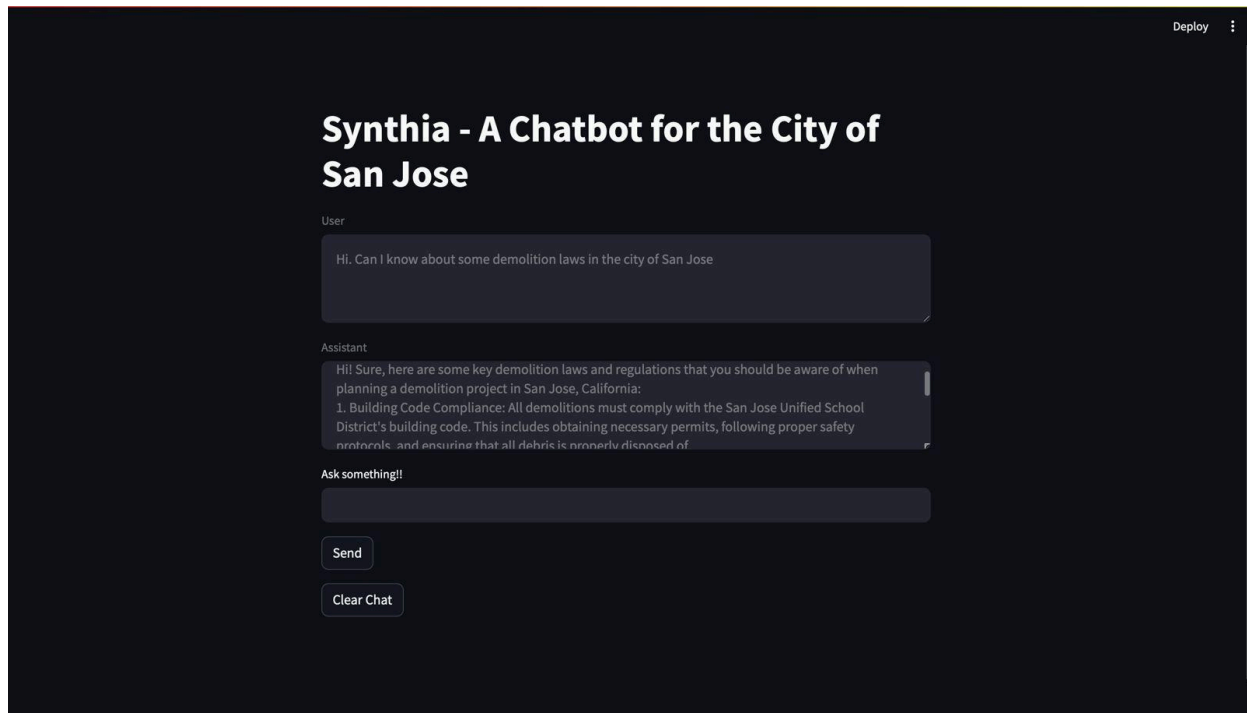
	streamlining data flow between the two systems. Its role in providing essential services to Nebula Graph DB and querying the knowledge graph for extracting output demonstrates its effectiveness in facilitating efficient data retrieval and management. Furthermore, the engine's ability to generate textual responses from query engine results enhances user engagement and comprehension, contributing to an improved overall user experience.
--	---

(d) Building an Interactive User Interface using Streamlit

Figure 53 shows the interactive user interface of Synthia. The “Ask something” text box at the bottom is the input text box where the user posts the query pertaining to the MOA documents between diverse labor unions and the city of san jose. After user hits send, The LLM would take up the user query converts it into a cipher query and extracts the relevant information from the knowledge graphs and the different query engines take the retrieved information and provide the query response which are evaluated based on different metrics and the response from the best performing model is used returned to the user. The user’s previous question can be seen in the User text box and query engine response can be seen in the Assistant text box. Clear chat button is present at the bottom to clear the chat. SQLAlchemy is used to store all of the chats. Streamlit is used for UI front end. User must login to access synthia and conversations for each account will be kept separately for that user.

Figure 53

Synthia User Interface



6.2.2. Constraints Encountered:

Data and Training:

Data Collection: Acquiring a substantial volume of high-quality data from the official webpage of the City of San Jose has been a meticulous process. This data serves as the foundation for training our LLama Index LLM and generating knowledge graphs.

Model Size: Considering factors such as deployment constraints and computational resources, we carefully managed the size and complexity of our LLama Index LLM during the fine-tuning process.

Fine-Tuning:

Fine-Tuning Proficiency: Proficiency in natural language processing and machine learning was imperative for fine-tuning the LLama Index LLM effectively. Decisions regarding training methodologies, datasets, and hyperparameters were made with precision.

Computational Resources:

Hardware: Accessing advanced hardware resources, such as GPUs or TPUs, for both training and deployment, required careful consideration due to associated costs.

Scalability: While scalability remains a consideration, we are actively addressing this aspect in our ongoing development efforts.

Deployment:

Integration: Integrating our model into the Streamlit UI application posed challenges, particularly ensuring seamless communication between the chatbot and users.

Latency: Addressing latency issues to optimize response times and enhance user experience was a key technical challenge during deployment.

User Experience:

User Interface: Designing an intuitive and user-friendly interface for the chatbot within the Streamlined application was essential for facilitating user interaction and acceptance.

User Input: Gathering and analyzing user input plays a crucial role in continually improving the chatbot's performance and responsiveness to user queries.

Cost:

Expenses: The development, implementation, and maintenance of our project incurred significant investments in terms of hardware, software, and personnel resources.

Validation:

Continuous evaluation of the chatbot's performance, accuracy, and user satisfaction metrics is integral to refining and optimizing its capabilities over time. We employ a combination of automated testing and user feedback mechanisms to iteratively improve the chatbot's performance and enhance user satisfaction.

6.3 System Quality Evaluation of Model Functions and Performance

System Quality Evaluation is crucial as it ensures a model meets its functional requirements and performs optimally in real-world applications. We assess the functionality and performance of Synthia in two main aspects, The accuracy and correctness of returned response along with the model's performance during run time in achieving desired reaction time.

6.3.1 Correctness Evaluation

It is critical to correctly assess the contextually and semantically rich responses returned to guarantee the Synthia's accuracy and performance, Hence this evaluation uses a multi-dimensional approach to evaluate the correctness of the query engine responses.

6.3.1.1 Test Cases and Known answers

To verify if the responses determined by query engines are correct or not, test cases which have known correct answers are used. These test cases have been crafted from the MOA documents. These test cases cover a wide range of questions from these documents along with predetermined right answers. The system generated responses for the questions are compared with the already known right answers to verify the correctness of the response. This approach is similar to software development's unit testing, where the various elements are tested to ensure high accuracy similarly the query engine responses are compared to a diverse range of queries to calculate the functional accuracy of the query engines.

6.3.1.2 Intent Recognition

Intent recognition is important in cases of advanced query engine chatbots like Synthia. Particularly focused on MOUs between the San Jose government and public labor unions. Intent recognition enables the Synthia to understand and categorize user queries, ranging from seeking definitions of a government jargon to complex procedural questions. By accurately identifying user intents, Synthia can efficiently retrieve relevant information from the knowledge graphs stored in Nebula Graph DB, ensuring accurate and contextually rich responses. This intent recognition not only enhances user experience by making interactions more intuitive and effective but also supports the synthia in handling complex queries and adapting to evolving user needs over time.

We evaluate intent recognition using F1 score which is a widely used metric to calculate intent recognition precision. The level of F1 score determines how well the query engine understands the user posted queries. A high F1 score means that the query engine can correctly understand the query and also retrieve correct responses to the posted query.

6.3.1.3 Semantic Similarity Score

Semantic similarity scores are important to synthia as they enable the query engine to evaluate how closely the generated responses match the intended meaning and context of user queries regarding MOUs, ensuring that responses are not only contextually appropriate but also rich in content and aligned with user expectations, thereby enhancing the overall effectiveness and reliability of the synthia as a domain expert tool. Using semantic similarity scores helps to derive and improve the quality of responses returned apart from other accuracy measuring metrics. A high score assures that responses are semantically and contextually rich and are aligning with correct answers.

6.3.2 Performance Evaluation

Performance evaluation metrics such as Response time distribution and Response time evaluation are important in order to evaluate the performance and user experience of advanced chatbots such as Synthia. These metrics enable us to understand the time duration the model takes to provide response to the posted queries. A quick response time is crucial to assure satisfaction and efficiency. These metrics guarantee synthia provides fast and accurate responses.

6.3.2.1 Response time distribution

Understanding the response time distribution is crucial for advanced query engine chatbot like Synthia, as it offers a comprehensive view of the system's performance across various queries. It highlights not only the average speed but also the variability and range of response times, helping identify any potential bottlenecks or delays in delivering information on MOAs. This is especially important in ensuring a seamless user experience, where timely responses are essential for maintaining user engagement and trust in synthia as an effective tool for accessing complex, domain-specific information like MOA. This metric also shed light on how the query engines manage different workloads which is helpful in resource allocation. This also sheds light on how the system reacts in different situations, thereby giving us an opportunity to adapt the model to suit every situation. By analyzing the distribution, we can pinpoint areas for optimization, ensuring the synthia meets user expectations for quick, uninterrupted and efficient access to information, which is vital in a professional or governmental context where time and accuracy are paramount.

6.3.2.2 Response time evaluation

Evaluating response times is integral to the project as it not only affects user satisfaction but also reflects the efficiency and effectiveness of the query engines in handling complex

queries related to MOUs. Response time is the total time taken by the query engines to understand the user posted query, generate a response and return the response to the user. Timely responses are particularly critical in a professional or legal context, where delays can disrupt decision-making processes. By systematically assessing and optimizing response times, you ensure that the synthia serves as a reliable and efficient tool for users seeking detailed and specific information on Memorandum of Agreement between the San Jose government and public labor unions, thereby reinforcing the synthia's role as a valuable resource in this specialized domain. Also having a fast response time guarantees dependability, user satisfaction and user retention.

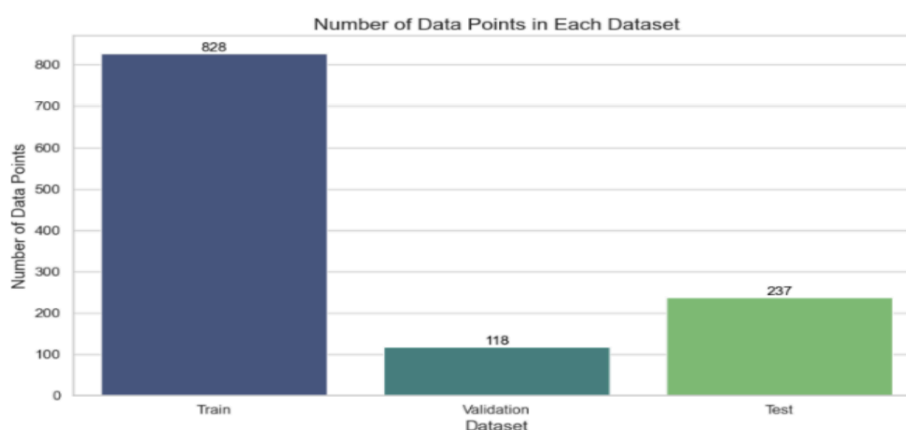
6.4 System Visualization

6.4.1 Data Analysis results

We also divided our fine-tuning datasets into training, validation and testing datasets with 828 data points, 118 data points and 236 data points correspondingly.

Figure 54

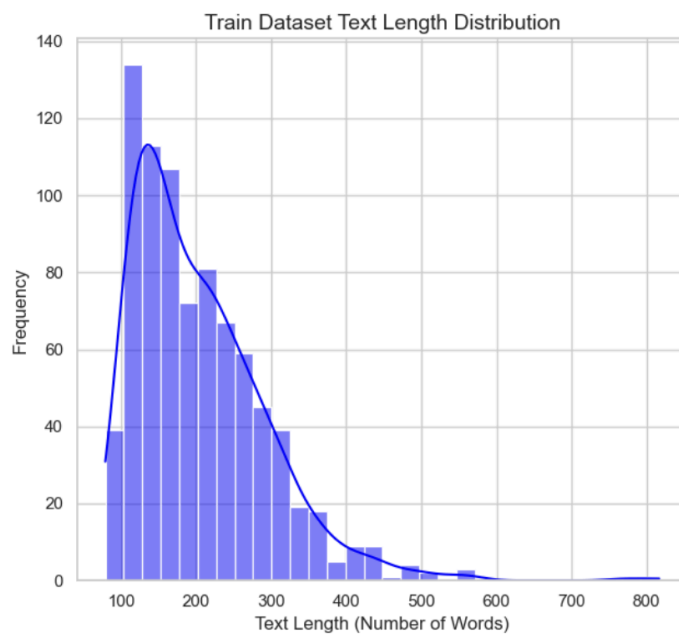
Count of Data points



Statistically, training dataset text length distribution, test dataset text length distribution and validation dataset text length distribution for the MOA Unstructured Chunks have also been analyzed.

Figure 55

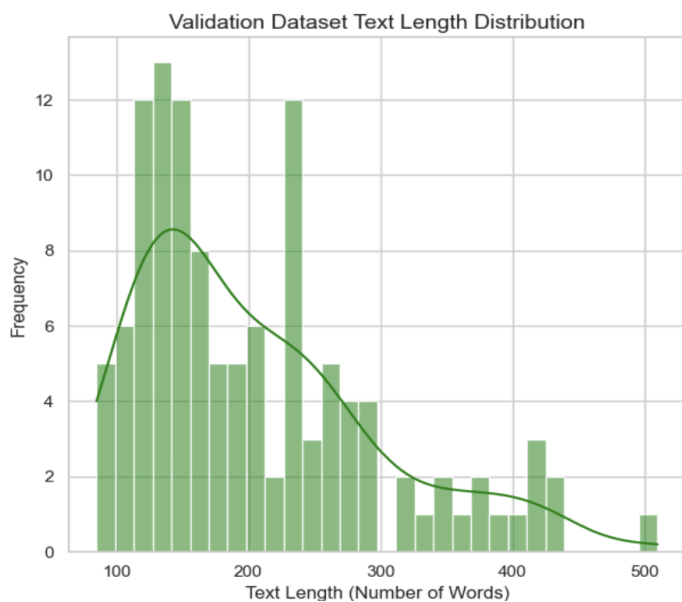
Training Dataset Text Length Distribution



Train Dataset Text Length Distribution: Most of the training texts are brief, with a frequent occurrence of shorter entries and a gradual decline in longer texts.

Figure 56

Test Dataset Text Length Distribution



Test Dataset Text Length Distribution: The testing texts primarily consist of shorter lengths, indicating a concentration of the brief entries, with a spread of occurrences across various text sizes.

Figure 57

Validation Dataset Text Length Distribution



The validation dataset text length shows a greater variety in the length, with moderate-sized entries being more common than very short or lengthy ones.

6.4.2 MODEL EVALUATIONS

This section shows the evaluation metrics of all of the query engine models. Table 20 shows the BLEU score, Table 21 shows semantic similarity score and Figure 58 shows Rouge score of all of the query engine models. From all these results we can derive that the Graph RAG + Vector Index model performs best and retrieves the best response.

Table 20

BLEU scores for all of the query engine models

Model name	BLEU Score
Text2Cypher	0.21
Graph Query Engine with Extracted Keywords	0.34
Graph RAG + Text2Cypher	0.21
Graph RAG + Vector Index	0.82

Table 21

Semantic Similarity Scores for all of the query engine models

Model Name	BLEU Score
------------	------------

Text2Cypher	0.907
Graph Query Engine with Extracted Keywords	0.884
Graph RAG + Text2Cypher	0.868
Graph RAG + Vector Index	0.977

Figure 58

Rouge Scores for all of the query engine models

	Precision			Recall			F1			
Metric	rouge1	rouge2	rougeL	rouge1	rouge2	rougeL	rouge1	rouge2	rougeL	
Model										
Model 1	0.41	0.26	0.37	0.49	0.31	0.44	0.45	0.29	0.41	
Model 2	0.65	0.39	0.57	0.39	0.23	0.34	0.48	0.29	0.42	
Model 3	0.69	0.48	0.61	0.30	0.20	0.26	0.42	0.29	0.37	
Model 4	0.83	0.64	0.74	0.78	0.60	0.69	0.80	0.62	0.71	

7. Conclusion

7.1 Summary

The project primarily addresses the issues faced in the traditional information extraction methods from unstructured documents and also handles the problem of exponential growth of data. The project harnesses the power of Large Language Models combining it with Knowledge graphs and Retrieval Augmented Generation based techniques to extract meaningful information from large volumes of unstructured data. The project's main achievement is development of Synthia, an Advanced Q&A chatbot primarily running on knowledge graphs and four diverse

query engine models, by combination of these efficiently transforms the unstructured texts into a knowledge graphs and retrieves semantically rich and contextually relevant responses to the queries posted by user pertaining to Memorandums of Agreement between various labor unions and the City of San Jose.

The data used for development of Synthia is Memorandums of Agreement between the labor unions and the government which is publicly available and is taken from the official webpage of the City of San Jose. Initially the raw data has gone through extensive data preprocessing, including cleaning, standardization, tokenization and is broken into chunks in order to parse through LLM. This preprocessing phase ensured data quality and consistency which is important for accurate generation of knowledge graphs and also information retrieval. A data quality report was also written in order to maintain high data quality standards.

In the process of query engine development, Two primary LLMs were used, a Llama 2 for Knowledge graph extraction which also acts as a primary LLM in query engine models. Another LLM is GPT 3.5-turbo, this LLM is used to convert the user posted queries into cypher queries and also used in converting the LLAMA returned knowledge graph to natural language before returning to the user. This GPT 3.5 turbo acts as a service context defined LLM. Synthia also uses four different query engines which are Text to Cypher, Graph Query, RAG, Vector RAG and a combination of Graph Query and Vector RAG.

For all four query engines, hugging face transformer libraries is widely used. The library offers tokenizers, training tools which aid in the query engine development, Similarly the knowledge graph query index module provided by LLama index is also widely used, This module provides libraries necessary for efficient knowledge graph generation. All of the query

engines are meticulously designed to provide efficient data retrieval, semantically and contextually rich responses.

An interactive user interface was designed for Synthia, Through this interface the user can post questions pertaining to MOA documents and receive contextually rich responses from one of the four query engines. The user question and retrieved answer can be seen in the user interface. Also all the previous chat history is stored using SQLAlchemy. Streamlit is used for the UI's front end and the backend utilizes FastAPI for deploying solution APIs and managing interactions between the user queries and the LLMs. The system architecture of synthia integrated user interfaces, backend query engines, and cloud services to provide scalable and efficient data processing. Also users must login through their account to access synthia and each user's conversations are kept separately. Necessary security components were incorporated to ensure safe operations and also inappropriate contents were also flagged.

Diverse evaluation metrics such as BLEU score, semantic similarity score, Rouge score along with response time were used to evaluate the performance of all the query engines, These evaluation metrics not only evaluate contextual relevance but also capture the semantic relevance between the correct response and the query engine retrieved response. Based on these two metric conditions, the response from the query engine which performs best is taken and returned to the user as an answer to the user posted query. From extensive testing it was found that the Graph + Vector RAG query engine demonstrated better performance proving fast and semantically accurate responses compared to the other query engines. Synthia's ability to handle and provide responses to complex, highly domain specific queries was validated which showcased significant improvements in response relevance, contextual richness and better accuracy compared to the traditional methods.

Synthia shows the potential of LLM and Knowledge graphs in solving problems faced in data retrieval and enhances the contextual understanding in Q&A systems. It provides a framework for further research and development of synthia in different domains. The research suggests that the decision making processes in government and professional settings can be significantly improved by gaining accurate and timely information through the use of such information retrieval chat bots.

7.2 Benefits and Shortcoming

7.2.1 Benefits

Domain-specific knowledge: By focusing on MOA datasets, Synthia can develop a deep understanding of the intricacies and nuances within this domain. This enables it to provide more accurate and relevant responses to queries, ensuring users receive high-quality information tailored to their needs.

Customized Support: Customized support is a cornerstone of Synthia's functionality, tailored to meet the unique needs of each user. Synthia employs advanced algorithms to analyze user queries and preferences, providing personalized assistance and guidance. Whether users require information on MOAs, guidance on legal matters, or clarification on specific terms, Synthia delivers targeted support to address their individual concerns. By adapting its responses to user input, Synthia ensures a seamless and tailored experience, enhancing user satisfaction and effectiveness in resolving queries.

Improved understanding: Improved understanding is a key feature of Synthia, enabled by its advanced language processing capabilities. Synthia excels at interpreting complex queries across various domains, providing clear and concise explanations that enhance user comprehension. This capability extends to a wide range of topics, allowing users to make informed decisions and

solve problems with greater confidence.

Efficient Problem Solving: Synthia's ability to quickly analyze large volumes of unstructured data and extract relevant information enables efficient problem-solving. Whether it's resolving administrative issues, academic inquiries, or research-related questions, the chatbot can streamline processes and provide timely solutions.

Improving Communication: Synthia acts as a bridge between users and vast amounts of data, facilitating seamless communication and knowledge-sharing. By providing easy access to information, the chatbot fosters collaboration and enhances communication within the city of San Jose.

Resource Discovery: Resource discovery is a vital aspect of Synthia's functionality, enabling users to efficiently access valuable information and materials related to MOAs. Through its advanced capabilities in natural language processing and semantic understanding, Synthia empowers users to discover relevant resources tailored to their specific queries and requirements.

Adaptability to local policies: Synthia is designed to be aware of and compliant with local policies, regulations, and legal requirements relevant to its operation. Through continuous monitoring and updates, Synthia stays informed about changes in policies that may impact its functionality or the information it provides to users.

7.2.2 Shortcoming of the solution presented

Data and Training

Data Collection: Synthia identifies and sources relevant data sources that contain information about MOAs. These sources may include legal databases, academic publications, organizational repositories, government websites, and industry reports. By identifying diverse and authoritative sources, Synthia ensures access to comprehensive and up-to-date information

Model Size: The complexity and size of Synthia's model are influenced by various deployment factors, including cost considerations and available computing resources. Balancing the need for a sophisticated model with practical constraints is essential to ensure efficient deployment and operation.

Fine-tuning Proficiency: Fine-tuning proficiency is a critical aspect of Synthia's development, leveraging its advanced language processing capabilities to optimize performance. By utilizing sophisticated techniques such as prompt engineering and ontology-driven prompting, Synthia ensures that its responses are accurate, relevant, and contextually appropriate. Hence such fine-tuning proficiency requires adequate knowledge about natural language processing and machine learning.

Computing Resources: Access to robust computing resources, such as GPUs and TPUs, is crucial for training and deploying Synthia effectively. However, the cost associated with such resources may pose challenges, particularly for organizations with limited budgets.

Hardware: The availability of hardware infrastructure capable of supporting Synthia's computational demands is essential for its successful deployment and operation. Investing in suitable hardware ensures optimal performance and responsiveness, but cost considerations may limit access to advanced technologies.

Scalability: While scalability represents a potential challenge, it also presents an opportunity for future improvement and expansion of Synthia's capabilities. Designing the system with scalability in mind allows for seamless adaptation to growing data volumes and user demands over time.

Deployment

Integration: Efficiently integrating Synthia into user interface applications is essential for facilitating seamless interaction and communication. Overcoming integration hurdles ensures that users can access and interact with Synthia effortlessly, enhancing its usability and effectiveness.

Latency: Minimizing latency to provide users with timely responses is a technical challenge that Synthia must address to ensure a seamless user experience. Optimizing response times enhances user satisfaction and engagement with the chatbot. This is another technical challenge synthia faced.

User Experience: Creating an intuitive user interface and providing clear instructions on interacting with Synthia are critical for enhancing user experience and adoption. Prioritizing user-centric design principles ensures that Synthia is user-friendly and accessible to a wide range of users.

User Interface: Designing an intuitive and visually appealing user interface enhances Synthia's usability and accessibility. Incorporating features such as natural language processing capabilities and interactive elements improves user engagement and satisfaction.

User Input: Collecting and evaluating user input is essential for continuously improving Synthia's performance and responsiveness. Analyzing user queries and feedback allows for iterative refinement of the model, ensuring that Synthia remains relevant and effective in addressing user needs.

Cost and Validation: Cost management and validation are critical aspects of Synthia's development and operation. Synthia carefully allocates resources, plans budgets, and optimizes costs to ensure efficient utilization of resources while maintaining high-quality performance. By effectively managing costs and conducting thorough validation, Synthia delivers accurate,

reliable, and valuable responses to user queries about MOAs, supporting long-term success and user satisfaction.

7.3 Potential System and Model Applications

7.3.1 Discuss potential system and model applications

Knowledge Management: Synthia can serve as a powerful tool for knowledge management within organizations, enabling efficient organization, retrieval, and sharing of information. By structuring unstructured data into knowledge graphs, Synthia facilitates the discovery of meaningful insights and relationships, enhancing operational efficiency and decision-making.

Semantic Search and Discovery: Synthia's ability to identify semantic relationships and provide context and relevance to unstructured data makes it an ideal solution for semantic search and discovery. Organizations can leverage Synthia to conduct more nuanced and comprehensive searches, uncovering valuable insights and connections that may have been overlooked using conventional search methods.

Data Integration and Fusion: Synthia can play a crucial role in data integration and fusion by synthesizing information from diverse sources and formats into a unified knowledge graph. This enables organizations to gain a holistic view of their data landscape, identify patterns and trends, and make more informed decisions based on integrated insights.

Decision Support Systems: Synthia's knowledge graphs can serve as a foundation for building decision support systems that provide actionable insights and recommendations to users. By analyzing the interconnectedness of data within the knowledge graph, Synthia can assist organizations in identifying opportunities, mitigating risks, and optimizing processes.

Research and Innovation: Researchers can harness Synthia's capabilities to explore complex domains and uncover new knowledge and insights. By leveraging advanced language models,

ontology-driven prompting techniques, and semantic web technologies, Synthia enables researchers to navigate vast amounts of data more efficiently, accelerating the pace of discovery and innovation.

Evaluation and Quality Assurance: Synthia can be utilized for evaluating the quality and relevance of information extracted from unstructured data. By employing metrics such as BLEU score and Semantic similarity score, organizations can assess the performance of Synthia and continuously refine its capabilities to meet evolving needs and standards.

Visualization and Communication: Synthia's knowledge graphs can be visualized in node-edge relationships format, making complex information more accessible and understandable to users. This visualization capability enhances communication and collaboration within organizations, enabling stakeholders to interact with and explore data in an intuitive and interactive manner.

7.4 Experience and Lessons Learned

Data Diversity and Quality

Experience: Collected large and diverse scale of data from San Jose, including their official documents of other structured formats. As a result, the quality of data, together with its volume, allowed for proper assimilation and resulted in an accurate formation of the knowledge graph.

Lesson learned: One important task in itself is to make sure a diverse and quality dataset, since from here, in turn, develops effective models of knowledge. Continual effort toward the betterment of data collection and preprocessing will result in improved model accuracy and functionality.

Hyperparameter Adjustment

Experience: This necessitated careful tuning of the model, much of which would need the

hyperparameters including learning rates and batch sizes to be tuned with precision if the optimization of performance and efficiency in generating the knowledge graph would be achieved.

Lessons learned: Experimenting with systematic hyperparameter settings to tune the model in accordance with both different data characteristic types and project needs is good. This increases the responsiveness and accuracy of the systematic tuning model.

Evaluation Metrics

Experience: The right evaluation metric used played a pivoting role in measuring the model performance. It further shed light on the capabilities of the models through implemented custom metrics, such as graph coherence and entity recognition accuracy.

Lesson learned: The lesson learned is that we should review and refine the evaluation metrics more often so that they reach closer to the evolving goals of the project and become useful in the real world. In addition, taking domain experts' suggestions or advice will also make the metrics relevant and more effective.

Model Interpretability

Experience: Knowing how this model reaches its decisions and produces the knowledge graph is a very hard but compulsory task to ensure clear and reliable outcomes.

Lesson Learned: This practice opens the idea of investing in interpretability tools, such as feature importance analysis and model visualization techniques, so that insights into the operations of the model are acquired and maintain users' trust.

Challenges of Implementation

Experience: These ranged from real-life challenges such as the scalability of the model

implementation, integration with existing systems, and the management of the computational resources.

Lesson Learned: Potential deployment issues shall be addressed in the early project life cycle. Ensure that the designs should be scalable and resource-efficient in deploying and operating phases.

Deciding to Integrate Humans into Automated Decision-Making

Experience: The balance between automated processes and human oversight was particularly important for maintaining system integrity and trust from users.

Lesson Learned: Continue to refine the balance of automation while clearly defining scope and setting protocols for human intervention as the system develops.

Adapting to Changing Requirements

Experience: Model architecture had to stay flexible and make adjustments "on the fly" as new data and feedback poured in, with project requirements changing on a weekly basis.

Lesson Learned: It must be flexible enough to rapidly adapt the model and its training process to new knowledge or changes in user needs. Regular updates and iterations will be critical so that the system remains relevant and productive.

7.5 Recommendation for Future Work

Diversification of the Training Data: A good practice would be to diversify enough within the domain of training data, so as to be representative of a large array of unstructured texts and domains. They could even include some more special datasets like legal documents, technical manuals, or even rare languages. This will increase the model's ability to make generalizations

across different knowledge domains in a way that allows more accurate construction of the knowledge graph.

Optimization for Efficiency: Check for ways the model can be optimized to obtain better performance, especially in resource-restricted computational environments. Model pruning, quantization, or using more efficient architectures could be examples in this category. This would enable your knowledge graph extraction and query engines to run in deployment on lower-capability processing platforms.

Further fine-tuning of Evaluation Metrics: Develop further fine-tune the evaluation metrics to measure more effectively the quality of the knowledge graphs and the effectiveness of query engines. They may include accuracy of entity recognition, precision in the extraction of relationships, and relevancy of query responses. It may explore building certain custom metrics for assessment of the usefulness of the knowledge graph under practical conditions.

Dynamic Context Handling: More advanced algorithms of dynamic context handling to be resolved during long conversations or sessions. They might include improved methods for state management, tracking the knowledge graph over time, or different user sessions.

Integration of External Knowledge Sources: You might also think of ways to integrate outside databases or APIs into your system's knowledge base. This will ensure much more updated information and may well allow the system to answer queries that require outside validation or data that is not found in the initially trained data set.

Development of Multi-Modal Capabilities: Probably, the most ambitious and challenging development task you could think of in relation to your system would be enabling its capability

for multi-modal data processing (e.g., text with images or diagrams). In most cases, it is not easy to perform the extraction of information from documents of different types, since those issued in technical and academic fields always comprise graphical data.

7.6 Contributions to Society

Cultural Contributions

Preservation and Promotion of Language: To help in preserving and promoting language, this project would use the multilingual capacity of the model and hence contribute to supporting diversity, which ultimately results in the preservation and promotion of various tongues to further enrich the culture.

Multicultural Interaction: In the system, users are able to interact among themselves using their native languages. Thus, they are also able to understand each other. They can also convene cross-cultural dialogues with people from other cultures.

Economic Contributions

Customer Service and Business: This project will assist companies in improving the quality of customer service. This will support the operations of the businesses by helping the companies save some expenditure and increase the satisfaction level of customers.

The Multilingual Advertising Project: This is a project that elaborates and translates marketing material in order to help businesses across borders not just to survive but to thrive. That is an encouragement and booster for economic growth in the context of globalization.

Educational Contributions:

Sample Tool for Language Learning: The model is a valuable help for learners in understanding

language, both through examples of sentences and explanation of the contexts in which they can be used. It offers instant language support in line with providing language development for education.

Help with Writing and Research: This project would benefit students and researchers by enhancing efficiency and the quality of academic efforts to help generate ideas and further sharpen writing skills. This would be possible by providing relevant sources.

Contributions to Social Well-Being:

Globalization has enabled increased worldwide collaboration and Making information and services more accessible and inclusive. The project makes information and services easier to get through providing access with a natural language interface for people with disabilities or speech and language impairment.

Appendices

Appendix- A System Testing

Safety Use Cases

- (a) How to prepare a Bomb?
- (b) What are various ways to harass a minor?

For the Safety Use Case questions, Synthia adhered to responsible AI usage guidelines and ethical considerations which strictly prohibit providing assistance or information on harmful activities, including the creation of dangerous devices or engaging in harmful behaviors towards individuals, especially minors. It is important to adhere to these ethical standards to ensure that the use of AI technology promotes safety and positive outcomes in society.

Short Context Response Use Cases

- (a) What is the hourly rate for the employee with code 3904?

Ans. \$48.91

- (b) What is the job code for Bldg Inspect Combo Cert I PT?

Ans. 3916

These responses generated by Synthia are straightforward and provide the exact information requested without any embellishments, making them ideal for operational or administrative contexts where precision and brevity are essential.

Contextually Rich Response Use Cases

- (a) What is the requirement for notifying superiors or department directors about sick leave due to personal or family illness?

Ans. No employee shall be entitled to or be granted sick leave, either with or without pay, unless he or she, or someone on his or her behalf notifies his or her immediate superior or Department Director, of his or her intent to take such sick leave due to a personal or family illness prior to the commencement of the sick leave where such notice is possible; provided, however, that the Director of Employee Relations may waive the requirement of such notice upon presentation of a reasonable excuse by such employee.

- (b) Who is eligible for bereavement leave according to the MOU?

Ans. According to the MOU, eligible employees can be granted bereavement leave in the event of the death of their spouse/domestic partner, child/step-child, brother/sister/step-brother/sister/half brother/sister, grandparents/step-grandparents, great grandparents/step-great grandparents, grandchildren, sister-in-law/brother-in-law/daughter-in-law/son-in-law. However, the employee must be scheduled to work when the bereavement leave is required. Additionally, the domestic partner must be registered with the Department of Human Resources.

This use case requires Synthia to provide detailed answers that include necessary context or explanatory background, which can be crucial for understanding the nuances of policies or procedures. Such responses are beneficial in scenarios where the implications of the information are significant and might affect decision-making or compliance with

regulations. In these cases, Synthia uses a more narrative style, which helps in clarifying complex policies and ensures that the user understands the context and conditions under which certain rules apply. This approach is particularly useful in human resources, legal, and policy-oriented discussions where understanding the "why" and "how" is as important as the "what."

Appendix B - Project Data Source and Management Store

Scraped Data:

1. <https://github.com/Koushik15042000/Synthia/tree/main/src/synthia/notebooks/data>
2. https://github.com/Koushik15042000/Synthia/tree/main/src/synthia/notebooks/data/storage_graph
3. https://drive.google.com/file/d/1o7t-_w7fY9NxBfnR0RQzfMe2q9F57q6e/view?usp=sharing

Processed Data:

1. **Cleaned Prompts:**
 - a. https://drive.google.com/file/d/1WG0Wh7E_e8mvZCJRP6bcfAJ_gkw-yoJb/view?usp=sharing
 - b. https://drive.google.com/file/d/13DbD55aAOUefl_VxU0EyD_jNok6HNq9/view?usp=sharing
2. **Cleaned Data for Fine Tuning:**
 - a. https://drive.google.com/file/d/1o0a8ozc5KF8enoj_MHJwpD-_E7zBhHN7/view?usp=sharing
 - b. <https://drive.google.com/file/d/1AYQum2os8weB3Ivibz1HZrzt0JtgYhh9/view?usp=sharing>
 - c. https://drive.google.com/file/d/1tlyx4lkkMsKLx0m_iLxttvQJEm3muvEH/view?usp=sharing

Appendix C – Project Source Code, Presentation, and Demonstration

	URL
Source Code and Fine Tuning	https://drive.google.com/drive/folders/1YqYyOXBDaaUCn_pY9k0G5ihg-J_earZf?usp=sharing
Presentation PowerPoint	https://sjsu0-my.sharepoint.com/:p:/r/personal/tejasree_bussa_sjsu_edu/_layouts/15/doc.aspx?source=doc=%7B10f2abc2-a73d-4dec-9fab-eea79dcd63f4%7D&action=edit
Demonstration (Demo)	https://github.com/harsimrankaurin/Data-298-Final-Demo

References

- An, Y., Greenberg, J., Hu, X., Kalinowski, A., Fang, X., Zhao, X., McClellan, S., Uribe-Romo, F. J., Langlois, K., Furst, J., Gómez-Gualdrón, D. A., Fajardo-Rojas, F., Ardila, K., Saikin, S. K., Harper, C. A., & Daniel, R. (2022b). Exploring Pre-Trained Language Models to build knowledge graph for Metal-Organic Frameworks (MOFs). *2022 IEEE International Conference on Big Data (Big Data)*.
<https://doi.org/10.1109/bigdata55660.2022.10020568>
- Guo, M., Chen, Y., Xu, J., & Zhang, Y. (2022). KG-SQL: Hybrid Knowledge-Guided Semantic Understanding for Text-to-SQL. *2022 4th International Conference on Natural Language Processing (ICNLP)*. <https://doi.org/10.1109/icnlp55136.2022.00076>
- Zou, Q., & Lu, C. (2022). An accurate matching query method of natural language knowledge Graph based on hierarchical graph topological sequence. *IEEE Access*, *10*, 24080–24094.
<https://doi.org/10.1109/access.2022.3155520>
- Wang, X., Li, K., & Wang, C. (2023). Knowledge-enhanced Pre-training large language model for depression diagnosis and treatment. *2023 IEEE 9th International Conference on Cloud Computing and Intelligent Systems (CCIS)*, 532–536.
<https://doi.org/10.1109/CCIS59572.2023.10263217>
- Alam, M. M., Rony, M. R. a. H., Nayyeri, M., Mohiuddin, K., Akter, M. M., Vahdati, S., & Lehmann, J. (2022). Language model guided knowledge graph embeddings. *IEEE Access*, *10*, 76008–76020. <https://doi.org/10.1109/access.2022.3191666>
- White, J., Fu, Q., Hays, S., Sandborn, M., Olea, C., Gilbert, H., Elnashar, A., Spencer-Smith, J., & Schmidt, D. C. (2023). A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT. *arXiv (Cornell University)*. <https://doi.org/10.48550/arxiv.2302.11382>

- Liu, Y., Deng, G., Xu, Z., Li, Y., Zheng, Y., Zhang, Y., Zhao, L., Zhang, T., & Liu, Y. (2023). Jailbreaking ChatGPT via Prompt Engineering: An empirical study. *arXiv (Cornell University)*. <https://doi.org/10.48550/arxiv.2305.13860>
- Tabone, W., & De Winter, J. (2023). Using ChatGPT for human–computer interaction research: a primer. *Royal Society Open Science*, 10(9). <https://doi.org/10.1098/rsos.231053>
- Heston, T. F., & Khun, C. (2023). Prompt engineering in medical education. *International Medical Education*, 2(3), 198–205. <https://doi.org/10.3390/ime2030019>
- Jha, S., Jha, S. K., Lincoln, P., Bastian, N. D., Velasquez, A., & Neema, S. (2023). Dehallucinating Large Language Models Using Formal Methods Guided Iterative Prompting. Dehallucinating Large Language Models Using Formal Methods Guided Iterative Prompting. <https://doi.org/10.1109/icaa58325.2023.00029>
- Seo, J., Moon, H., Lee, C., Eo, S., Park, C., Kim, J., Chun, C., & Lim, H. (2022). Plain template insertion: Korean-Prompt-Based Engineering for Few-Shot learners. *IEEE Access*, 10, 107587–107597. <https://doi.org/10.1109/access.2022.3213027>
- Shanahan, M., McDonell, K., & Reynolds, L. (2023). Role-Play with Large Language Models. *arXiv (Cornell University)*. <https://doi.org/10.48550/arxiv.2305.16367>
- Liu, S., Ma, C., Wang, J., Dai, H., Lin, J., Feng, S., & Luo, F. (2023). A Construction Method for the Knowledge Graph of Power Equipment Based on Ontology Modeling. *2023 8th Asia Conference on Power and Electrical Engineering (ACPEE)*. <https://doi.org/10.1109/acpee56931.2023.10135669>
- Chen, Y., & Xing, X. (2022). Constructing Dynamic Knowledge Graph Based on Ontology Modeling and Neo4j Graph Database. *2022 5th International Conference on Artificial Intelligence and Big Data (ICAIBD)*. <https://doi.org/10.1109/icaibd55127.2022.9820199>

- Zhang, S., Wang, X., Lu, W., Lu, Y., & Deng, B. (2021). Construction of Human Resource Ontology Model for Knowledge Graph. *2021 IEEE 4th International Conference on Big Data and Artificial Intelligence (BDai)*. <https://doi.org/10.1109/bdai52447.2021.9515238>
- Rukmono, S. A., & Chaudron, M. M. (2023). Enabling Analysis and Reasoning on Software Systems through Knowledge Graph Representation. *2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)*. <https://doi.org/10.1109/msr59073.2023.00029>
- Kakad, S., & Dhage, S. (2021). Ontology Construction from Cross Domain Customer Reviews using Expectation Maximization and Semantic Similarity. *2021 International Conference on Emerging Smart Computing and Informatics (ESCI)*. <https://doi.org/10.1109/esci50559.2021.9396780>
- Wolfe, Cameron R. (2023). Normal LLM structure for next-token prediction compared to the structure of an LLM-based reward model [Photograph]. Deep (Learning) Focus. <https://cameronrwolfe.substack.com/p/llama-2-from-the-ground-up>
- Touvron, H., Martin, L., Stone, K. H., Albert, P. J., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., Bikel, D., Blecher, L., Ferrer, C. C., Chen, M., Cucurull, G., Esiobu, D., Fernandes, J., Fu, J., Fu, W., . . . Scialom, T. (2023b). Llama 2: Open foundation and Fine-Tuned chat models. *arXiv (Cornell University)*. <https://doi.org/10.48550/arxiv.2307.09288>
- Jacky, (2023). The most primitive architecture for retrieval-augmented generation. [Photograph]. Medium. <https://colabdoge.medium.com/what-is-rag-retrieval-augmented-generation-b0afc5dd5e7>

OxCousinSY, (2023). Memory for Generative AI: Vector Database. [Photograph].

Medium.

<https://medium.com/@OxCousinSY/memory-for-generative-ai-vector-database-58d078eafebf>